



Strathmore University

@iLabAfrica Centre

Applied Machine Learning

From First Principles to Production Systems

Javier Serrano
Emmanuel Lee Bundi

Master in Data Science and Analytics
DSA 8401

Nairobi, 2026

Applied Machine Learning: From First Principles to Production Systems in the African Fintech Ecosystem

Javier Serrano & Emmanuel Lee Bundi
iLabAfrica — Strathmore University, Nairobi, Kenya

Lecture notes for DSA 8401 — Applied Machine Learning,
Master in Data Science and Analytics.

© 2026 The authors. All rights reserved.
Draft edition — for enrolled students. Not for redistribution.

Typeset with L^AT_EX using the book class.
Figures produced with TikZ; code listings with listings.

Contact: jgarcia@strathmore.edu

Version 0.1 — September 7, 2026

Contents

Preface	xi
I Foundations and Classical Machine Learning	1
1 Introduction to Machine Learning	3
1.1 Why This Chapter Comes First	4
1.2 What Is Machine Learning?	5
1.2.1 An operational definition	5
1.2.2 ML inverts the direction of programming	6
1.2.3 When ML earns its keep — and when it does not . . .	7
1.2.4 The formal core: empirical risk minimization	7
1.2.5 The Bayes-optimal reference	10
1.2.6 Notation for the rest of the book	11
1.3 The Machine-Learning Landscape	11
1.3.1 The four learning paradigms	11
1.3.2 Two orthogonal operational axes	13
1.3.3 No free lunch, and what it licenses	15
1.4 Why ML Systems Fail in Production	16
1.4.1 The small green box	16
1.4.2 The notebook-to-production gap, localized	17
1.5 The End-to-End ML Lifecycle	18
1.5.1 From CRISP-DM to a monitoring-driven loop	18
1.5.2 It is a loop, not a waterfall	19
1.6 Framing the Problem and Choosing the Metric	20
1.6.1 Step 1 is where projects are won	20
1.6.2 Regression metrics: how the business pays for errors	21
1.6.3 Classification under imbalance: why accuracy lies . .	21
1.7 A First Look at the Data	24
1.7.1 The raw material	24
1.7.2 A feature dictionary, declared up front	24
1.7.3 What to interrogate, and why	25
1.8 Data, Splits, and Leakage	27

1.8.1	Splits: the discipline that protects your estimate of $R(f)$	27
1.8.2	Data leakage: the silent killer of fintech models	28
1.9	Baselines, Evaluation, and Generalization	30
1.9.1	Why generalization is hard: the bias–variance decomposition	30
1.9.2	Always ship a baseline first	31
1.9.3	Cross-validation: cheap insurance on the estimate . .	31
1.10	Presenting and Deciding	33
1.10.1	The decision memo	33
1.10.2	Champion, challenger, and shadow mode	33
1.11	Fairness, Explainability, and Regulation	34
1.11.1	Making “fair” precise enough to test	34
1.11.2	Explainability as a deployment requirement	35
1.12	From Model to System: MLOps and the Agentic Outlook . .	35
1.12.1	The MLOps spine of this course	36
1.12.2	Where LLMs and agents fit — and where they do not	37
1.13	Chapter Summary	38
	Exercises	41
	Key Terms	42
	Further Reading	42
2	Data Preparation and Feature Engineering	45
2.1	Introduction: From Raw Data to Model-Ready Data	46
2.1.1	The Six Dimensions of Data Quality	47
2.1.2	The Unit of Prediction	48
2.2	Missing Data	48
2.2.1	Rubin’s Taxonomy of Missingness	49
2.2.2	Diagnosis Before Treatment	50
2.2.3	Imputation I: Simple Statistics with Indicators	50
2.2.4	Imputation II: Model-Based Methods	51
2.2.5	A Decision Framework	52
2.3	Outliers and Duplicates	53
2.3.1	Detecting Outliers	53
2.3.2	Treating Outliers, and When Not To	54
2.3.3	Duplicates and Entity Resolution	54
2.4	Scaling and Transformations	55
2.4.1	Why Scale, and With What	55
2.4.2	Taming Skew: Log and Power Transforms	56
2.5	Encoding Categorical Variables	57
2.5.1	Low Cardinality: One-Hot and Ordinal	57
2.5.2	High Cardinality I: Frequency Encoding and the Hashing Trick	58
2.5.3	High Cardinality II: Target Encoding	58
2.6	Feature Engineering	59

2.6.1	Engineered versus Learned Features	59
2.6.2	Domain Features for Mobile-Money Credit Scoring	60
2.6.3	Datetime Features and Cyclical Encoding	61
2.6.4	Interactions, Binning, and Crosses	61
2.6.5	What Makes a Feature Good?	62
2.7	Data Leakage	63
2.7.1	Definition and Why It Is Dangerous	63
2.7.2	Six Common Causes	64
2.7.3	Temporal and Group-Aware Splits	64
2.7.4	A Detection Protocol	65
2.7.5	The Structural Antidote: Pipelines	66
2.8	From Pipelines to Feature Platforms	67
	Assignment 2 (graded, 20 marks)	68
	Exercises	69
	Key Terms	70
	Further Reading	70
3	Classical Machine Learning and Rigorous Evaluation	71
3.1	Introduction	72
3.2	The Classical Toolbox	73
3.2.1	Why classical models still run production tabular ML	73
3.2.2	Logistic regression	73
3.2.3	Softmax regression for multiclass problems	75
3.2.4	Maximum-margin classification: support vector machines	75
3.2.5	Neighbours and Bayes: two more baselines	76
3.2.6	Choosing among the classical learners	77
3.3	Beyond Accuracy: Metrics that Match the Problem	78
3.3.1	The accuracy trap	78
3.3.2	The confusion matrix	78
3.3.3	Precision, recall, and the F_β family	79
3.3.4	From metrics to money: cost-sensitive evaluation	79
3.4	Ranking Quality: ROC and Precision–Recall Analysis	80
3.4.1	Scores, not labels: the threshold as a free parameter	80
3.4.2	The ROC curve and AUC	81
3.4.3	Precision–recall curves: the imbalance-honest view	82
3.5	Calibration: Do the Probabilities Mean Anything?	82
3.6	Cross-Validation and Honest Uncertainty	84
3.6.1	Why one split is not enough	84
3.6.2	Cross-validating the pipeline	85
3.6.3	Nested cross-validation for model selection	86
3.6.4	Comparing models honestly	86
3.7	Learning from Imbalanced Data	87
3.7.1	Three families of remedies	87

3.7.2	SMOTE: synthesising minority examples	88
3.7.3	The cardinal sin: resampling before splitting	89
3.8	Case Study: Evaluation Design for the Thin-File Credit Scorer	90
3.9	Chapter Summary	91
4	Decision Trees, Ensembles and Hyperparameter Tuning	95
4.1	Introduction	96
4.2	Decision Trees: The Building Block	98
4.2.1	A tree is a sequence of questions	98
4.2.2	The same object, drawn as a partition	98
4.2.3	How a tree is grown	99
4.2.4	Controlling complexity: stopping and pruning	100
4.2.5	The flaw that motivates the rest of the chapter	101
4.3	Why Ensembles Work	102
4.3.1	The wisdom of independent crowds	102
4.3.2	The variance of an average	103
4.4	Voting Classifiers	104
4.5	Bagging and Random Forests	106
4.5.1	Bootstrap aggregating	106
4.5.2	Out-of-bag evaluation	106
4.5.3	Random Forests: decorrelating the trees	107
4.6	Boosting	109
4.6.1	The idea, before the algebra	109
4.6.2	A worked trace	110
4.6.3	From residuals to gradients	111
4.6.4	AdaBoost, the historical special case	112
4.6.5	Shrinkage and early stopping	113
4.7	Regularised Gradient Boosting: XGBoost, LightGBM, CatBoost	113
4.7.1	Regularisation inside the objective	114
4.7.2	The library landscape	115
4.7.3	Monotonic constraints and regulated deployment	115
4.8	Stacked Generalisation	117
4.9	Hyperparameter Optimisation	118
4.9.1	What is a hyperparameter, and why is it different?	118
4.9.2	A ladder of strategies	119
4.9.3	Grid search versus random search	119
4.9.4	Learning from previous trials: Bayesian search and TPE	121
4.9.5	Spending less per trial: successive halving and Hyperband	122
4.9.6	Designing the search space	123
4.9.7	Honest evaluation of a tuned model	123
4.10	Case Study: Round Three of the Credit Scorer	125
	Exercises	127
	Key Terms	128

Further Reading	129
5 Unsupervised Learning and Dimensionality Reduction	131
5.1 Introduction: Learning Without Labels	132
5.2 The Unsupervised Landscape	134
5.3 Clustering with K -Means	136
5.3.1 The objective	136
5.3.2 Choosing k	137
5.3.3 Implementation	139
5.4 Density-Based Clustering: DBSCAN	140
5.5 Gaussian Mixtures and EM	142
5.5.1 The model	142
5.5.2 Fitting by Expectation–Maximisation	143
5.5.3 Selecting k by information criteria	145
5.5.4 Mixtures as anomaly detectors	146
5.6 The Curse of Dimensionality	147
5.7 Principal Component Analysis	148
5.7.1 Variance maximisation	148
5.7.2 Reconstruction-error minimisation	149
5.7.3 Computation and the choice of d	149
5.7.4 Variants you will actually need	151
5.8 Case Study: The Segmentation Pipeline in Production	152
5.8.1 Profiling: turning clusters into personas	152
5.8.2 Stability: a segment that moves is not a segment	153
Exercises	156
Key Terms	157
Further Reading	158
 II Deep Learning and Generative AI	 159
6 Deep Learning Fundamentals	161
6.1 Introduction: from engineered features to learned representations	162
6.1.1 When a network is the right instrument	164
6.2 The artificial neuron and the multi-layer perceptron	165
6.3 Activation functions	168
6.4 The training objective	170
6.4.1 Binary cross-entropy from the Bernoulli likelihood	171
6.4.2 Choosing the output layer and loss together	172
6.5 Gradient descent	173
6.5.1 Why the learning rate dominates everything else	174
6.6 Backpropagation	174
6.7 Optimisers beyond plain gradient descent	177

6.7.1	Momentum	177
6.7.2	Adam	178
6.7.3	Learning-rate schedules	179
6.8	Pathologies of deep training	179
6.9	Initialisation	180
6.10	Normalisation layers	181
6.11	Regularisation	182
6.11.1	Dropout	182
6.11.2	Early stopping and weight decay	184
6.12	Input scaling and conditioning	184
6.13	Entity embeddings	185
6.14	Implementation in TensorFlow and Keras	187
6.14.1	The training loop, written out	187
6.14.2	The baseline network	188
6.14.3	The embedding challenger	189
6.14.4	Hyperparameter search and debugging	190
6.15	Evaluation, calibration and governance	190
6.15.1	Ranking versus calibration	191
6.15.2	Explanation	191
6.15.3	Fairness and segment-level performance	192
	Exercises	194
	Key Terms	197
	Further Reading	198
7	Computer Vision with CNNs	199
7.1	Introduction	200
7.1.1	Four tasks, one backbone	202
7.2	Why Convolution?	203
7.2.1	Why a dense network fails on images	203
7.2.2	The convolution operator	205
7.2.3	Pooling and global average pooling	209
7.2.4	Receptive fields and the case for depth	210
7.2.5	The data-efficiency argument	212
7.3	Modern Architectures: ResNet and EfficientNet	212
7.3.1	Residual networks	213
7.3.2	EfficientNet and compound scaling	216
7.3.3	Choosing a backbone	218
7.4	Transfer Learning	218
7.4.1	Two regimes	219
7.4.2	The two-phase recipe	221
7.4.3	Implementation in TensorFlow and Keras	222
7.4.4	The batch-normalisation trap	224
7.4.5	Data augmentation as a statement about your domain	225
7.4.6	Reading the training curves	227

7.5	Evaluation, Explainability and Deployment	227
7.5.1	Evaluation under imbalance and asymmetric cost	227
7.5.2	Explaining a convolutional prediction	229
7.5.3	Fairness, governance and deployment	230
	Exercises	234
	Key Terms	236
	Further Reading	236
8	Sequence Models and Time Series	239
8.1	Introduction	239
8.2	Core Theory	239
8.3	Implementation	239
9	Transformers, Large Language Models and RAG	241
9.1	Introduction	241
9.2	Core Theory	241
9.3	Implementation	241
III	MLOps and Production Systems	243
10	MLOps: Packaging, Serving, Tracking, CI/CD	245
10.1	Introduction	245
10.2	Core Theory	245
10.3	Implementation	245
11	Model Monitoring, Drift and ML System Design	247
11.1	Introduction	247
11.2	Core Theory	247
11.3	Implementation	247
	Appendices	251
A	Capstone Project	251
B	Solutions to Exercises	253
	Chapter 1	253
	Chapter 2	260
	Chapter 3	265
	Chapter 7 — Computer Vision with CNNs	290

Preface

This book collects the lecture notes of **DSA 8401 — Applied Machine Learning**, taught in the Master in Data Science and Analytics at iLabAfrica, Strathmore University. It is written for graduate students who already program and know basic statistics, and who want to go beyond training models in a notebook: the goal throughout is *production-grade* machine learning — data pipelines, rigorous evaluation, deployment, and monitoring.

Three commitments shape every chapter:

1. **An end-to-end view.** Models are treated as components of systems. MLOps (Docker, FastAPI, MLflow, CI/CD) is not an afterthought but the destination of the whole arc.
2. **African context as pedagogy.** Worked examples are anchored in the African technology and financial ecosystem: thin-file mobile-money credit scoring (the running thread of the course), trade-corridor underwriting, clinical pathway modelling, and streaming fraud detection. This is a deliberate instructional choice, not decoration.
3. **Modern architectures.** Transformers, LLMs, RAG pipelines, and an outlook on agentic systems are integrated where they matter for applied work.

How this book maps to the course

Each chapter corresponds to one teaching week (three hours: a 1.5-hour lecture plus a 1.5-hour practical lab). The two MLOps weeks are consolidated into a single chapter, and the capstone project brief lives in Appendix [A](#).

Chapter	Topic	Course week
1	End-to-End Machine Learning	1
2	Data Preparation & Feature Engineering	2
3	Classical ML & Rigorous Evaluation	3
4	Ensemble Methods & Hyperparameter Tuning	4
5	Unsupervised Learning & Dim. Reduction	5
6	Deep Learning Fundamentals	6
7	Computer Vision with CNNs	7
8	Sequence Models & Time Series	9
9	Transformers, LLMs & RAG	10
10	MLOps: Serving, Tracking & CI/CD	11–12
11	Monitoring, Drift & System Design	13
A	Capstone Project	14–15

Prerequisites and companion material

We assume the Data Mining unit, working Python, and comfort with linear algebra and probability at the level of a first graduate course. The primary textbook companion is [Géron \(2022\)](#); [Bishop and Bishop \(2024\)](#) and [Huyen \(2022\)](#) support the deep-learning and MLOps parts respectively. Each chapter closes with a lab section that mirrors the Jupyter notebook used in the practical session.

Javier Serrano & Emmanuel Lee Bundi
Nairobi, 2026

Part I

Foundations and Classical Machine Learning

Part II

**Deep Learning and Generative
AI**

CHAPTER 7

Computer Vision with Convolutional Neural Networks

Course week 7

Learning Objectives

After working through this chapter you will be able to:

- Explain, in terms of both parameter count and the absence of a locality prior, why a fully connected network is the wrong architecture for images, and state the three inductive biases that convolution supplies in its place.
- Compute feature-map shapes, parameter counts and receptive fields for an arbitrary convolutional stack, and justify the preference for deep stacks of small kernels over shallow stacks of large ones.
- Explain residual learning as a fix for an optimisation failure rather than a representational one, and compound scaling as a budgeting rule rather than a heuristic.
- Apply transfer learning correctly: frozen-backbone feature extraction followed by staged fine-tuning at a reduced learning rate, and recognise the two silent failures, the preprocessing mismatch and the batch-normalisation trap, from their symptoms.
- Design an augmentation policy as an explicit statement of which transformations preserve the label in your own domain, and say what goes wrong when that statement is false.
- Evaluate an image classifier under class imbalance and asymmetric costs, set its operating threshold from the cost matrix, and audit its reasoning with Grad-CAM.
- State the obligations that the Data Protection Act 2019 and the CBK Digital Credit Providers Regulations 2022 place on a biometric vision system, and reason quantitatively about model size and quantisation for deployment on a mid-range handset.

7.1 Introduction

Chapter 6 built a network out of dense layers and trained it by backpropagation. Every idea developed there survives intact: the loss is still an explicit scalar objective, the gradients are still computed by one backward sweep, and initialisation, normalisation and regularisation still do the jobs they were given. What changes in this chapter is the *architecture*, and it changes for a specific reason. A dense layer treats its input as an unordered list of numbers. An image is not an unordered list of numbers. Two pixels that sit side by side are related in a way that two pixels at opposite corners

are not, and a network that has to discover that fact from data is spending its capacity on something the architect could simply have told it.

Encoding that fact into the architecture is what a convolutional layer does. The consequences are large enough that the whole modern history of computer vision follows from them, and they are direct enough that the entire construction can be derived in a few pages.

There is a second, more commercial consequence. Because the features a convolutional network learns on one collection of photographs are largely reusable on another, vision is the first domain in this course where the rational engineering move is *not* to train a model. It is to download one that somebody with a data centre already trained on a million labelled images, and adapt it in a few minutes on a laptop. Understanding why that works, and the small number of ways it silently fails, occupies the second half of the chapter.

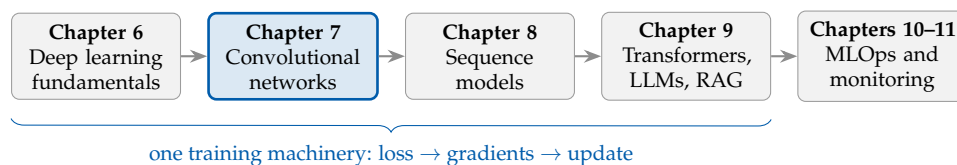


Figure 7.1: Part II proceeds by specialising the architecture, not the training procedure. Chapter 6 gave us networks that accept vectors; this chapter gives us networks that accept images; Chapter 8 gives us networks that accept ordered sequences. The optimiser never changes.

Case Study

Identity Verification, Round Six Our Nairobi digital lender now onboards borrowers entirely through a mobile application. A prospective customer photographs a national identity document and takes a selfie; within three minutes the system must decide whether to open an account and admit the applicant to the credit model of the preceding chapters.

Three vision models sit inside that flow. A **document classifier** identifies which of five accepted document types was uploaded, or rejects the image as unusable. A **tamper detector** flags splicing, re-photography of a screen, and printed copies. A **face matcher** compares the selfie with the portrait printed on the document.

The error economics are the ones established in Chapter 3 and used ever since. A fraudulent onboarding that survives into the loan book costs roughly KES 10,000; a genuine applicant wrongly rejected costs about KES 800 in forgone margin and acquisition spend, and, less measurably, excludes exactly the thin-file customer the business exists to serve. Two things are new. First, the input is an image, so none of the feature engineering of Chapter 2 applies. Second, a facial image is *sensitive personal data* under the Data Protection Act 2019, which makes the governance requirements stricter here than anywhere else in this book.

7.1.1 Four tasks, one backbone

It is worth fixing vocabulary before building anything. Computer vision problems are conventionally divided into four tasks, illustrated in Figure 7.2: *classification* assigns one label to the whole image; *localisation* adds a bounding box; *detection* finds an unknown number of objects, each with a box; and *segmentation* labels every pixel.

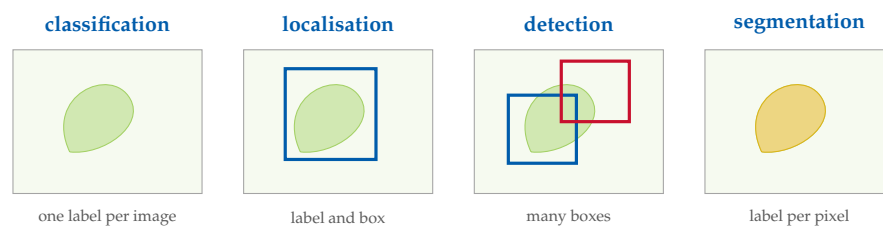


Figure 7.2: The four canonical vision tasks. This chapter treats classification, but the convolutional *backbone* is shared across all four; only the head differs. That is the structural reason a backbone trained for one task transfers to another.

This chapter treats classification only. The reason the other three matter here is architectural: all four are built on the same convolutional feature

extractor, or *backbone*, and differ only in the small head bolted on top. When we come to transfer learning in Section 7.4, that shared structure is precisely what we exploit.

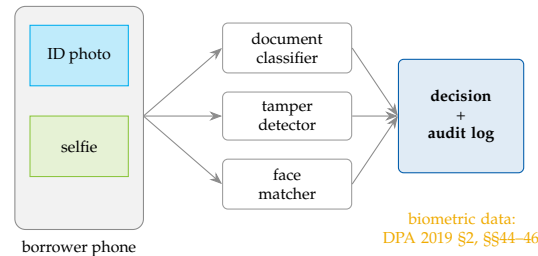


Figure 7.3: The lender’s onboarding flow. Three independent classifiers feed one decision, and every artefact of that decision must be retained: the regulatory obligations of Section 7.5.3 are a property of the pipeline, not of any one model.

7.2 Why Convolution?

Convolution is not merely the customary architecture for images. It is the correction of two specific assumptions that a dense layer makes about its input and that are false of photographs. Section 7.2.1 identifies them, and shows that only one of the two is usually noticed. The subsections that follow build the replacement: the convolution operator itself, together with the shape arithmetic you will do constantly (Section 7.2.2); the two pooling operations that accompany it (Section 7.2.3); and the argument that such layers must be stacked deep rather than made wide (Section 7.2.4). Section 7.2.5 then restates the whole construction in the currency that actually binds a Nairobi team, the number of labelled images required before a model is worth deploying.

7.2.1 Why a dense network fails on images

A colour image of height H , width W and three channels flattens to a vector of dimension HWc . For the 224×224 input that every pre-trained backbone expects, that is $224 \cdot 224 \cdot 3 = 150,528$ numbers. Two separate problems follow, and it is worth being clear that they are separate, because only the first is usually noticed and only the second is fatal.

The parameter explosion

The first problem is arithmetic. A dense layer connecting 150,528 inputs to 1,000 hidden units holds $150,528 \times 1,000 + 1,000 \approx 1.5 \times 10^8$ parameters. That is a *single* hidden layer, and it already carries six times as many parameters as the whole of ResNet-50, which we shall meet in Section 7.3.1.

Trained on the few thousand labelled identity documents a lender can realistically assemble, such a model will memorise the training set exactly and generalise to nothing.

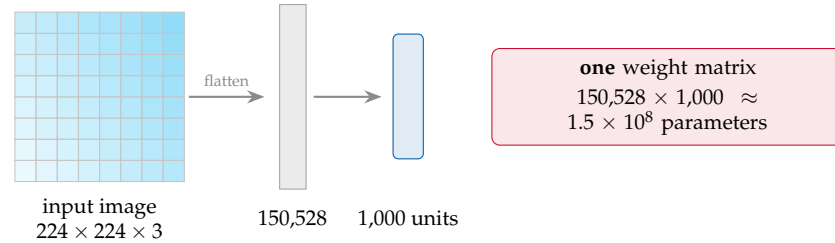


Figure 7.4: A single dense layer on a 224×224 RGB image. The parameter count exceeds that of the entire ResNet-50 network by a factor of six, before any depth has been added.

The missing locality prior

The second problem is subtler and matters more. Flattening destroys the information that two pixels are adjacent.

To see how completely, consider what happens if we permute the input coordinates by some fixed permutation P and compensate by permuting the columns of the first weight matrix, $\mathbf{W}'^{(1)} = \mathbf{W}^{(1)}P^\top$. Because $P^\top P = I$, the pre-activations are unchanged and the network computes exactly the same function. The permuted parameters have the same count, the same architecture, and, under any initialiser with i.i.d. entries, the same prior probability of being drawn. A dense network therefore cannot distinguish a photograph from the same photograph with its pixels scrambled by a fixed permutation. Whatever it eventually learns about adjacency, it learns from data, separately for every pair of positions, and it must relearn it for every position at which a pattern might appear.

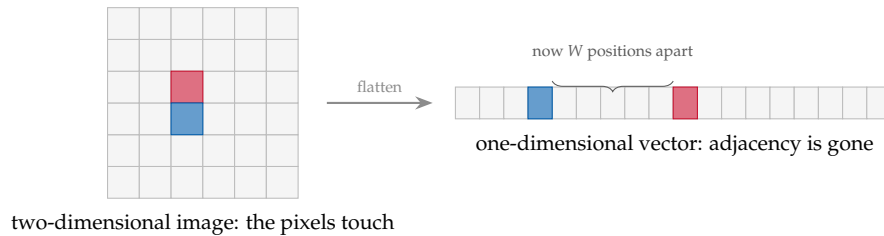


Figure 7.5: Flattening maps neighbouring pixels to coordinates W apart. Nothing in a dense architecture records that the two highlighted coordinates were ever related.

Three inductive biases

Convolution repairs both problems at once by constraining the hypothesis class in three ways, shown in Figure 7.6.

Locality. A unit responds to a small contiguous patch rather than to the whole image. Evidence for an edge, a corner, or a hologram is local evidence.

Weight sharing. The same detector is applied at every position. A serial number in the top-left corner is the same evidence as the same serial number in the centre, so it should be scored by the same weights.

Equivariance. Shifting the input shifts the response by the same amount, so the representation carries *where* as well as *what*.

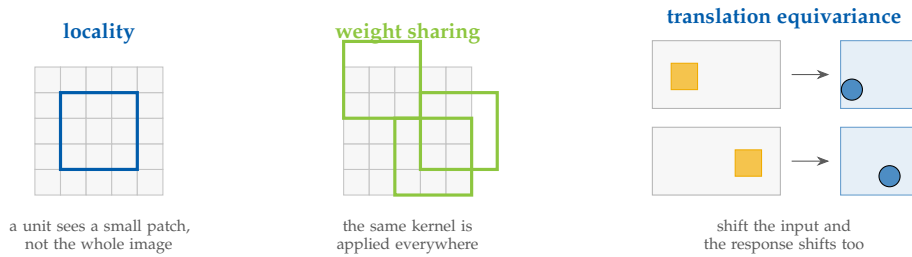


Figure 7.6: The three assumptions convolution makes about images. Each is a restriction on the hypothesis class, and each happens to be true of photographs, which is why the restriction improves generalisation instead of harming it.

Key Idea

Constraints that are true beat parameters The comparison to hold on to is with Chapter 6's dense layer. Convolution is not a different kind of computation; it is the same computation with the weight matrix constrained to be sparse and to have tied entries. In one dimension with a single channel, a convolution is a banded Toeplitz matrix with f distinct values, against $n_{\text{in}}n_{\text{out}}$ free values for an unconstrained dense layer. The constraint is a very strong prior. Because the prior is correct for photographs, it buys generalisation that no amount of extra data would have bought otherwise.

7.2.2 The convolution operator

Definition 7.1 (Two-dimensional convolutional layer). Let $\mathbf{X} \in \mathbb{R}^{H \times W \times c_{\text{in}}}$ be an input tensor and let $\mathbf{K} \in \mathbb{R}^{f \times f \times c_{\text{in}} \times c_{\text{out}}}$ be a kernel tensor with bias $\mathbf{b} \in$

$\mathbb{R}^{c_{\text{out}}}$. With zero padding p and stride s , the layer output $\mathbf{S} \in \mathbb{R}^{H' \times W' \times c_{\text{out}}}$ is

$$\mathbf{S}_{i,j,k} = \sum_{u=0}^{f-1} \sum_{v=0}^{f-1} \sum_{q=1}^{c_{\text{in}}} \tilde{\mathbf{X}}_{si+u, sj+v, q} \mathbf{K}_{u,v,q,k} + b_k, \quad (7.1)$$

where $\tilde{\mathbf{X}}$ is $\mathbf{X}$ zero-padded by p on each side.

Strictly, (7.1) is a cross-correlation rather than a convolution, since a true convolution flips the kernel. Because $\mathbf{K}$ is learned, the distinction changes nothing: the network simply learns the flipped kernel. Every deep-learning library implements (7.1) and calls it convolution, and so will we.

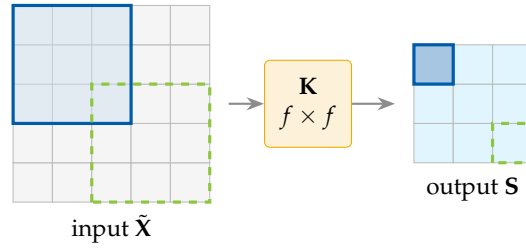


Figure 7.7: The kernel is applied at every admissible position of the padded input, and each placement contributes exactly one entry of the output feature map. The two highlighted placements produce the two highlighted outputs.

What does a single kernel actually compute? Before these weights were learned, they were designed by hand. Figure 7.8 applies the classical Sobel-style vertical-edge filter to an intensity step: the interior of each flat region produces zero, and the boundary lights up. In a convolutional network nobody writes those numbers down, but in essentially every network ever inspected, the first-layer kernels that backpropagation discovers look remarkably like edge and colour-blob detectors. This is the first hint of the feature hierarchy we return to in Section 7.2.4.

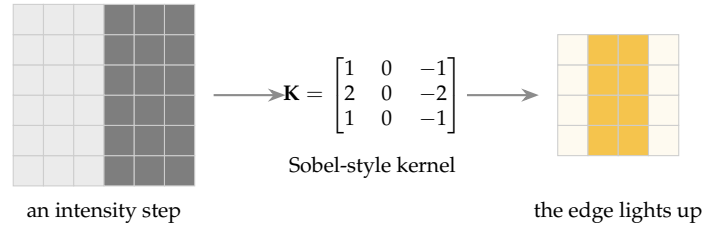


Figure 7.8: A hand-designed vertical-edge kernel. Learned first-layer kernels look much like this, which is the first indication that early features are generic rather than task-specific.

Padding, stride, and the two formulas you will use constantly

Two knobs control the geometry of the output. *Padding* p adds a border of zeros so that the kernel can be centred on boundary pixels; *stride* s is the step between successive kernel placements. Figure 7.9 shows both.

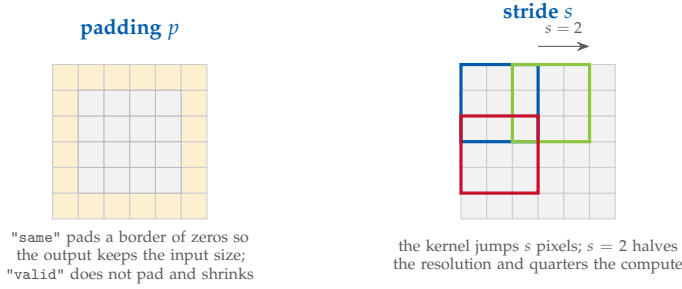


Figure 7.9: The two knobs that control output geometry. Both enter the shape formula (7.2).

Proposition 7.1 (Output shape and parameter count). *For a layer as in Definition 7.1, the spatial output size along either axis is*

$$n_{out} = \left\lfloor \frac{n_{in} + 2p - f}{s} \right\rfloor + 1, \quad (7.2)$$

and the layer holds

$$\text{\#params} = f^2 c_{in} c_{out} + c_{out} \quad (7.3)$$

parameters, independently of H and W . Its multiply–accumulate cost, by contrast, is $H'W'f^2c_{in}c_{out}$ and therefore does grow with resolution.

Proof. For (7.2): the padded input has length $n_{in} + 2p$, and a kernel whose left edge sits at t occupies indices $t, \dots, t + f - 1$, so t is admissible exactly when $t \leq n_{in} + 2p - f$. Admissible left edges are $0, s, 2s, \dots$, and the number of non-negative integers m with $ms \leq n_{in} + 2p - f$ is $\lfloor (n_{in} + 2p - f)/s \rfloor + 1$. For (7.3): the kernel tensor has $f^2c_{in}c_{out}$ entries and there is one bias per output channel, neither of which involves H or W ; each of the $H'W'c_{out}$ outputs is a sum of f^2c_{in} products. $\square$

Two special cases are worth memorising, because they cover almost everything you will write. Taking $s = 1$ and $p = (f - 1)/2$ for odd f gives $n_{out} = n_{in}$, which Keras calls `padding="same"`. Taking $p = 0$ gives `padding="valid"`, which shrinks the map by $f - 1$ at unit stride.

The second half of Proposition 7.1 is the source of a common surprise. Moving from a 224×224 input to a 384×384 input changes no parameter count anywhere in the network, yet can triple the training time and exhaust the GPU. Weights do not scale with resolution; *activations* do, and it is activations that fill memory.

Channels

Real inputs have depth. Figure 7.10 shows the shape of the computation: each of the c_{out} kernels spans *all* c_{in} input channels and produces exactly *one* output channel. The characteristic shape of a convolutional stack follows: spatial resolution falls while channel depth rises, so that the representation trades "where" for "what" as it climbs.

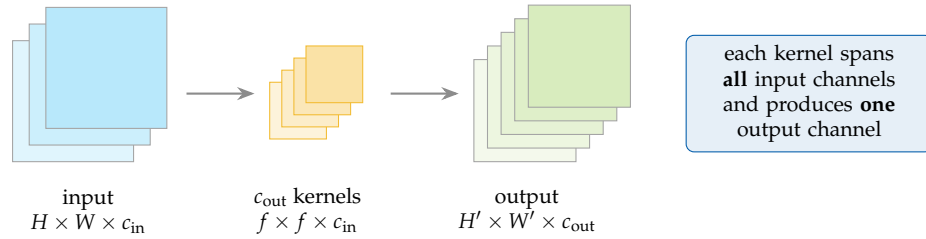


Figure 7.10: A convolutional layer maps a tensor to a tensor. Kernels are four-dimensional objects; the count (7.3) follows by counting their entries.

Table 7.1 applies both formulas to a small stack. Compare its total, 379,781 parameters, with the 1.5×10^8 of the single dense layer in Figure 7.4: three orders of magnitude, for a model that is strictly more capable on images.

Table 7.1: Shapes and parameter counts for a small stack on a 224×224 RGB document photograph. Every entry follows from Proposition 7.1.

Layer	Output shape	Parameters
Input	$224 \times 224 \times 3$	0
Conv2D(64, 7, s=2, same)	$112 \times 112 \times 64$	9,472
MaxPooling2D(3, s=2, same)	$56 \times 56 \times 64$	0
Conv2D(128, 3, same)	$56 \times 56 \times 128$	73,856
MaxPooling2D(2)	$28 \times 28 \times 128$	0
Conv2D(256, 3, same)	$28 \times 28 \times 256$	295,168
MaxPooling2D(2)	$14 \times 14 \times 256$	0
GlobalAveragePooling2D	256	0
Dense(5, softmax)	5	1,285
Total		379,781

Equivariance, and its limits

Weight sharing buys equivariance, but only exactly, and only at unit stride.

Proposition 7.2 (Translation equivariance). *Let $\mathcal{C}$ be the convolution of Definition 7.1 with stride s , away from boundary effects, and let T_τ denote translation*

by τ . If s divides both components of τ then $\mathcal{C}(T_\tau \mathbf{X}) = T_{\tau/s} \mathcal{C}(\mathbf{X})$. For $s = 1$ this holds for every integer translation.

Proof. Write $\mathbf{Y} = T_\tau \mathbf{X}$, so $Y_{a,b,q} = X_{a-\tau_1, b-\tau_2, q}$. Substituting into (7.1) and putting $i' = i - \tau_1/s$, $j' = j - \tau_2/s$, which are integers exactly because s divides τ , gives $si + u - \tau_1 = si' + u$ and $sj + v - \tau_2 = sj' + v$, so $\mathcal{C}(\mathbf{Y})_{i,j,k} = \mathcal{C}(\mathbf{X})_{i',j',k}$. $\square$

The divisibility condition is not a technicality to be waved through. A ResNet-50 backbone has a total stride of 32, so its deepest features are equivariant only to translations that are multiples of 32 pixels. A borrower photographing an identity card by hand shifts the content by arbitrary amounts, and the model's response genuinely does change. Two remedies follow directly, and we shall use both: include small random translations in the augmentation policy so that approximate invariance is learned from data (Section 7.4.5), and avoid evaluating on a single fixed crop when several crops are affordable.

7.2.3 Pooling and global average pooling

A pooling layer replaces each $f \times f$ window of a feature map by a single summary, the maximum or the mean, applied independently to each channel. It has no parameters. It does three jobs: it reduces spatial resolution, it enlarges the receptive field of everything above it, and it buys a small amount of invariance.

The invariance is worth stating precisely, because it is routinely overstated. If the maximum within a pooling window is unique and a small translation leaves that maximiser inside the same window, the pooled output is unchanged, because the maximum of a set does not depend on how its elements are indexed. That is genuine invariance, but it is *local*: it covers sub-window jitter, not displacement across the image. Modern architectures often replace pooling with strided convolutions, which learn the downsampling rather than fixing it, but the role in the architecture is the same.

Global average pooling deserves separate emphasis. At the top of the backbone it replaces each $H' \times W'$ map by its mean, turning ResNet-50's $7 \times 7 \times 2048$ tensor into a 2048-vector. The alternative, `Flatten`, would produce a 100,352-vector, and a classifier on top of it would carry 49 times as many parameters. Global average pooling also makes the network tolerant of a change in input resolution, since the width of its output no longer depends on H' and W' at all. That tolerance is what makes the resolution scaling of Section 7.3.2 possible.

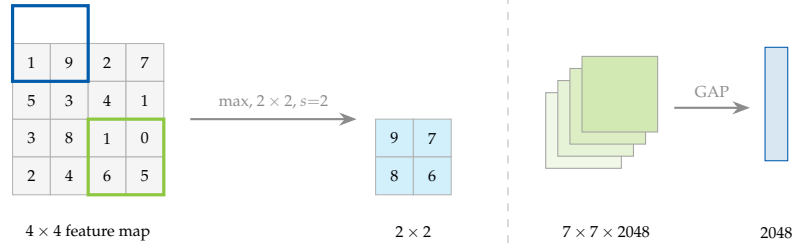


Figure 7.11: Left: max pooling summarises each window by its largest value. Right: global average pooling collapses each channel to its spatial mean, which is how every modern backbone terminates.

7.2.4 Receptive fields and the case for depth

The *receptive field* of a unit is the set of input pixels that can influence it. It is the right way to think about what a layer could possibly represent: a unit whose receptive field is 7×7 cannot detect a feature spanning thirty pixels, however it is trained.

Proposition 7.3 (Receptive-field recursion). *For a stack of layers $\ell = 1, \dots, L$ with kernel sizes f_ℓ and strides s_ℓ , define the jump $j_0 = 1$, $j_\ell = j_{\ell-1}s_\ell$, and $r_0 = 1$,*

$$r_\ell = r_{\ell-1} + (f_\ell - 1)j_{\ell-1}. \quad (7.4)$$

Then r_L is the side length in input pixels of the receptive field of a unit in layer L .

Proof. By induction. At $\ell = 0$ a unit is a pixel, so $r_0 = 1$, and adjacent units are one pixel apart, so $j_0 = 1$. Suppose the claim holds at $\ell - 1$. A unit of layer ℓ aggregates f_ℓ consecutive units of layer $\ell - 1$ along each axis; the leftmost sees input pixels $[t, t + r_{\ell-1} - 1]$ and the rightmost sees $[t + (f_\ell - 1)j_{\ell-1}, t + (f_\ell - 1)j_{\ell-1} + r_{\ell-1} - 1]$, whose union is an interval of length $r_{\ell-1} + (f_\ell - 1)j_{\ell-1}$. One step in layer ℓ is s_ℓ steps in layer $\ell - 1$ and hence $s_\ell j_{\ell-1} = j_\ell$ input pixels. $\square$

Two consequences drive architecture design. The first, illustrated in Figure 7.12, is that three stacked 3×3 convolutions at unit stride have the same 7×7 receptive field as one 7×7 convolution, yet use $27c^2$ weights rather than $49c^2$, a saving of 45%, while inserting two additional non-linearities along the way. Depth is simply a better use of a weight budget than kernel width, and this is the design principle behind VGG and, through it, of essentially everything since.

The second consequence is a caution. The jump j_ℓ multiplies, so receptive field grows slowly until the network has downsampled a few times, and quickly thereafter. That is why architectures front-load their striding: the 7×7 stride-two stem and the immediate max-pool in ResNet-50 exist to raise the jump early, so that the expensive residual stages operate on units that already see a useful region of the image.

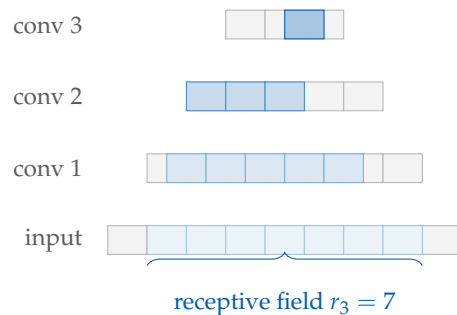


Figure 7.12: Each additional 3×3 layer at unit stride widens the receptive field by two pixels. Three such layers see the same 7×7 window as one 7×7 layer, using 45% fewer weights and two more non-linearities.

What the layers end up representing

Putting locality, weight sharing and growing receptive fields together produces a characteristic hierarchy, sketched in Figure 7.13. Early layers, whose units see a handful of pixels, learn oriented edges, colour opponency and local contrast. Middle layers compose these into textures, corners and repeated motifs. Late layers, whose units see most of the image, respond to object parts: eyes, glyphs, holograms. The head then reads task-specific evidence off the last of these.

Nothing instructs any layer to represent these things. The only signal reaching layer two is the gradient of the loss propagated back through layer three, and representations that reduce the loss survive while others are pushed out of existence, exactly as in Chapter 6. What is new is that the hierarchy is remarkably stable across tasks. The edge detectors a network learns for classifying dogs are the edge detectors it would have learned for classifying identity documents. That empirical fact, and nothing else, is the justification for the whole of Section 7.4.

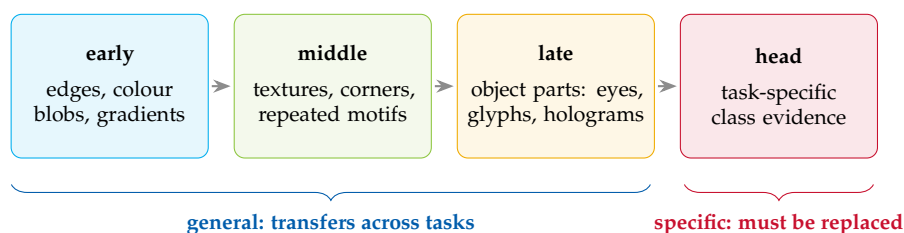


Figure 7.13: The representational hierarchy that a convolutional network converges to without being told. The division marked by the two braces is the one transfer learning exploits.

7.2.5 The data-efficiency argument

We could now build the stack of Table 7.1 and train it from scratch. It would work, and it would disappoint. Figure 7.14 shows why, schematically: a network trained from random initialisation needs a great deal of labelled data before it becomes competitive, whereas one initialised from a pre-trained backbone starts far higher and improves more gently.

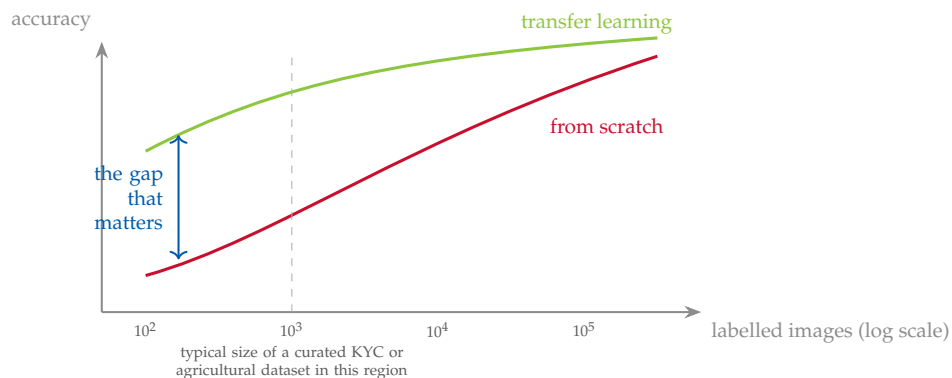


Figure 7.14: Schematic, not measured. The horizontal position of the dashed line, rather than the exact shape of the curves, is the point: most datasets a Kenyan fintech or agritech team can assemble sit in the region where the gap between the two regimes is at its widest.

The dashed line marks the regime our lender actually inhabits. Assembling thirty thousand labelled identity documents is a serious operational project; assembling the million-image ImageNet corpus is not something a lending business will ever do. But the network trained on that corpus is a free download. The remainder of this chapter is, in effect, an extended argument for using it, followed by the details that determine whether using it works.

7.3 Modern Architectures: ResNet and EfficientNet

A backbone is a piece of engineering you adopt rather than design, but adopting one responsibly means knowing what makes it worth adopting. Two ideas do most of that work, and this section takes them in turn. Residual connections (Section 7.3.1) removed the depth ceiling that had held since the first deep stacks were trained. Compound scaling (Section 7.3.2) replaced guesswork about how large a network should be with an explicit budget. Section 7.3.3 closes with the comparison a design review will ask for, and names the two backbones used in the rest of this book.

7.3.1 Residual networks

Before we borrow a backbone, we should understand the two architectural ideas that make the good ones good. The first is residual learning.

The degradation problem

Before 2015, adding layers beyond roughly twenty made networks worse. What made this startling, and what makes it worth a section in a textbook, is that they became worse on the *training* set, as Figure 7.15 shows. Overfitting is therefore not the explanation.

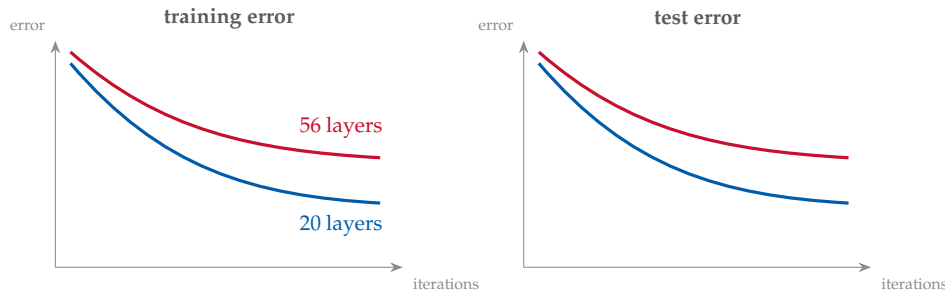


Figure 7.15: The deeper plain network is worse on *training* error too, so the failure is one of optimisation, not of capacity. Redrawn schematically after He et al. (2016).

He et al. (2016) pointed out that the situation is logically strange. A 56-layer network can represent any 20-layer network exactly, by making the extra 36 layers compute the identity. A solution at least as good as the shallow one therefore *exists* in the deeper hypothesis class. Gradient descent simply could not find it. The problem is not what the network can represent but what the optimiser can reach.

The residual block

Definition 7.2 (Residual block). A residual block with residual branch $\mathcal{F}(\cdot; \theta)$ computes

$$\mathbf{y} = \phi(\mathcal{F}(\mathbf{x}; \theta) + \mathbf{x}), \quad (7.5)$$

with elementwise addition. Where $\mathcal{F}$ changes the shape of its input, a 1×1 projection $W_s \mathbf{x}$ replaces $\mathbf{x}$ on the shortcut.

The reframing this achieves can be said in one sentence: the block no longer learns a mapping, it learns the *deviation* from the identity. If the optimal mapping happens to be the identity, the block only has to drive $\theta \rightarrow 0$, which is easy and which weight decay is already pushing towards anyway. In a plain network, by contrast, computing the identity requires the weight matrix to become exactly I , a specific configuration that weight

decay actively pushes *away* from. Residual networks make the easy thing the default and ask the optimiser to learn only what departs from it.

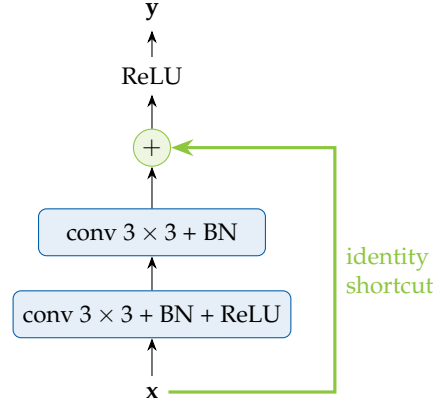


Figure 7.16: The block of Definition 7.2. The shortcut carries no parameters, so the entire cost of the block is in the residual branch.

Why the shortcut fixes the gradient

The optimisation benefit has a clean derivation. In the pre-activation form of the block, where ϕ is applied inside $\mathcal{F}$, the stack is simply $\mathbf{x}_{\ell+1} = \mathbf{x}_\ell + \mathcal{F}_\ell(\mathbf{x}_\ell)$.

Theorem 7.1 (The gradient highway). *For such a stack and any loss $\mathcal{L}$,*

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}_\ell} = \frac{\partial \mathcal{L}}{\partial \mathbf{x}_L} \prod_{k=\ell}^{L-1} \left(\mathbf{I} + \frac{\partial \mathcal{F}_k}{\partial \mathbf{x}_k} \right) = \frac{\partial \mathcal{L}}{\partial \mathbf{x}_L} \left(\mathbf{I} + \sum_{k=\ell}^{L-1} \frac{\partial \mathcal{F}_k}{\partial \mathbf{x}_k} + \dots \right), \quad (7.6)$$

so the gradient reaching layer ℓ contains an additive term equal to $\partial \mathcal{L} / \partial \mathbf{x}_L$ exactly, whatever the depth.

Proof. Differentiating the recursion gives $\partial \mathbf{x}_{k+1} / \partial \mathbf{x}_k = \mathbf{I} + \partial \mathcal{F}_k / \partial \mathbf{x}_k$, and the chain rule over $k = \ell, \dots, L-1$ gives the product. Expanding it term by term yields $\mathbf{I}$, obtained by choosing the identity from every factor, plus the sum of single Jacobians, plus higher-order cross terms. $\square$

Compare this with the plain-network case of Chapter 6, where the propagated error is a bare product $\prod_k \mathbf{W}_k^\top \text{diag}(\phi')$ that decays geometrically as soon as the factors have norm below one; the exponential bound proved there showed that this decay is the generic case rather than a rare accident. In (7.6) the identity term cannot vanish: no matter how small every residual Jacobian becomes, layer ℓ still receives $\partial \mathcal{L} / \partial \mathbf{x}_L$ undamped. That is the whole reason 50-, 101- and 152-layer networks became trainable, and it is why the vanishing-gradient discussion of Chapter 6 does not need to be repeated here.

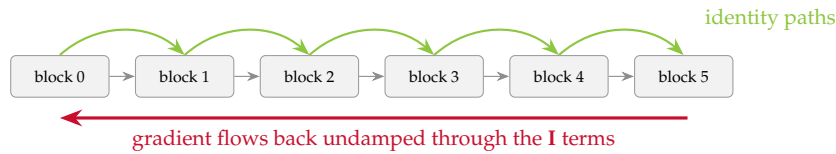


Figure 7.17: The identity shortcuts of Theorem 7.1 give every block a direct path to the loss. The product in (7.6) can no longer collapse to zero.

Bottlenecks, and the shape of ResNet-50

Depth is only useful if it is affordable. A plain 3×3 convolution mapping C channels to C costs $9C^2$ weights, which at $C = 256$ is nearly 590,000 per block. The *bottleneck* design of Figure 7.18 squeezes the channel count down to b with a cheap 1×1 convolution, performs the expensive spatial mixing at that reduced width, and restores the channels with a second 1×1 . The cost is $2Cb + 9b^2$, which at $b = C/4 = 64$ is about 70,000: an eight-fold saving, per block, across sixteen blocks.

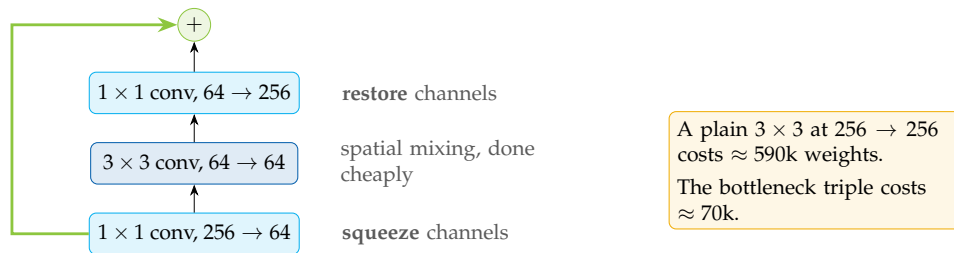


Figure 7.18: The bottleneck design spends its budget on depth rather than on width. ResNet-50 uses $b = C/4$ throughout.

Figure 7.19 assembles the whole network, and marks the single most important line in this chapter. Everything to the left of the dashed line is the backbone, a generic feature extractor ending in a tensor of shape $7 \times 7 \times 2048$. Everything to the right is the head: global average pooling and a dense layer producing one thousand ImageNet class scores. Passing `include_top=False` to `keras.applications.ResNet50V2` downloads the left-hand side alone.

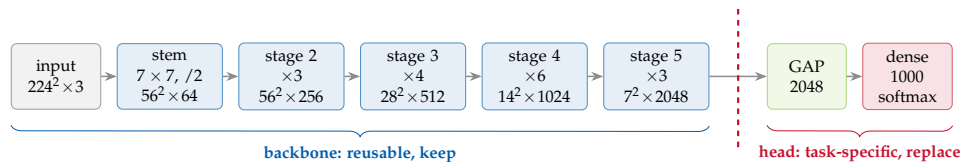


Figure 7.19: ResNet-50, with the cut that transfer learning exploits. Resolution falls from 224 to 7 while depth rises from 3 to 2048: the network trades *where* for *what* as it climbs.

7.3.2 EfficientNet and compound scaling

The second idea concerns growth. Given a larger compute budget, one may deepen a network (d), widen it (w), or feed it higher-resolution images (r). Figure 7.20 shows the three options. Each helps, and each saturates: scaling any one dimension alone plateaus near 80% ImageNet top-1 accuracy, because a network that is very deep but narrow runs out of channels to carry information, one that is very wide but shallow runs out of composition depth, and one fed very large images but built shallow never grows a receptive field big enough to use them.

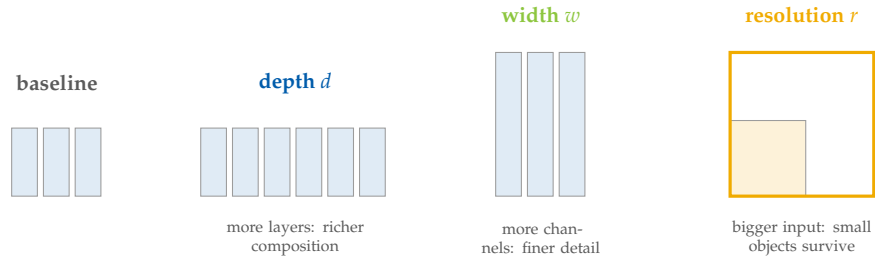


Figure 7.20: Depth, width and resolution. Each saturates on its own; Tan and Le (2019) showed they must be scaled jointly and in fixed proportion.

The proposal of Tan and Le (2019) is to scale all three by a single coefficient ϕ ,

$$d = \alpha^\phi, \quad w = \beta^\phi, \quad r = \gamma^\phi, \quad \text{subject to } \alpha\beta^2\gamma^2 \approx 2, \quad \alpha, \beta, \gamma \geq 1. \quad (7.7)$$

Proposition 7.4 (Compound scaling is a budgeting rule). *The multiply-accumulate cost of a convolutional stack is proportional to $d w^2 r^2$. Under (7.7) the cost multiplier is therefore $(\alpha\beta^2\gamma^2)^\phi$, so setting the constrained product to 2 makes each unit increase in ϕ a doubling of the compute budget.*

Proof. By Proposition 7.1 one layer costs $H'W'f^2c_{\text{in}}c_{\text{out}}$. Widening by w multiplies both channel counts, contributing w^2 ; raising resolution by r multiplies H' and W' , contributing r^2 ; deepening by d multiplies the number of layers. Substituting (7.7) gives $d w^2 r^2 = (\alpha\beta^2\gamma^2)^\phi$. $\square$

The exponents in the constraint are therefore not a tuning choice; they are dictated by how each dimension enters the FLOP count. The coefficients themselves are found once by a small grid search on the B0 baseline, giving $\alpha = 1.2$, $\beta = 1.1$, $\gamma = 1.15$ in the original paper, after which ϕ generates the whole family shown in Figure 7.21.

The building block is worth one paragraph, because it explains where the savings come from. An *MBConv* block, shown in Figure 7.22, expands the channel count with a 1×1 convolution, performs spatial mixing with

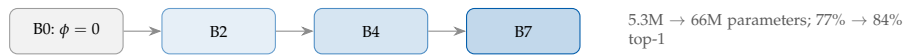


Figure 7.21: One coefficient generates the whole family. Every member is a down-load; none of them needs to be designed again.

a *depthwise* convolution that applies one kernel per channel rather than one per channel pair, rescales each channel by a learned squeeze-and-excitation gate, and projects back down. The depthwise convolution costs f^2c weights instead of f^2c^2 , which for $c = 256$ is a saving of more than two orders of magnitude on that layer; the squeeze-and-excitation gate spends a little of the saving on letting the network decide, per image, which channels matter.

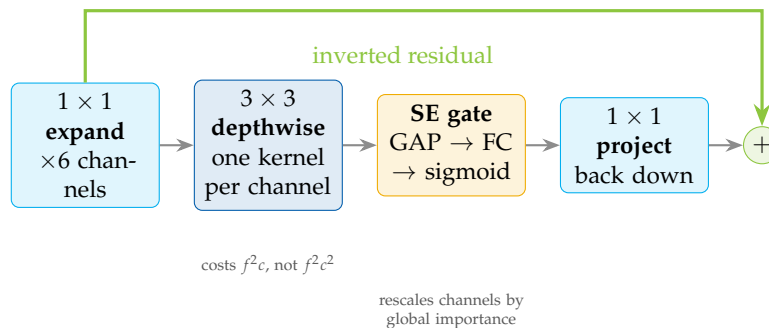


Figure 7.22: Cheap spatial mixing plus a learned channel gate. The inverted residual runs around the outside, so Theorem 7.1 applies here too.

The result is a family that dominates ResNet on the accuracy-per-parameter frontier, as Figure 7.23 shows. For our lender, where inference may run on the customer's own handset over a connection they pay for by the megabyte, that frontier is not an academic curiosity: it is the difference between an application people install and one they do not.

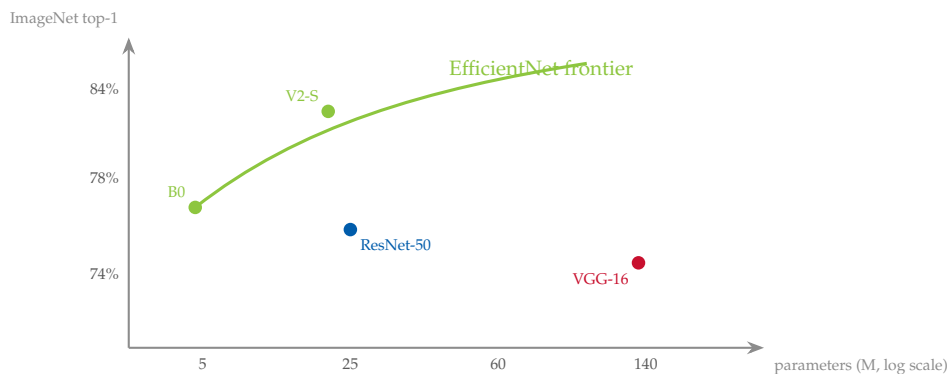


Figure 7.23: Accuracy rises while parameter count falls. Published ImageNet figures, plotted schematically.

7.3.3 Choosing a backbone

Table 7.2 sets out the comparison in the form a design review expects. The rule of thumb is to prototype with ResNet50V2, which is predictable and forgiving under fine-tuning, and to ship with EfficientNetV2 when size or latency binds. Better still, run both on your own data: the whole comparison costs an afternoon.

Table 7.2: Choosing between the two backbones used in this book. Figures are for the Keras implementations at 224×224 input.

Criterion	ResNet50V2	EfficientNetV2-B0
Parameters, size	25.6M, $\approx$ 98 MB	7.2M, $\approx$ 26 MB
Accuracy per FLOP	good	better
Fine-tuning behaviour	robust, forgiving	excellent, more sensitive to the learning rate
Preprocessing	resnet_v2.preprocess_input; range $[-1, 1]$	built into the model; pass raw $[0, 255]$
Edge deployment	heavy without compression	designed for it

Pitfall

The preprocessing mismatch Every Keras backbone ships with a matching preprocessing function, and they disagree with each other. ResNetV2 expects inputs scaled to $[-1, 1]$; EfficientNetV2 performs its own rescaling internally and expects raw $[0, 255]$ pixels. Feeding a backbone the wrong range raises *no error of any kind*. The tensor has the right shape and the right dtype, the model trains, the loss falls. What happens instead is that activations saturate, the pre-trained features become close to meaningless, and accuracy settles five to ten points below where it should be, a loss easily misread as "deep learning does not work on our data". When you change backbones, change the preprocessing in the same commit, and make the pairing a single function in your codebase so that the two cannot drift apart.

7.4 Transfer Learning

We can now state the plan of Figure 7.25. Somebody spent weeks of data-centre time learning a backbone ϕ_θ on ImageNet's 1.2 million images and 1,000 classes. We download θ , discard the thousand-class head, attach a five-class head g_ψ of our own, and learn $\hat{y} = g_\psi(\phi_\theta(\mathbf{x}))$ on three thousand

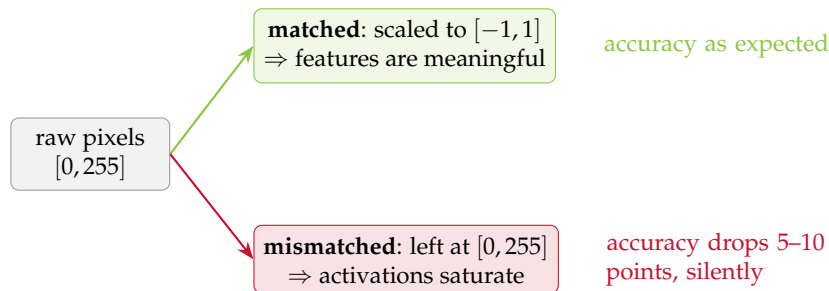


Figure 7.24: The most common silent bug in transfer learning. Nothing in the stack trace tells you it has happened.

identity documents. The general visual features were paid for once, by an organisation with the resources to pay for them; what remains is the part specific to our task, our labels and our regulator.

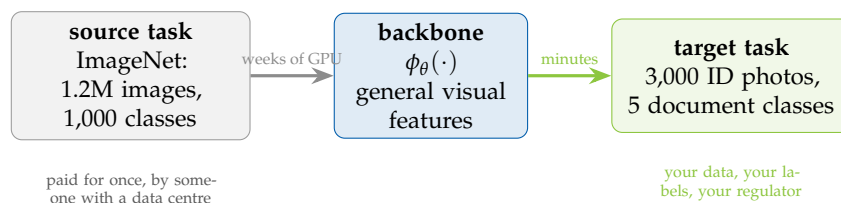


Figure 7.25: Transfer learning re-uses ϕ_θ and learns only g_ψ , optionally letting the top of ϕ_θ drift afterwards.

Why should features learned for dogs and aeroplanes be useful for identity documents? Yosinski et al. (2014) answered this empirically, by measuring how well each layer's features transfer to a new task. The result, sketched in Figure 7.26, is a smooth decay: early layers encode properties of natural images *in general* and transfer almost for free; late layers encode evidence for the specific classes of the source task and transfer poorly. Our document classifier needs edges just as much as a dog classifier does. It does not need the golden-retriever detector.

7.4.1 Two regimes

That curve implies two operating points, drawn in Figure 7.27. In **feature extraction** the backbone is frozen entirely and only the new head is trained. In **fine-tuning** the top of the backbone is also unfrozen and allowed to adapt.

Feature extraction has a property worth pausing on. With θ frozen, the features $\mathbf{z}_i = \phi_\theta(\mathbf{x}_i)$ are fixed numbers, and training a softmax head on them is multinomial logistic regression: the objective is a linear term plus a log-sum-exp of an affine function of the parameters, and log-sum-exp is

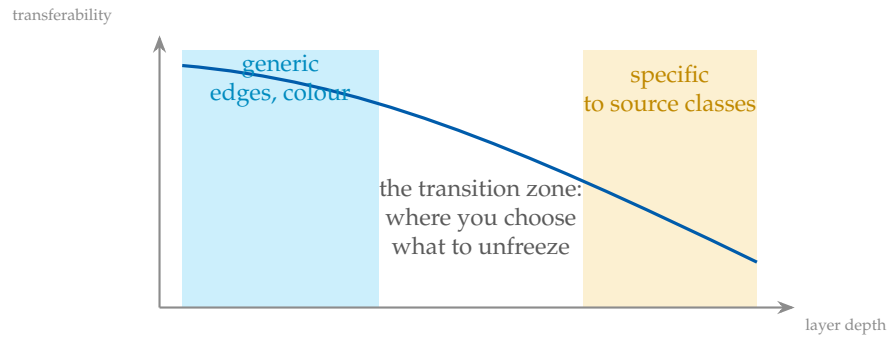


Figure 7.26: Feature transferability, after Yosinski et al. (2014). The shape of this curve is what makes the freeze-versus-fine-tune choice a decision about *depth*.

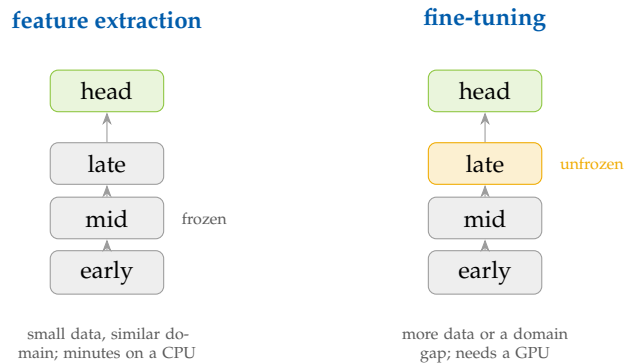


Figure 7.27: Feature extraction and fine-tuning. In practice one does both, in this order; Section 7.4.2 explains why the order is not negotiable.

convex, so the problem is *convex*. There are no bad local minima, no initialisation sensitivity and no need for a GPU. You can compute the features once for the whole dataset and fit the head on those vectors in seconds. When the target set is small and the domain close to ImageNet, this is frequently the entire project, and it is always the right first experiment.

Which regime applies is largely determined by two quantities, and Figure 7.28 reads the decision off them. Photographs of documents, faces and crops sit close to ImageNet’s domain; satellite, radiographic and infrared imagery sit far from it. Most African fintech and agritech vision problems land in the bottom-left cell, which is the cell where transfer learning works best and costs least.

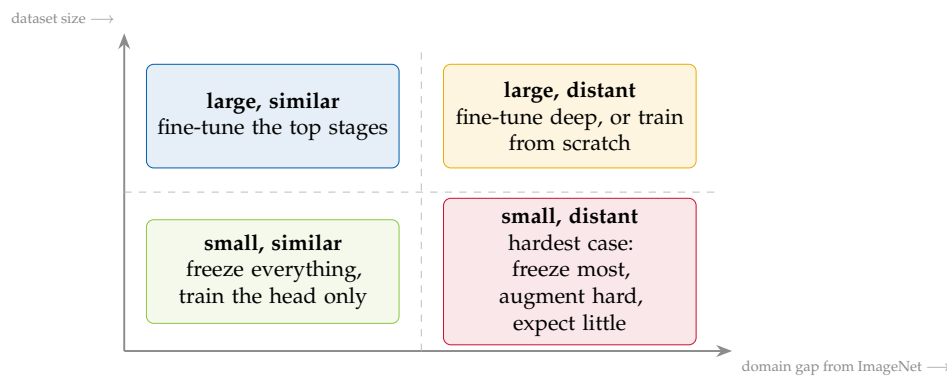


Figure 7.28: The regime follows from target-set size and domain distance. Most problems in this book sit in the bottom-left cell.

7.4.2 The two-phase recipe

Key Idea

Freeze, train the head, then unfreeze gently **Phase A**. Freeze the entire backbone. Train the new head to convergence with $\eta \approx 10^{-3}$.

Phase B. Unfreeze the top stage or stages. *Recompile* with $\eta \approx 10^{-5}$, one to two orders of magnitude smaller, and train with early stopping.

The schedule is drawn in Figure 7.29, and the reason for it is a statement about gradient magnitudes. At initialisation the head’s weights are random, so its predictions are near-uniform and its gradients are large and carry almost no information about the task. If the backbone is trainable from step one, those large, uninformative gradients propagate down and overwrite features that took a data centre weeks to learn, before the head has learned anything at all. This is *catastrophic forgetting*, and Figure 7.30 draws the geometry: the pre-trained weights sit in a good basin of the loss

surface, and a large step throws the parameters out of it.

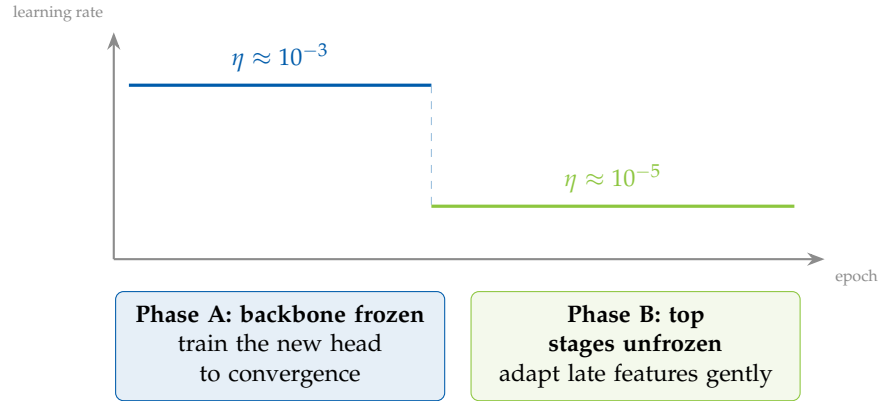


Figure 7.29: The learning-rate schedule of transfer learning. The step down at the unfreeze point is the single most important hyper-parameter decision in the whole procedure.

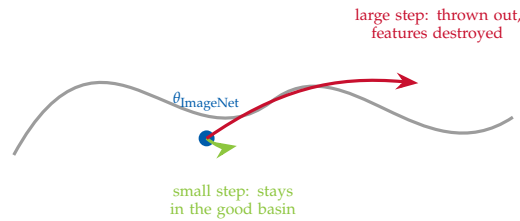


Figure 7.30: Why the learning rate must fall at the unfreeze point. We wish to nudge θ away from θ_{ImageNet} , not to re-derive it.

A refinement, useful when several stages are unfrozen at once, is to assign layer ℓ of L its own rate,

$$\eta_{\ell} = \eta_{\text{top}} \cdot \zeta^{L-\ell}, \quad \zeta \in [0.3, 0.9], \quad (7.8)$$

so that generic early layers barely move while late layers adapt freely (Figure 7.31). In Keras this is implemented either with per-layer learning-rate multipliers or, more simply and just as effectively, by progressive unfreezing one stage at a time.

7.4.3 Implementation in TensorFlow and Keras

```
from tensorflow import keras
from tensorflow.keras import layers

backbone = keras.applications.ResNet50V2(
```

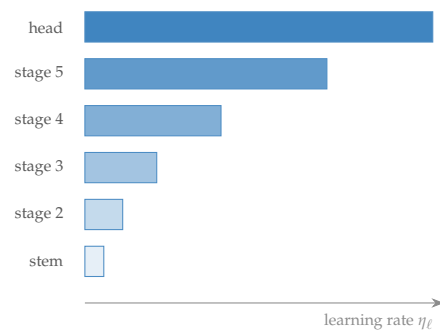


Figure 7.31: Deeper layers should move less than shallower ones, in the geometric progression (7.8).

```
include_top=False, weights="imagenet", input_shape=(224, 224, 3))
backbone.trainable = False                # freeze every weight
inputs = layers.Input(shape=(224, 224, 3))
x = augment(inputs)                       # augmentation layers
x = keras.applications.resnet_v2.preprocess_input(x)
x = backbone(x, training=False)           # keep BN in inference mode
x = layers.GlobalAveragePooling2D()(x)
x = layers.Dropout(0.3)(x)
outputs = layers.Dense(N_CLASSES, activation="softmax")(x)
model = keras.Model(inputs, outputs)
model.compile(optimizer=keras.optimizers.Adam(1e-3),
              loss="sparse_categorical_crossentropy", metrics=["accuracy"])
```

Listing 7.1: Phase A: frozen backbone, new head.

```
backbone.trainable = True                  # unfreeze, then re-freeze
for layer in backbone.layers:
    if not layer.name.startswith("conv5"):
        layer.trainable = False
    if isinstance(layer, layers.BatchNormalization):
        layer.trainable = False           # never update BN statistics

model.compile(optimizer=keras.optimizers.Adam(1e-5), # 100x smaller
              loss="sparse_categorical_crossentropy", metrics=["accuracy"])

callbacks = [
    keras.callbacks.EarlyStopping(patience=5, monitor="val_loss",
                                   restore_best_weights=True),
    keras.callbacks.ModelCheckpoint("best.keras", save_best_only=True),
]
```

Listing 7.2: Phase B: unfreeze the top stage and drop the learning rate.

Re-compiling in Phase B is mandatory rather than cosmetic. Keras caches the list of trainable weights at compile time, so flipping trainable flags without recompiling silently changes nothing at all, and you will

spend an afternoon wondering why fine-tuning had no effect.

7.4.4 The batch-normalisation trap

The second silent failure is more dangerous than the first, because it destroys a model that was working ten seconds earlier.

Recall from Chapter 6 that a batch-normalisation layer behaves differently in the two modes: in training mode it normalises by the statistics of the current mini-batch, and at inference it uses running averages accumulated during training. A pre-trained backbone carries running averages computed over ImageNet. If the backbone is called without `training=False`, its normalisation layers switch to batch statistics during fine-tuning and progressively overwrite those averages with statistics from your own small, often non-i.i.d. batches. Every downstream activation shifts, and the features the head learned to read in Phase A cease to exist.

Pitfall

The BatchNorm trap *Symptom*. Validation accuracy rises smoothly throughout Phase A, then collapses within one or two epochs of unfreezing, often to below the frozen-backbone baseline (Figure 7.32).

Why it is worse than it looks. The running statistics are themselves overwritten during training, so the damage survives a switch back to inference mode: the checkpoint on disk is already spoiled. This is a bug to prevent, not one to diagnose afterwards.

Fix. Call the backbone as `backbone(x, training=False)` at graph construction time, and set `layer.trainable = False` on every BatchNormalization layer, exactly as in Listing 7.2.

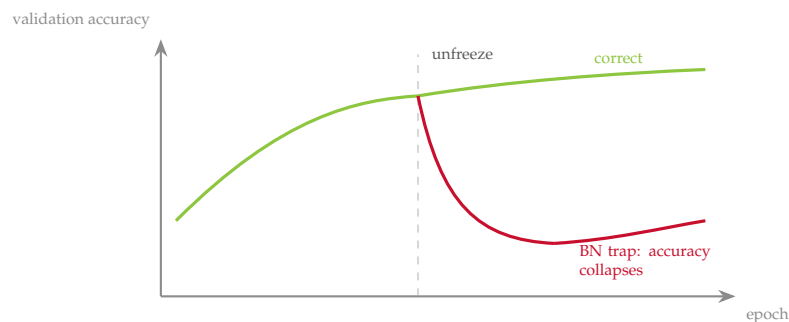


Figure 7.32: A collapse at the unfreeze point, rather than a gradual degradation, is almost always this bug rather than an excessive learning rate.

7.4.5 Data augmentation as a statement about your domain

Augmentation applies random label-preserving transformations at training time, so that instead of the empirical risk of Chapter 6 we minimise

$$\mathbb{E}_{(\mathbf{x}, y)} \mathbb{E}_{T \sim \mathcal{T}} [\mathcal{L}(f_{\theta}(T(\mathbf{x})), y)], \quad (7.9)$$

where $\mathcal{T}$ is a distribution over transformations. Figure 7.33 shows a typical set.

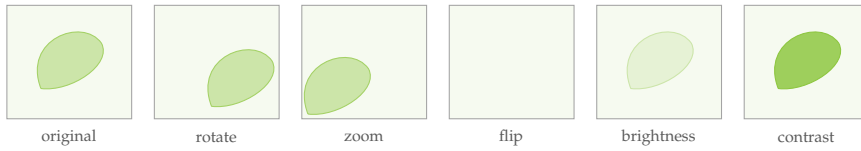


Figure 7.33: A label-preserving transformation set for crop photographs. Whether the same set is label-preserving for identity documents is the subject of the next page, and the answer is no.

It is tempting to treat (7.9) as a generic regulariser and copy an augmentation block between projects. It is not generic. To see what it is doing, suppose $\mathcal{T}$ is a finite group G and write $\bar{f}(\mathbf{x}) = |G|^{-1} \sum_{T \in G} f(T\mathbf{x})$ for the group average. For squared loss the bias–variance identity gives

$$\mathbb{E}_T \left[(f(T\mathbf{x}) - y)^2 \right] = (\bar{f}(\mathbf{x}) - y)^2 + \text{Var}_T[f(T\mathbf{x})],$$

so the augmented objective fits the label *and* penalises the model for varying across G . That second term is the whole point: augmentation is an explicit instruction to be invariant to G .

An instruction to be invariant is only sensible if the label really is constant on the orbit $\{T\mathbf{x} : T \in G\}$. If it is not, two things go wrong at once. The transformed image enters training with the wrong label, so we have manufactured label noise deliberately; and the variance term actively penalises the model for telling $\mathbf{x}$ and $T\mathbf{x}$ apart, which is a regulariser pulling in precisely the wrong direction. No quantity of extra data recovers the loss, because the Bayes-optimal classifier for the augmented distribution is itself worse than the one for the true distribution.

Key Idea

The same transform can be correct or catastrophic For a photograph of a diseased cassava leaf, a horizontal flip is label-preserving: a mirrored leaf is still a diseased leaf. For a photograph of an identity document, a horizontal flip produces mirrored text, which in the lender's data is a *tamper signal*. Including it in G trains the model to be invariant to exactly the evidence the tamper detector exists to find. Augmentation policy is a domain decision. Write it down, justify each transform in one line, and put the justification in the model card.

Crop disease photograph

a mirrored leaf is
still a diseased leaf

horizontal flip: correct

ID document photograph

mirrored text is a *tam-*
per signal, yet the la-
bel says "genuine"

**horizontal flip: teaches
the model to ignore fraud**

Figure 7.34: The same transformation, applied to two domains, with opposite consequences.

For the document classifier, sensible defaults are small rotations of about $\pm 10^\circ$, since borrowers photograph documents at an angle; modest brightness and contrast jitter, since lighting varies enormously; slight perspective warping; and random occlusion, which mimics fingers, glare and shadow. Flips, quarter-turns and heavy colour shifts are excluded, and the exclusion is documented.

```
augment = keras.Sequential([
    layers.RandomRotation(0.03),          # about +/- 10 degrees
    layers.RandomZoom(0.10),
    layers.RandomBrightness(0.15),
    layers.RandomContrast(0.15),
    # NO RandomFlip for documents: mirrored text is a tamper signal
], name="augmentation")
```

Listing 7.3: Augmentation as layers inside the model: active during fit, inert during predict.

One connection back to Chapter 3 is worth making explicitly. There we rebalanced tabular data with SMOTE, interpolating between minority examples. Interpolating between two images in pixel space produces a ghost, not a plausible photograph, so SMOTE has no vision analogue. In vision, augmentation of the minority class does the job SMOTE does for tabular data, and class weighting does the rest.

7.4.6 Reading the training curves

Fine-tuning fails in a small number of recognisable ways, and Figure 7.35 shows three of them. Healthy training has both losses falling with a small and stable gap. Overfitting shows the training loss continuing to fall while the validation loss turns upwards, and calls for stronger augmentation, more dropout, freezing more of the backbone, or simply stopping earlier. Under-training shows both curves flat and high, and calls for unfreezing more, raising the learning rate, or training longer. A fourth signature, not drawn here, is the abrupt collapse at the unfreeze point of Figure 7.32, which is almost always the BatchNorm trap rather than anything to do with the learning rate.

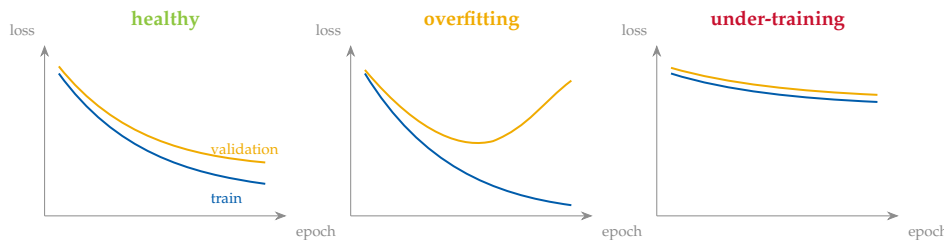


Figure 7.35: The three diagnoses you will make most often while fine-tuning. Each has a distinct remedy, listed in the text.

7.5 Evaluation, Explainability and Deployment

A model that fine-tunes cleanly is not yet a model you can put in front of a customer. Three obligations stand between the two, and they are the subject of this section. The metric and the operating threshold must reflect what the errors actually cost, rather than what `argmax` assumes (Section 7.5.1). The prediction must be explicable to a reviewer and an auditor, which for a convolutional network means being able to say which pixels carried the decision (Section 7.5.2). And the deployed artefact must satisfy the Data Protection Act 2019, fit on the handset of a customer who pays for every megabyte, and keep working when the camera hardware changes underneath it (Section 7.5.3).

7.5.1 Evaluation under imbalance and asymmetric cost

Vision datasets are as imbalanced as credit datasets, and the consequences are identical. Figure 7.36 shows a five-class distribution in which the majority class holds 61% of the images: a constant predictor scores 61% accuracy and is worthless. The protocol of Chapter 3 carries over without modification. Report the per-class confusion matrix, per-class recall, and

macro-averaged F_1 ,

$$\text{macro-}F_1 = \frac{1}{K} \sum_{k=1}^K \frac{2P_k R_k}{P_k + R_k}, \quad (7.10)$$

which weights every class equally so that a rare but costly class cannot be quietly ignored.

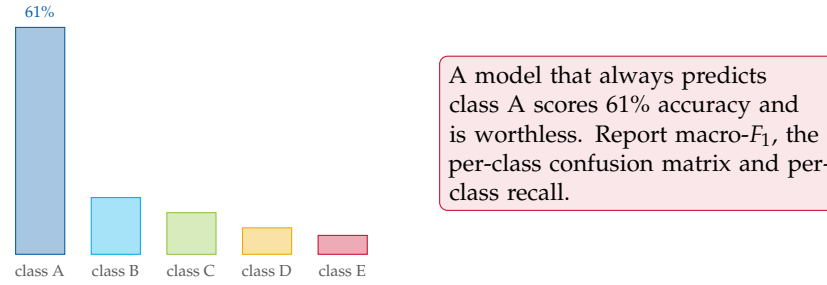


Figure 7.36: Class imbalance in an image corpus behaves exactly as it did in Chapter 3. Accuracy remains the wrong headline metric.

Class weighting is the vision equivalent of the reweighting used in Chapter 3: pass `class_weight` to `fit`, with the weights that `compute_class_weight` returns in "balanced" mode, and let the minority-class augmentation of Section 7.4.5 do the rest.

For a binary screening decision such as the tamper detector, the cost matrix fixes the operating threshold directly.

Proposition 7.5 (Cost-optimal threshold). *Let $\hat{p}$ be a calibrated estimate of $\Pr(y = 1 \mid \mathbf{x})$, and let a false negative cost C_{FN} and a false positive cost C_{FP} , with correct decisions costing nothing. The expected-cost-minimising rule predicts 1 when*

$$\hat{p} > t^* = \frac{C_{FP}}{C_{FP} + C_{FN}}. \quad (7.11)$$

Proof. Predicting 1 has expected cost $(1 - \hat{p})C_{FP}$; predicting 0 has expected cost $\hat{p}C_{FN}$. The former is smaller exactly when $C_{FP} < \hat{p}(C_{FP} + C_{FN})$. $\square$

With the lender's figures, $C_{FN} = \text{KES } 10,000$ and $C_{FP} = \text{KES } 800$, so $t^* = 800/10,800 \approx 0.074$: escalate a document to human review whenever the tamper probability exceeds about seven percent. That is far below the 0.5 a naive `argmax` applies, and it is meaningful only if the probabilities are calibrated. Deep networks are systematically over-confident, so the temperature scaling of Chapter 6 should be fitted on the validation fold and frozen alongside the weights before (7.11) is applied.

7.5.2 Explaining a convolutional prediction

A credit committee cannot act on "the network said so", and neither can a rejected applicant. Gradient-weighted class activation mapping (Selvaraju et al., 2017) produces a coarse spatial map showing which regions of the input drove a particular class score. Writing A^k for the k -th feature map of the last convolutional layer and y^c for the pre-softmax score of class c ,

$$\alpha_k^c = \frac{1}{H'W'} \sum_{i,j} \frac{\partial y^c}{\partial A_{i,j}^k}, \quad L^c = \text{ReLU} \left(\sum_k \alpha_k^c A^k \right), \quad (7.12)$$

after which L^c is upsampled to input resolution and overlaid as a heat map (Figure 7.37).

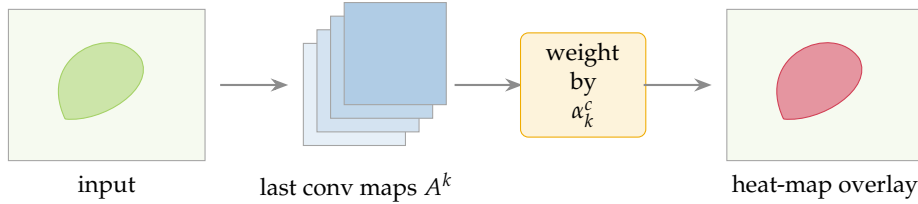


Figure 7.37: Grad-CAM weights the last convolutional feature maps by the spatially pooled gradient of the class score, then upsamples the result.

Why should a gradient-weighted sum of feature maps mean anything? Because on the architectures we actually use it is not an approximation at all. Suppose the head is global average pooling followed by a linear layer, so that $y^c = \sum_k w_k^c \bar{A}^k + b^c$ with $\bar{A}^k = (H'W')^{-1} \sum_{i,j} A_{i,j}^k$. Then every spatial location enters the average with the same weight, so $\partial y^c / \partial A_{i,j}^k = w_k^c / (H'W')$ for all (i,j) , whence $\alpha_k^c = w_k^c / (H'W')$ and

$$L^c = \frac{1}{H'W'} \text{ReLU} \left(\sum_k w_k^c A^k \right).$$

That is exactly the class activation map: the feature maps weighted by the classifier's own class weights, up to a positive constant that the normalisation removes. Grad-CAM is therefore an exact read-out of the linear head for global-average-pooling architectures, and a principled generalisation of it when the head is more complicated. The bias contributes nothing, as one would hope: a class-independent offset carries no spatial information.

```
import tensorflow as tf

def grad_cam(model, image, class_index, last_conv="conv5_block3_out"):
    grad_model = tf.keras.Model(
        model.inputs, [model.get_layer(last_conv).output, model.output])
```

```

with tf.GradientTape() as tape:
    fmaps, preds = grad_model(image[None, ...])
    score = preds[:, class_index]
    alpha = tf.reduce_mean(tape.gradient(score, fmaps), axis=(1, 2))
    cam = tf.nn.relu(tf.einsum("bk,bijk->bij", alpha, fmaps))
    return cam / (tf.reduce_max(cam) + 1e-8)

```

Listing 7.4: Grad-CAM in `tf.keras`.**Pitfall**

Shortcut learning A model can reach high validation accuracy by keying on the data-collection process rather than on the phenomenon. If every fraudulent document in the training set happened to be scanned on the same device, the model will learn that device's compression artefacts; if the clinic that photographed the diseased plants used a different phone from the one that photographed the healthy ones, the model will learn the phone. Both models collapse on deployment day, and neither collapse is visible in the validation metrics. Grad-CAM is the cheapest available audit: sample the model's most confident predictions, and especially its confident *errors*, and check whether the heat map falls on the evidence or on the background (Figure 7.38).

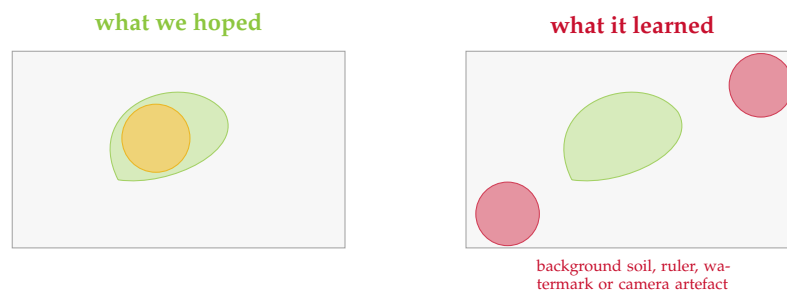


Figure 7.38: The failure Grad-CAM exists to catch. Validation accuracy cannot distinguish these two models; the heat map can.

7.5.3 Fairness, governance and deployment

Aggregate accuracy hides who is failing. Buolamwini and Gebru (2018) evaluated commercial face-classification products that reported near-perfect overall accuracy and found error rates above 30% for darker-skinned women. The aggregate number was never wrong; it was answering the wrong question, and Figure 7.39 shows the shape of the problem it concealed.

The obligation this creates is procedural rather than merely ethical. Every evaluation of a face or document model must be stratified by skin tone,

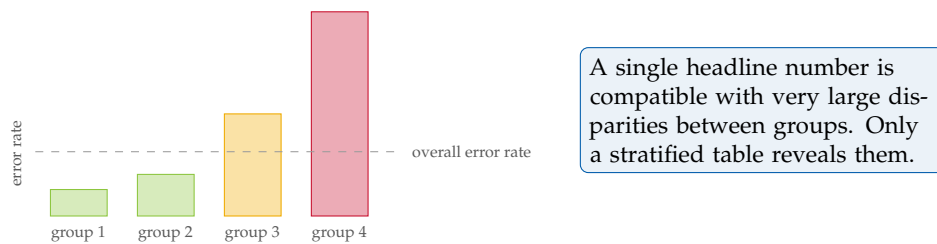


Figure 7.39: The pattern found by Buolamwini and Gebru (2018). The obligation that follows is procedural: stratify every evaluation and publish the table.

age band, gender, device tier and lighting condition, and it is the stratified table, not the headline number, that belongs in the model card.

Kenya Context

A system that processes facial images and identity documents sits at the strictest end of Kenyan data-protection law, and the requirements translate into specific artefacts rather than general good intentions.

The **Data Protection Act, 2019** classifies biometric data, including facial images used for identification, as *sensitive personal data* (section 2), which requires an explicit lawful basis and, in practice, consent recorded at onboarding. Purpose limitation and data minimisation (sections 25 and 30) mean that selfies collected to verify identity may not be quietly reused to train a general-purpose face model: that is a new purpose requiring a new basis. Section 35 confers a right not to be subject to a decision based solely on automated processing where the decision produces legal or similarly significant effects, so a rejected applicant must have access to a **human review path**. Sections 44 to 46 restrict cross-border transfer of sensitive personal data, which makes hosting inference on an overseas API a compliance decision and not merely an engineering one.

The **CBK Digital Credit Providers Regulations, 2022** require licensed lenders to maintain model governance, auditability and complaint handling for automated decisions.

The practical consequence is a retention schema. For every rejection, persist the model version, a hash of the input, the predicted probabilities, the Grad-CAM artefact of Section 7.5.2 and the reviewer's decision. Chapter 11 turns that list into a monitoring specification.

Size, latency and quantisation

In a market where customers pay for every megabyte they download, model size is a product requirement rather than an engineering preference, and the dominant lever is numerical precision. A model with P parameters

stored at b bits occupies $Pb/8$ bytes, so moving from float32 to int8 divides weight storage by four. If the weights of a tensor lie in $[-r, r]$ and are quantised symmetrically with step $\Delta = 2r/255$, no weight moves by more than $\Delta/2 = r/255$, since rounding changes its argument by at most one half. Figure 7.40 puts the resulting sizes side by side.

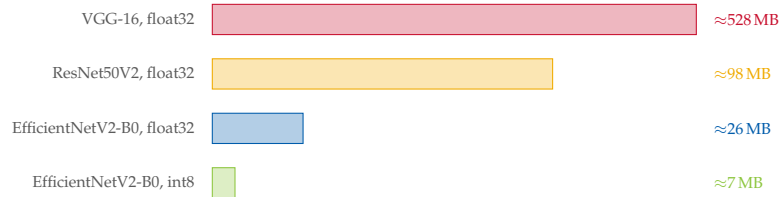


Figure 7.40: Post-training int8 quantisation in TFLite cuts weight storage roughly fourfold and accelerates inference on ARM processors, typically for well under one point of accuracy.

```
converter = tf.lite.TFLiteConverter.from_keras_model(model)
converter.optimizations = [tf.lite.Optimize.DEFAULT]
converter.representative_dataset = representative_gen # ~200 real images
converter.target_spec.supported_ops = [tf.lite.OpsSet.TFLITE_BUILTINS_INT8]
tflite_model = converter.convert()
open("kyc_classifier_int8.tflite", "wb").write(tflite_model)
```

Listing 7.5: Exporting a quantised model for on-device inference.

Finally, and this is where the chapter hands over to Part III, image distributions drift for entirely mundane reasons. A handset generation changes the camera pipeline; the government reissues an identity-card design; the rainy season changes field lighting. None of these is a modelling failure, and none of them is visible without instrumentation. The vision model is one node in the system that Figure 7.41 sketches and that Chapters 10 and 11 build in full.

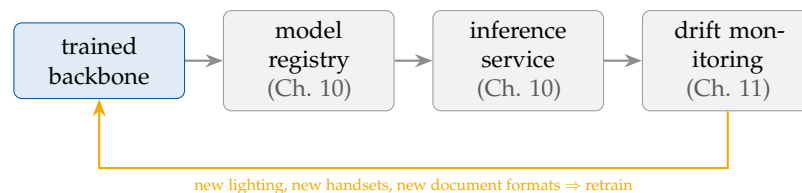


Figure 7.41: The vision model is one node in a larger system. Monitoring closes the loop, and the artefacts required by the Kenya Context box are what make the loop auditable.

Lab 7: A KYC Document Classifier by Transfer Learning (1.5 h)

The notebook `lab07_computer_vision.ipynb` builds the document classifier of the running case study on a synthetic KYC corpus: 12,000 photographs across the five accepted types plus an *unusable* class, and a tamper-detection split. No real document or facial image is distributed, for the reasons of Section 7.5.3.

1. **Setup and shapes (10 min).** Build a `tf.data` pipeline that decodes, resizes to 224×224 and batches, split by capture session so no document straddles the boundary. Compute the first three stages' output shapes and parameter counts by hand from Proposition 7.1, then check `model.summary()`.
2. **Frozen backbone (20 min).** Reproduce Listing 7.1 with ResNet50V2. Record validation macro- F_1 , trainable and frozen parameter counts, and time per epoch — the data-efficiency claim of Section 7.2.5 made concrete.
3. **Augmentation as a domain statement (15 min).** Implement Listing 7.3, justifying each transformation you include and each you exclude. Then add `RandomFlip` deliberately, retrain the tamper detector and report its recall — the argument of Section 7.4.5 observed rather than asserted.
4. **Fine-tuning, and the two silent failures (20 min).** Move to Listing 7.2. Then induce each failure on purpose — omit `preprocess_input`; unfreeze the batch-normalisation layers — and match the curves to Figure 7.32 and Section 7.4.6.
5. **Threshold and audit (15 min).** Fit temperature scaling on the validation fold, then set the tamper detector's threshold from Proposition 7.5 with $C_{FN} = \text{KES } 10,000$ and $C_{FP} = \text{KES } 800$. Report the expected cost at t^* and at 0.5. Grad-CAM five true and five false positives (Listing 7.4) and name one shortcut.
6. **Ship it (10 min).** Export an int8 model with Listing 7.5. Report file size, single-image CPU latency, the change in macro- F_1 , per-document-type recall and the worst-served segment.

Deliverable: the executed notebook, the augmentation policy as a table of transformations with justifications, the exported `.tflite` file, and a one-page model card giving the operating threshold, the segment breakdown and the image retention period.

Exercises

1. **Shapes and parameters.** A network receives $224 \times 224 \times 3$ inputs and applies, in order:

```
Conv2D(64, 7, strides=2, padding="same")
MaxPooling2D(3, strides=2, padding="same")
Conv2D(128, 3, padding="same")
MaxPooling2D(2)
Conv2D(256, 3, padding="same")
MaxPooling2D(2)
GlobalAveragePooling2D
Dense(5, softmax)
```

Using Proposition 7.1, give the output shape and parameter count of every layer and the total. Then state what changes if the input is $384 \times 384 \times 3$ instead, and why.

2. **Receptive field.** Using (7.4), compute the receptive field after each layer of the stack with (f, s) pairs $(7, 2), (3, 2), (3, 1), (3, 1), (3, 2), (3, 1)$. How many further 3×3 stride-one layers would be needed to exceed a receptive field of 100 pixels?
3. **Equivariance with stride.** Using Proposition 7.2, show that a stride-2 convolution is *not* equivariant to a one-pixel translation, by exhibiting a one-dimensional input, a kernel and a shift for which the outputs are not translates of one another. Then explain the practical consequence for a model that classifies documents photographed by hand, and name two mitigations from this chapter.
4. **Bottleneck economics.** A block maps $C = 512$ channels to 512. Compare the parameter count of a plain 3×3 convolution with that of a bottleneck of width $b = 128$. At what b do the two costs coincide? Comment on why ResNet chooses $b = C/4$ rather than the break-even value.
5. **Compound scaling.** Take $\alpha = 1.2, \beta = 1.1, \gamma = 1.15$. (a) Verify the constraint in (7.7). (b) Using Proposition 7.4, compute d, w, r and the FLOP multiplier for $\phi = 1, 2, 3$. (c) A deployment budget allows eight times the FLOPs of B0. What ϕ does that permit, and what input resolution does it imply if B0 uses 224? (d) In one short paragraph, say why this arithmetic is not by itself an argument for scaling up on a five-class document task.
6. **Grad-CAM and CAM.** Section 7.5.2 shows that for a global-average-pooling head followed by a linear layer, Grad-CAM reduces to the class activation map. Reproduce that derivation, explain why the bias

term b^c plays no part, and state precisely what changes if the head instead contains a hidden ReLU layer.

7. **Cost-optimal thresholds.** The tamper detector outputs a calibrated probability. Using $C_{\text{FN}} = \text{KES } 10,000$ and $C_{\text{FP}} = \text{KES } 800$: (a) compute t^* ; (b) if 4% of submitted documents are tampered, compute the expected cost per application of the always-approve policy and of the always-reject policy; (c) a colleague proposes reporting accuracy at $t = 0.5$. Explain in two or three sentences why that number is misleading here.
8. **The BatchNorm trap, quantified.** A batch-normalisation layer has ImageNet running statistics $\mu_R = 0$, $\sigma_R = 1$ and learned parameters $\gamma = 1.2$, $\beta = -0.3$. On a target-domain batch the statistics are $\mu_B = 0.8$, $\sigma_B = 2.5$. For a pre-activation $z = 1.5$, compute the layer output in inference mode and in training mode with $\epsilon = 0$, and the difference. Explain how a discrepancy of this size, repeated across the 53 normalisation layers of ResNet-50, produces the collapse of Figure 7.32 rather than a gradual degradation.
9. **Augmentation policy.** For each of the following transformations, state whether it belongs in the augmentation set for (i) a crop-disease classifier and (ii) the identity-document classifier, justifying each answer in one sentence: horizontal flip; $\pm 90^\circ$ rotation; $\pm 8^\circ$ rotation; random brightness of $\pm 20\%$; random JPEG compression; random occlusion of a 30×30 patch. Then use the bias–variance decomposition of Section 7.4.5 to explain formally what goes wrong when a transformation that does not preserve the label is included.
10. **Quantisation budget.** EfficientNetV2-B0 has 7.2M parameters; ResNet50V2 has 25.6M. (a) Give the float32, float16 and int8 sizes of each. (b) A product requirement caps the downloaded model at 10 MB. Which combinations of backbone and precision satisfy it? (c) For a weight tensor with values in $[-3, 3]$ quantised symmetrically to 8 bits, compute the step size and the maximum rounding error, and comment on whether that error is likely to matter relative to the label noise already present in the training set.

Key Terms

Backbone	The convolutional feature extractor of a network, excluding the task-specific head; what <code>include_top=False</code> returns.
Bottleneck block	A 1×1 , then 3×3 , then 1×1 triple that reduces channel width before the expensive spatial convolution.
Catastrophic forgetting	Destruction of pre-trained features by large gradients from an untrained head, or by an excessive learning rate at the unfreeze point.
Compound scaling	Joint scaling of depth, width and resolution by a single coefficient ϕ under a fixed compute constraint.
Equivariance	Property whereby transforming the input transforms the output correspondingly, as opposed to leaving it unchanged, which is invariance.
Feature extraction	Transfer regime in which the backbone is frozen and only a new head is trained; a convex problem.
Fine-tuning	Transfer regime in which part of the backbone is unfrozen and trained at a reduced learning rate.
Global average pooling	Reduction of each feature map to its spatial mean, removing the flatten-then-dense parameter tail.
Grad-CAM	Gradient-weighted class activation mapping; a class-discriminative spatial explanation of a convolutional prediction.
Receptive field	The region of the input that can influence a given unit; grows by (7.4).
Residual connection	An identity shortcut added to a block's output, making the identity the default mapping and giving the gradient an undamped path.
Shortcut learning	Reliance on artefacts of the data-collection process rather than on the phenomenon of interest.
Stride	The step, in pixels, between successive kernel placements; divides the equivariance guarantee.
Weight sharing	Application of the same kernel at every spatial position, tying parameters across locations.

Further Reading

Géron (2022), chapter 14, is the closest companion to this chapter and uses the same TensorFlow/Keras stack; it covers convolutional layers, the classical architectures and `keras.applications` in roughly the order followed

here. [Bishop and Bishop \(2024\)](#), chapter 10, gives the more theoretical treatment, framing convolution as a structured hypothesis class and handling equivariance carefully; it is the better source if [Proposition 7.2](#) left you wanting more.

On the architectures themselves, [He et al. \(2016\)](#) is short, readable and worth reading in the original, particularly for the degradation experiments that motivate [Section 7.3.1](#); the follow-up paper on pre-activation blocks gives the cleanest form of [Theorem 7.1](#). [Tan and Le \(2019\)](#) and [Tan and Le \(2021\)](#) develop compound scaling and its refinement. [Yosinski et al. \(2014\)](#) is the empirical basis for the freeze-versus-fine-tune decision and repays reading before you make that decision on your own data.

On evaluation and governance, [Selvaraju et al. \(2017\)](#) introduces Grad-CAM and [Buolamwini and Gebru \(2018\)](#) makes the case that stratified evaluation is not optional; the latter is short and should be read in full. For the regulatory material the primary instruments are [Republic of Kenya \(2019\)](#) and [Central Bank of Kenya \(2022\)](#), and [Huyen \(2022\)](#) treats the deployment and monitoring concerns that [Chapters 10](#) and [11](#) develop in detail.

Part III

**MLOps and Production
Systems**

Appendices

Capstone Project

Solutions to Exercises

This appendix collects worked solutions, one section per chapter. Solutions are deliberately fuller than a marking scheme requires: they model the style of argument, framing before formalism, business units before metrics, honesty caveats stated explicitly, that the assignments and the capstone are graded against.

Chapter 1: Introduction to Machine Learning

Solution to Exercise 1 (Framing). (a) Streaming fraud detection with a 300-alerts/day operations team.

Mitchell specification.

- *T*: score each transaction at initiation, before settlement, and emit an alert for those most likely to be fraudulent.
- *E*: historical transaction logs with fraud labels arriving with a delay of days to weeks, from chargebacks, customer complaints, and investigator dispositions; labels are incomplete (undetected fraud is labelled “legitimate”).
- *P*: fraud value captured (or recall) *at a fixed budget of 300 alerts per day* — operationally, precision@*k* with $k = 300/\text{day}$, weighted by transaction value.

Framing block. **Objective:** minimize net fraud losses subject to the investigation-capacity constraint and a false-positive customer-friction budget. **ML task:** binary classification (or anomaly scoring) on a *stream*; the drift speed of adversarial behaviour argues for online or frequently retrained batch learning (Table 1.4), evaluated prequentially. **Metric:** value-weighted recall at the 300-alert budget; secondary, time-to-detect for new fraud patterns. **Baseline:** the incumbent rules engine — its capture rate at the same alert volume is the number to beat. **Constraints:** per-transaction

scoring latency in the tens of milliseconds; adversarial drift; delayed and censored labels; rollback capability for bad online updates.

Two honesty points earn full marks: the label censoring (undetected fraud contaminates the negatives, so measured recall is optimistic), and the feedback loop — blocking transactions changes which frauds are ever observed, the streaming analogue of the selective-labels problem of [Section 1.4](#).

(b) Thirty-day readmission from clinical pathway data.

Mitchell specification.

- *T*: at discharge, estimate the probability that the patient is readmitted within 30 days.
- *E*: historical admission episodes — diagnoses, procedures, length of stay, vitals trajectories, prior utilization — joined to the observed readmission outcome.
- *P*: recall (or expected cost captured) at the capacity of the post-discharge intervention programme (follow-up calls, home visits); PR-AUC as the threshold-free summary, given a low base rate.

Framing block. **Objective:** allocate a fixed post-discharge intervention capacity to the patients whose readmission it is most likely to prevent. **ML task:** binary classification at the discharge-decision point; batch retraining is adequate (clinical drift is slow relative to fraud). **Metric:** recall at the intervention budget; per-segment calibration, since resource allocation across wards and demographics will be scrutinized. **Baseline:** the incumbent clinical risk score (e.g. a LACE-style index) or current clinician triage. **Constraints:** explainability to clinicians; data-protection compliance for health records; fairness across demographic groups; label subtleties — death within 30 days is a competing risk that must be handled explicitly, not silently coded as “no readmission”.

The deepest framing observation: the deployment intent is *interventional* (we act on high-risk patients), so post-deployment outcome data is shaped by the model’s own decisions — future retraining must account for the intervention, exactly the hidden feedback loop of [Section 1.4](#).

Solution to Exercise 2 (ML or not).

(a) Blocking transactions above a regulatory limit: rule. The requirement is a rule — crisp, complete, auditable, and legally fixed. ML could only approximate it, adding error where zero error is mandated and destroying auditability. Right-hand column of [Table 1.1](#) on every criterion.

(b) SIM-swap account takeover: ML, wrapped in rules. The signal is high-dimensional and behavioural (device changes, dormancy-then-drain patterns, atypical beneficiaries), and the adversary adapts, so any static rulebook rots within months — the left-hand column on every criterion.

The mature design keeps hard rules as guardrails (e.g. mandatory step-up authentication within N hours of a SIM change) with an ML risk score ranking the grey zone; the rules also generate the labelled outcomes that train the model, the “rules first, ML second” pattern of [Section 1.2.3](#).

(c) Routing support tickets by language: ML, but off the shelf. Hand-written rules (character ranges, keyword lists) are brittle across code-switching and short texts, so learning is justified; but language identification is a solved commodity problem, and a lightweight pretrained classifier meets the accuracy, latency, and cost bar. The build-or-not lesson: “use ML” does not imply “train your own” — the total cost-of-ownership argument of [Table 1.1](#) favours the smallest maintained artifact that clears the requirement.

Solution to Exercise 3 (ERM: mean and median as optimal constants).

Squared error. For a constant predictor c , the empirical risk is

$$J(c) = \frac{1}{n} \sum_{i=1}^n (y_i - c)^2, \quad J'(c) = -\frac{2}{n} \sum_{i=1}^n (y_i - c),$$

and setting $J'(c) = 0$ gives $c^* = \frac{1}{n} \sum_i y_i = \bar{y}$, the sample mean; $J''(c) = 2 > 0$ confirms the minimum.

Absolute error. $G(c) = \frac{1}{n} \sum_i |y_i - c|$ is convex and piecewise linear, so we argue by subgradient. For c not equal to any y_i ,

$$G'(c) = \frac{1}{n} \left(\#\{i : y_i < c\} - \#\{i : y_i > c\} \right),$$

which is negative while fewer than half the points lie below c and positive once more than half do. The minimum is therefore attained where the two counts balance: at the sample median (for even n , anywhere on the interval between the two middle order statistics — the minimizer need not be unique).

Connection to [Section 1.6.2](#). Minimizing RMSE trains the model toward the conditional *mean*, which is pulled by extreme outcomes; minimizing MAE trains it toward the conditional *median*, which is robust to them. Choosing between [Eq. \(1.4\)](#) is therefore choosing *which summary of the target distribution the model learns* — a business decision (how do errors actually cost?) dressed as a metric choice.

Solution to Exercise 4 (Cost-sensitive thresholds).

Two-way derivation. For an applicant with calibrated default probability $\hat{p}$, compare conditional expected costs:

$$\mathbb{E}[C \mid \text{approve}] = \hat{p} c_{\text{FN}}, \quad \mathbb{E}[C \mid \text{decline}] = (1 - \hat{p}) c_{\text{FP}}.$$

Decline exactly when $\hat{p} c_{\text{FN}} > (1 - \hat{p}) c_{\text{FP}}$; rearranging,

$$\hat{p} > \frac{c_{\text{FP}}}{c_{\text{FP}} + c_{\text{FN}}} = t^*,$$

which is Eq. (1.6). The derivation *uses* calibration: $\hat{p}$ must be interpretable as a probability for the expectations to be meaningful, which is why calibration is a precondition, not a nicety.

Three-way extension. Add the action *refer*, costing a review fee c_R and (idealizing) resulting in the correct decision, so it incurs no error cost. The three conditional expected costs are $\hat{p} c_{FN}$ (approve), $(1 - \hat{p}) c_{FP}$ (decline), and c_R (refer). Comparing pairwise:

- approve beats refer iff $\hat{p} c_{FN} < c_R$, i.e. $\hat{p} < t_L := c_R / c_{FN}$;
- decline beats refer iff $(1 - \hat{p}) c_{FP} < c_R$, i.e. $\hat{p} > t_U := 1 - c_R / c_{FP}$.

The optimal policy is therefore: *approve* if $\hat{p} < t_L$, *refer* if $t_L \leq \hat{p} \leq t_U$, *decline* if $\hat{p} > t_U$. The referral band is non-empty iff $t_L < t_U$, i.e.

$$c_R < \frac{c_{FN} c_{FP}}{c_{FN} + c_{FP}},$$

the harmonic combination of the two error costs; cheap review widens the band, expensive review collapses it to the single threshold t^* — which, one checks, always lies inside $[t_L, t_U]$ when the band exists. This is exactly the “ $\hat{p}$ near t^* routes to a human” edge of Fig. 1.9, now with the band derived rather than asserted. (A fuller model would let the human err too; the band then narrows in proportion to reviewer accuracy, a good extension discussion.)

Solution to Exercise 5 (Leakage audit).

(a) Number of repayment reminders sent: target leakage. Reminders exist only after origination and are triggered by delinquency — the feature is a downstream *consequence* of the label. At application time it is identically empty, so a model trained with it learns to read the outcome off its own paperwork and collapses in production (Definition 1.3).

(b) Median mobile-money balance over the 90 days before application: safe — conditional on discipline. The window $[t_0 - 90d, t_0)$ respects the as-of contract of Table 1.6. The residual risks to state for full marks: the feature must be computed from event data as it stood at t_0 (late-arriving corrections or restated ledgers re-introduce temporal leakage), and the training-time computation must share code with the serving-time computation or training/serving skew replaces leakage as the failure mode.

(c) Standard deviation of balance over the loan term: temporal leakage. The loan term lies strictly *after* the origination decision the model is meant to inform. The feature is computable in the historical warehouse (which is why it slips into training sets) but can never be computed at prediction time. Textbook violation of the as-of rule.

(d) Merchant-ID one-hot with random row-wise splits: group leakage. Repeat merchants appear on both sides of a random split, so the identifier

lets the model memorize *who* defaulted rather than learn *which behaviours* predict default; validation scores inflate and the model is useless for the new applicants it exists to score (whose IDs it has never seen). Remedies: split by merchant group *and* time; drop the raw identifier in favour of behavioural aggregates with proper as-of semantics; if entity-level history is genuinely predictive, encode it as backdated features (e.g. prior loans, prior repayment record as of t_0), which are legitimate.

Solution to Exercise 6 (Bias–variance: the vanishing cross-terms).

Setup as in Section 1.9.1: $y = g(x) + \varepsilon$ with $\mathbb{E}[\varepsilon] = 0$, $\text{Var}(\varepsilon) = \sigma^2$, ε independent of the training sample S ; $\hat{f}$ is trained on S ; $\bar{f}(x) = \mathbb{E}_S[\hat{f}(x)]$. Fix x and expand (all expectations over S and ε jointly):

First cross-term. Expanding $\mathbb{E}[(g + \varepsilon - \hat{f})^2] = \mathbb{E}[(g - \hat{f})^2] + 2\mathbb{E}[\varepsilon(g - \hat{f})] + \mathbb{E}[\varepsilon^2]$, the middle term factorizes because ε is independent of S and $\hat{f}$ is a function of S (and $g(x)$ is a constant for fixed x):

$$\mathbb{E}[\varepsilon(g - \hat{f})] = \mathbb{E}[\varepsilon] \mathbb{E}_S[g - \hat{f}] = 0 \cdot \mathbb{E}_S[g - \hat{f}] = 0.$$

Independence is used to factorize; the zero mean of ε kills the product. This leaves $\mathbb{E}[(g - \hat{f})^2] + \sigma^2$.

Second cross-term. Insert and subtract $\bar{f}$: $(g - \hat{f}) = (g - \bar{f}) + (\bar{f} - \hat{f})$, so

$$\mathbb{E}_S[(g - \hat{f})^2] = (g - \bar{f})^2 + 2(g - \bar{f}) \mathbb{E}_S[\bar{f} - \hat{f}] + \mathbb{E}_S[(\bar{f} - \hat{f})^2].$$

Here $(g - \bar{f})$ is a deterministic quantity (both terms are fixed numbers once x is fixed), so it exits the expectation; and by the *definition* of $\bar{f}$, $\mathbb{E}_S[\bar{f} - \hat{f}] = \bar{f} - \mathbb{E}_S[\hat{f}] = 0$. The cross-term vanishes, leaving $\text{Bias}^2 + \text{Variance} + \sigma^2$, which is Eq. (1.7). Summary of where each assumption bites: independence of ε and S kills the first cross-term; the definition of $\bar{f}$ as the S -average kills the second.

Solution to Exercise 7 (Splits, empirically).

A model answer on synthetic data with controlled drift (students' numbers on real data will differ in size, not in sign):

```
import numpy as np
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import average_precision_score
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

rng = np.random.default_rng(8401)
n, d = 30_000, 10
t = np.sort(rng.uniform(0, 1, n))          # "application date" in [0,1]
X = rng.normal(size=(n, d))
w = np.array([2.0, -1.5, 1.0, 0.5, 0, 0, 0, 0, 0, 0])
```

```

drift = 2.5 * t                                     # concept drifts over time
logits = X @ w + drift * X[:, 3] - 3.2
y = rng.binomial(1, 1 / (1 + np.exp(-logits)))

model = Pipeline([("s", StandardScaler()),
                  ("m", LogisticRegression(max_iter=2_000,
                                           class_weight="balanced"))])

# (a) Random shuffled split -- time ignored
Xa_tr, Xa_te, ya_tr, ya_te = train_test_split(
    X, y, test_size=0.25, stratify=y, random_state=8401)
pa = model.fit(Xa_tr, ya_tr).predict_proba(Xa_te)[: , 1]

# (b) Temporal split -- train on the past, test on the future
cut = int(0.75 * n)                                # rows already time-ordered
pb = (model.fit(X[:cut], y[:cut])
      .predict_proba(X[cut:]))[: , 1]

print(f"random split PR-AUC: {average_precision_score(ya_te, pa):.3f}")
print(f"temporal split PR-AUC: {average_precision_score(y[cut:], pb):.3f}")

```

Listing B.1: Random vs. temporal split under drift: the random split reports an optimistic number that no deployment can honour.

Direction and explanation. The random split scores higher. Under shuffling, training and test rows are interleaved in time, so the training set contains examples from the *same regime* as every test example — the model has effectively seen the future, and the drift term never has to be extrapolated. The temporal split forces genuine forward prediction into a drifted regime, which is exactly what deployment does (Fig. 1.6). **The honest number is the temporal one;** the gap between the two is not noise but a first, free estimate of how much drift will cost in production — worth reporting in the decision memo of Section 1.10.

Solution to Exercise 8 (Fairness).

Let F_A, F_B be the score CDFs of the calibrated model $\hat{p}$ in the two regions, with base rates $\pi_A = \mathbb{E}[\hat{p} \mid A] = 0.03$ and $\pi_B = \mathbb{E}[\hat{p} \mid B] = 0.09$ (calibration plus the tower property force the group mean score to equal the group base rate). Approval is $\hat{p} \leq t$ for a single global t .

Impossibility across thresholds. Demographic parity at *every* threshold would mean $F_A(t) = F_B(t)$ for all t , i.e. identical score distributions — hence identical means, contradicting $0.03 \neq 0.09$. So there is no global-threshold policy family satisfying parity uniformly.

The single-threshold case. At one fixed t , parity requires the accident $F_A(t) = F_B(t)$ at that exact point. But any calibrated model that actually uses the risk information distinguishing the regions places more probability mass at higher scores for region B (in the natural case, F_B is stochastically dominated by — i.e. sits below — F_A), giving $F_B(t) < F_A(t)$: region B's approval rate is strictly lower at every useful threshold. Even in contrived

cases where the CDFs cross at some t , the coincidence is knife-edge — it fails for neighbouring thresholds and evaporates at the first retrain — so it cannot ground a policy. The structural conclusion: with unequal base rates, a calibrated score plus a single global threshold *cannot* deliver demographic parity; achieving parity would require group-specific thresholds, which raises its own legal and ethical questions, or abandoning calibration — the conflict promised in [Section 1.11](#).

Proposal to the regulator. A defensible position for the running case: retain per-group *calibration* (an uncalibrated score corrupts every downstream decision through [Eq. \(1.6\)](#)) and adopt *equal opportunity* — equal true-positive rates among repayers — as the operative criterion, i.e. good borrowers in both regions face the same approval odds; then justify any residual approval-rate gap by the base-rate difference while committing to close it upstream (product design, data enrichment for the thin-file region). **Post-deployment monitoring:** per-region approval rate, TPR/FPR among matured labels, calibration curves, and their drift over time — the fairness axis of the Week-13 monitors, reported segment-by-segment as [Section 1.11](#) prescribes.

[Solution to Exercise 9 \(Hybrid architecture\).](#)

A full-marks answer reproduces the topology of [Fig. 1.9](#) with the component classification made explicit:

- **Classical ML (the decision core).** The gradient-boosted or logistic scoring model with its leakage-safe feature pipeline ([Listing 1.5](#)), served behind the versioned API of [Listing 1.7](#). It alone emits $\hat{p}$; it is trained, calibrated, validated, and monitored under the discipline of Chapters 1–4 and 11–13.
- **LLM components (the language edges).** (i) A RAG service over lending policy, KYC regulation, and product documents, answering underwriter questions with citations; (ii) an extraction model converting unstructured inputs (statements, free-text loan purpose, chat) into *typed features* that enter the classical pipeline — validated like any other upstream data source; (iii) an agent orchestrating the workflow: gather data, call the scoring API, draft the reason-coded communication.
- **Human review.** Applications with $\hat{p} \in [t_L, t_U]$ — the referral band derived in Solution 4 — route to a credit officer, with the agent’s brief attached.

Where calibration must be preserved end-to-end — the marking crux: the probability $\hat{p}$ must flow *numerically unaltered* from the scorer through the API to the thresholding policy and into any human-facing brief. The

agent may *explain* the score; it must never *re-estimate*, round, re-rank, or paraphrase it into a different number, and no LLM output may feed back into the decision path — the moment a stochastic text generator sits between the calibrated score and the threshold of [Eq. \(1.6\)](#), the optimality of the policy, the audit trail, and the fairness monitoring of Solution 8 are all void. Diagrammatically: every arrow into the approve/refer/decline node originates at the classical scorer; LLM arrows terminate at humans or at the scorer's *inputs*, never at its output.

Chapter 2: Data Preparation and Feature Engineering

Solution to Exercise 2.1. Consider the electronic medical records of a network of Kenyan outpatient clinics.

MCAR. A batch of laboratory results is lost when the connection between the lab analyser and the records system fails for an afternoon. Which patients' rows are affected is determined by the timing of the outage, not by the patients or their values: $P(M \mid X) = P(M)$ ([Definition 2.1](#)). Mean imputation of the missing lab values yields an *unbiased* estimate of the population mean (the missing rows are a random sample), although it understates the variance (Exercise 2.3).

MAR. Blood-pressure readings are missing far more often at rural dispensaries that lack a working monitor than at urban clinics. Conditional on the *observed* facility-type column, missingness carries no further information about the reading itself ([Definition 2.2](#)). Unconditional mean imputation is *biased* whenever blood pressure differs systematically by facility type (urban and rural patient populations differ): the imputed mean is dominated by urban readings. Imputation that conditions on facility type — a group-wise mean, or the model-based imputers of the chapter — removes the bias.

MNAR. HIV-test results are missing because some patients decline testing, and the propensity to decline is associated with the patient's own (unobserved) status and risk perception: $P(M \mid X)$ depends on X_{mis} ([Definition 2.3](#)). Mean imputation is *biased*, and no imputation scheme built from the observed data alone removes the bias; per [Remark 2.1](#), the honest treatment records the refusal itself as an indicator feature and interprets downstream estimates with the selection mechanism in mind.

In summary, mean imputation is unbiased for the mean only under MCAR; under MAR it requires conditioning on the drivers of missingness; under MNAR it is biased regardless.

Solution to Exercise 2.2. Let $R_i \in \{0, 1\}$ indicate that row i is a complete case, and let X_i be its (complete-data) value. MCAR states $P(R \mid X) = P(R)$,

i.e. $R \perp X$. The density of X among complete cases is, by Bayes' rule,

$$f(x | R = 1) = \frac{P(R = 1 | X = x) f(x)}{P(R = 1)} = \frac{P(R = 1) f(x)}{P(R = 1)} = f(x),$$

where the middle equality uses independence. The complete cases are therefore distributed identically to the full population — a random subsample. Consequently, with $n_{cc} = \sum_i R_i$ the (random) number of complete cases,

$$\mathbb{E} \left[\frac{1}{n_{cc}} \sum_{i: R_i=1} X_i \mid n_{cc} > 0 \right] = \mathbb{E}[X],$$

by taking the expectation first conditionally on the pattern $(R_1, \dots, R_n)$ — under which each retained X_i has mean $\mathbb{E}[X]$ by independence — and then over the pattern. The complete-case mean is unbiased; the only cost of MCAR missingness is the loss of efficiency from the reduced sample size. $\square$

Solution to Exercise 2.3. With point-estimate imputation (`sample_posterior=False`), every imputed entry is set to the conditional-mean prediction $\hat{\mathbb{E}}[X_{\text{mis}} | X_{\text{obs}}]$ of the final regression sweep of Eq. (2.2). Decompose the true variance by the law of total variance:

$$\text{Var}(X) = \underbrace{\mathbb{E}[\text{Var}(X | X_{\text{obs}})]}_{\text{residual spread}} + \underbrace{\text{Var}(\mathbb{E}[X | X_{\text{obs}}])}_{\text{explained spread}}.$$

The imputed values realise only the second term: they sit exactly on the conditional-mean surface, with zero residual spread. The completed column's sample variance therefore converges to $\text{Var}(\mathbb{E}[X | X_{\text{obs}}])$, understating $\text{Var}(X)$ by the entire residual component $\mathbb{E}[\text{Var}(X | X_{\text{obs}})]$, and the understatement grows with the missing fraction.

Downstream, any procedure that treats the completed data as if observed — ordinary least squares, a t -test, a confidence interval for a feature mean — inherits standard errors that are too small: intervals are too narrow and p -values anti-conservative, because the analysis pretends the imputed entries carry the same information as real measurements. The remedies are those of proper multiple imputation: draw fills from the posterior (`sample_posterior=True`), create M completed datasets, and combine estimates with Rubin's rules, which add the between-imputation variance the point-estimate shortcut discards.

Solution to Exercise 2.4. Two mechanisms, both consistent with the chapter, explain a *drop* in AUC after trimming $|z| > 3$ rows.

(a) *The outliers were the signal.* In credit risk the extreme tail of behaviour is disproportionately informative about default: erratic whale deposits, balance-inflation spikes, abnormal withdrawal bursts (see the pitfall in

Section 2.3). Deleting $|z| > 3$ rows removes a non-random, minority-class-enriched slice of the training data — defaulters exhibit extreme behaviour more often than repayers — so the model loses precisely the discriminative region of the feature space, and its ranking quality on an untrimmed evaluation set falls.

(b) *The analyst created a training–serving mismatch.* Rows were removed from the *training* data, but the validation/test data (and, eventually, production traffic) still contain extreme rows. The model is now evaluated on a region of the input space it was never fitted on — a self-inflicted violation of the distribution-overlap requirement of **Section 2.6**. A subsidiary aggravating factor: on heavy-tailed transaction data the Gaussian $|z| > 3$ rule flags a large fraction of perfectly legitimate rows (the z-score’s μ and σ are themselves inflated by the tail), so the deletion is both extensive and systematically targeted at high-activity customers.

Solution to Exercise 2.5. From Eq. (2.7), the maximum maps to

$$x'_{\max} = \frac{x_{\max} - x_{\min}}{x_{\max} - x_{\min}} = 1$$

identically — the scaled value of the largest point is 1 whatever its magnitude, which is exactly why a single extreme outlier cannot be “detected” in the scaled data: it is normalised into the boundary.

Let m be the median. Its scaled value is $m' = (m - x_{\min}) / (x_{\max} - x_{\min})$. Replace the maximum by $10 x_{\max}$ (all other order statistics unchanged); the new scaled median is

$$m'' = \frac{m - x_{\min}}{10 x_{\max} - x_{\min}}, \quad \frac{m''}{m'} = \frac{x_{\max} - x_{\min}}{10 x_{\max} - x_{\min}}.$$

For $x_{\min} = 0$ this ratio is exactly $1/10$: the median’s scaled value collapses by a factor of ten, and with it the entire bulk of the distribution is crushed toward 0 while the outlier alone occupies the top of the unit interval. This is the fragility that motivates the robust scaler of **Eq. (2.8)**.

Solution to Exercise 2.6. Within level c , model each target as conditionally Bernoulli, $y_i \mid \theta_c \sim \text{Bernoulli}(\theta_c)$, and place the conjugate prior

$$\theta_c \sim \text{Beta}(\alpha, \beta), \quad \alpha = m \bar{y}, \quad \beta = m(1 - \bar{y}),$$

so that the prior mean is $\alpha / (\alpha + \beta) = \bar{y}$ (the global rate) and the prior strength — the number of pseudo-observations — is $\alpha + \beta = m$. Observing n_c rows of level c with $s_c = \sum_i y_i = n_c \bar{y}_c$ successes, conjugacy gives the posterior $\theta_c \mid \text{data} \sim \text{Beta}(\alpha + s_c, \beta + n_c - s_c)$, whose mean is

$$\mathbb{E}[\theta_c \mid \text{data}] = \frac{\alpha + s_c}{\alpha + \beta + n_c} = \frac{m \bar{y} + n_c \bar{y}_c}{m + n_c} = \text{TE}(c),$$

which is [Eq. \(2.11\)](#) exactly. The limits confirm the shrinkage behaviour: as $n_c \rightarrow 0$, $\text{TE}(c) \rightarrow \bar{y}$ (an unseen or rare level is pulled entirely to the prior), and as $n_c \rightarrow \infty$, $\text{TE}(c) \rightarrow \bar{y}_c$ (abundant evidence overwhelms the prior). The smoothing parameter m is thus interpretable as the number of global-prior pseudo-counts each level must “outweigh” before its own history dominates — and setting it too small is precisely what lets a level seen once memorise its own label ([Section 2.5.3](#)). $\square$

Solution to Exercise 2.7. *Mechanism.* Individually, a start date and an end date are weak predictors of churn. Jointly they encode the label almost perfectly at extraction time: an employee who has churned has a populated `contract_end_date` lying before the snapshot date, whereas a current employee has a null, sentinel, or future-dated value. Any model able to combine the two features (or a feature and the implicit snapshot date) reconstructs “has this contract already terminated?” — which is the churn label, or a near-deterministic proxy for it. The end date is a *post-outcome field*, written by the HR system as a consequence of the very event being predicted: cause 6 of [Section 2.7.2](#), hiding behind two innocuous-looking columns that pass a univariate correlation screen.

A screen for pairwise leakage. Extend the univariate screen of [Section 2.7](#) to constructed pairs: (i) for every pair of same-typed columns (two dates, two amounts), automatically generate their difference (and, for amounts, their ratio) and rank these derived features by $|\rho(\cdot, y)|$ or mutual information with the target; (ii) as a model-based variant, fit a depth-2 decision tree on each feature pair alone and flag any pair whose standalone AUC approaches the full model’s. Either variant catches start/end-date leakage immediately (the difference is tenure-to-termination; the tiny tree learns “end date < snapshot”). The screen complements, but does not replace, the lineage question — *which system writes this field, and when?* — that identifies post-outcome fields even when their statistical footprint is subtle.

Solution to Exercise 2.8. Assume the hash of [Eq. \(2.10\)](#) assigns each of the K levels independently and uniformly to one of d buckets. Two given levels collide with probability $1/d$, and there are $\binom{K}{2}$ pairs, so by linearity of expectation the expected number of colliding pairs is

$$\mathbb{E}[\text{\#collisions}] = \binom{K}{2} \frac{1}{d} = \frac{K(K-1)}{2d}.$$

For $K = 10^4$ and $d = 256$:

$$\frac{10^4 \times (10^4 - 1)}{2 \times 256} = \frac{99\,990\,000}{512} \approx 1.95 \times 10^5$$

colliding pairs — with an average bucket load of $K/d \approx 39$ levels, so *every* bucket is shared. This is not necessarily fatal (tree models tolerate the induced noise, and the load is spread evenly), but it quantifies the trade-off:

at this cardinality, $d = 256$ buys a fixed 256-column footprint at the price of heavy aliasing, and raising d to 2^{12} – 2^{14} reduces the expected pair collisions by one to two orders of magnitude while remaining far cheaper than a 10^4 -column one-hot encoding.

Solution to Exercise 2.9. *Leakage mechanism.* Support-line contact is not exogenous to repayment trouble. Two concrete routes: (i) borrowers who are already struggling call to ask about extensions, penalties, or restructuring *after* the loan being scored has been disbursed — so if the 7-day window is computed relative to label-observation time (or to “now” in the training warehouse) rather than to the scoring timestamp t , the feature ingests post- t information in direct violation of the as-of rule (Eq. (2.1)); (ii) worse, some inbound calls are *returns of collections outreach* — the collections team calls delinquent borrowers, who call back — making the count a laundered copy of the delinquency flag, exactly the post-outcome pattern of `days_since_collections_call` in Listing 2.11 (cause 6 of Section 2.7.2).

Redesign. Keep the legitimate signal (a customer’s baseline propensity to need support may genuinely correlate with risk) by: fixing the window to end strictly before the scoring timestamp, $[t - 7d, t)$, enforced by the feature platform’s point-in-time semantics (Section 2.8); excluding call types causally downstream of the credit process (collections callbacks, repayment-difficulty queries about an *active* loan), which requires the call-category field and its write-time lineage; and validating the rebuilt feature with the correlation screen — its association with the target should fall from “implausible” to “modest” if the leak, and not the baseline behaviour, was carrying the load.

Solution to Exercise 2.10. A five-sentence rebuttal. First, native NaN routing (Chen and Guestrin, 2016) solves only the mechanical problem of fitting a tree through incomplete rows; it does not answer the diagnostic questions — *why* is the value missing, and is the missingness itself predictive? — that Lab 2’s MCAR/MAR/MNAR audit exists to settle. Second, under MNAR the explicit indicator of Remark 2.1 is a first-class, interpretable feature whose effect can be measured, reviewed for fairness, and explained to a credit officer, whereas a learned default branch direction is an implicit, per-split artefact that cannot. Third, the pipeline must remain model-agnostic: the moment we benchmark against a regulatory logistic scorecard (Eq. (2.13)) or a neural baseline (Chapter 6), models without native handling need the imputation stage anyway, and comparing models across different missing-data treatments confounds the comparison. Fourth, training-serving consistency (Section 2.8) requires that whatever missing-value semantics the model learned be reproduced exactly in the online path — a fitted, serialised imputer inside the Pipeline of Listing 2.12 is a deployable contract, while “the library handles it” is not. Fifth, most of Lab 2’s effort — parsing, validity rules, entity resolution, leakage

hunting — is not imputation at all, and no amount of native NaN support dedupes a customer with three SIM cards.

Chapter 3: Classical Machine Learning and Rigorous Evaluation

Solution to Exercise 3.1. A defensible committee-minute sentence: “*Holding every other feature in the model fixed, applicants with late-night transaction activity have $e^{0.62} \approx 1.86$ times the odds of default of otherwise-identical applicants without it.*” The load-bearing words are *odds* and *holding every other feature fixed*.

The important caveat: the coefficient measures a *conditional association*, not a *causal effect*. It is a partial effect within this particular model and this particular preprocessing pipeline (Chapter 2): add, remove, or re-encode correlated features and the coefficient — and hence the “effect” of late-night activity — can change substantially. It licenses no intervention claim (“discouraging late-night transactions would reduce defaults” does not follow). A secondary caveat worth a footnote in the minute: an odds ratio is not a risk ratio. At the portfolio’s 3% base rate the two are numerically close ($OR \approx RR$ when the outcome is rare), but the identification should still be avoided in a document regulators may read.

Solution to Exercise 3.2. *Derivative of the sigmoid.* With $\sigma(z) = (1 + e^{-z})^{-1}$,

$$\frac{\partial \sigma}{\partial z} = \frac{e^{-z}}{(1 + e^{-z})^2} = \underbrace{\frac{1}{1 + e^{-z}}}_{\sigma(z)} \cdot \underbrace{\frac{e^{-z}}{1 + e^{-z}}}_{1 - \sigma(z)} = \sigma(z)(1 - \sigma(z)).$$

Gradient of the log loss. Write the per-example loss with $\hat{p} = \sigma(z)$, $z = \theta^\top \mathbf{x}$:

$$\ell(\theta) = -y \log \hat{p} - (1 - y) \log(1 - \hat{p}).$$

By the chain rule,

$$\frac{\partial \ell}{\partial z} = -\frac{y}{\hat{p}} \hat{p}(1 - \hat{p}) + \frac{1 - y}{1 - \hat{p}} \hat{p}(1 - \hat{p}) = -y(1 - \hat{p}) + (1 - y) \hat{p} = \hat{p} - y,$$

and since $\partial z / \partial \theta = \mathbf{x}$, $\nabla_{\theta} \ell = (\hat{p} - y) \mathbf{x}$. Averaging over the m training examples gives Eq. (3.4): $\nabla_{\theta} J = \frac{1}{m} \sum_i (\hat{p}_i - y_i) \mathbf{x}_i$. The remarkable cancellation of the $\hat{p}(1 - \hat{p})$ factor is a signature of pairing the sigmoid with its *matched* loss (cross-entropy); the same cancellation reappears at the output layer of every neural classifier in Chapter 6.

Solution to Exercise 3.3. From Eq. (3.11), $F_{\beta} = (1 + \beta^2)PR / (\beta^2 P + R)$.

Limit $\beta \rightarrow 0$: the expression tends to $PR/R = P$ directly.

Limit $\beta \rightarrow \infty$: divide numerator and denominator by β^2 :

$$F_\beta = \frac{(1 + \beta^{-2}) PR}{P + \beta^{-2} R} \xrightarrow{\beta \rightarrow \infty} \frac{PR}{P} = R.$$

So β interpolates between a pure-precision and a pure-recall score, which is exactly why it should be set from the cost structure.

Numerical part. With $P = 0.40$, $R = 0.75$, $\beta = 2$:

$$F_2 = \frac{(1 + 4) \cdot 0.40 \cdot 0.75}{4 \cdot 0.40 + 0.75} = \frac{1.50}{2.35} \approx 0.638,$$

noticeably above $F_1 = 2 \cdot 0.3/1.15 \approx 0.522$ because F_2 rewards this classifier's recall-heavy profile.

Solution to Exercise 3.4. Fix an instance with calibrated probability p of being positive. The two actions have expected costs

$$\mathbb{E}[\text{cost} \mid \text{flag}] = p c_{TP} + (1 - p) c_{FP}, \quad \mathbb{E}[\text{cost} \mid \text{pass}] = p c_{FN} + (1 - p) c_{TN}.$$

Flagging is optimal iff $p c_{TP} + (1 - p) c_{FP} \leq p c_{FN} + (1 - p) c_{TN}$, i.e.

$$(1 - p) (c_{FP} - c_{TN}) \leq p (c_{FN} - c_{TP}).$$

Under the “reasonableness” conditions $c_{FP} > c_{TN}$ and $c_{FN} > c_{TP}$ (each error costs more than the corresponding correct decision — otherwise one action dominates and no threshold exists), both bracketed terms are positive and we may divide:

$$t^* = \frac{c_{FP} - c_{TN}}{(c_{FP} - c_{TN}) + (c_{FN} - c_{TP})}.$$

Setting $c_{TP} = c_{TN} = 0$ recovers Eq. (3.13). Only cost *differences* within each true class matter — adding a constant to a whole row of the cost matrix leaves t^* unchanged. Concretely: if every flag triggers a manual review costing KES 200 regardless of outcome ($c_{TP} = c_{FP, \text{new}} - c_{FP, \text{old}} = 200$ added to the flag column), then with the case-study numbers $t^* = (1000)/(1000 + 9800) \approx 0.093$: review costs push the threshold up, and the scorer should flag slightly less eagerly.

Solution to Exercise 3.5. Let the positive scores be $s_1^+, \dots, s_{m_+}^+$ and the negative scores $s_1^-, \dots, s_{m_-}^-$, and assume first that all scores are distinct. Sort all $m_+ + m_-$ instances by decreasing score and trace the empirical ROC curve from $(0, 0)$: each positive encountered moves the curve *up* by $1/m_+$, each negative moves it *right* by $1/m_-$.

Consider the horizontal step contributed by a particular negative j . Its height above the FPR axis at that moment is

$$(\text{number of positives already passed})/m_+ = \#\{i : s_i^+ > s_j^-\}/m_+.$$

The area of that step is therefore $\#\{i : s_i^+ > s_j^-\} / (m_+ m_-)$. Summing over all negatives,

$$\text{AUC} = \frac{1}{m_+ m_-} \sum_{j=1}^{m_-} \sum_{i=1}^{m_+} \mathbb{1}[s_i^+ > s_j^-] = \frac{U}{m_+ m_-},$$

where $U = \sum_{i,j} \mathbb{1}[s_i^+ > s_j^-]$ is precisely the Mann–Whitney U statistic of the positive-class scores. Dividing by the number of pairs turns the count into the empirical probability $P(s^+ > s^-)$ for a uniformly random (positive, negative) pair, which is [Proposition 3.2](#).

Ties. If k_+ positives and k_- negatives share a score, the ROC convention traces a straight diagonal segment through the tied block, contributing area equal to what the pairs would contribute at $\frac{1}{2}$ each; correspondingly U is defined with $\frac{1}{2}$ per tied pair. This yields the tie-corrected statement $\text{AUC} = P(s^+ > s^-) + \frac{1}{2}P(s^+ = s^-)$ of [Eq. \(3.15\)](#).

Solution to Exercise 3.6. Suppose scores are independent of labels. Fix any threshold t and let $A = \{\mathbf{x} : s(\mathbf{x}) \geq t\}$ be the alert set. By independence, the label distribution *inside* A equals the marginal label distribution, so

$$\mathbb{E}[\text{Precision}(t)] = P(y = 1 \mid \mathbf{x} \in A) = \pi,$$

the prevalence — at *every* threshold, hence at every achievable recall level. The PR “curve” of a label-independent scorer is therefore the horizontal line at height π , and its area (average precision) is

$$\text{AP} = \int_0^1 \pi d(\text{recall}) = \pi.$$

On the case-study portfolio $\pi \approx 0.03$, so chance PR–AUC ≈ 0.03 . Reporting “PR–AUC = 0.31” without the baseline invites two opposite misreadings: against an implicit 0.5 anchor (borrowed from ROC intuition) it looks poor, whereas against the true chance level it is a tenfold lift. The convention of [Section 3.4.3](#) — always state PR–AUC *together with* the prevalence — exists to force the correct reading.

Solution to Exercise 3.7. From [Eq. \(3.17\)](#), $\text{se} = \sqrt{r(1-r)/n_+} \leq 0.02$ requires

$$n_+ \geq \frac{r(1-r)}{\text{se}^2} = \frac{0.7 \times 0.3}{0.02^2} = \frac{0.21}{0.0004} = 525 \text{ positives.}$$

At 3% prevalence that means $525/0.03 = 17,500$ validation rows — for the validation split *alone*, before training data is even counted.

The implication is sobering and is the entire motivation of [Section 3.6](#): on a small credit portfolio (say 10,000 labelled loans in total, ~ 300 defaulters), *no single split can estimate recall to within two points*. The honest options

are exactly the chapter’s toolkit: aggregate information across stratified folds (Definition 3.7), attach bootstrap intervals to whatever held-out estimate is reported (Section 3.6.4), and refuse to interpret differences smaller than the resulting uncertainty band.

Solution to Exercise 3.8. With $n_{01} = 63$ (only model B correct) and $n_{10} = 41$ (only model A correct), Eq. (3.19) gives

$$\chi^2 = \frac{(|63 - 41| - 1)^2}{63 + 41} = \frac{21^2}{104} = \frac{441}{104} \approx 4.24.$$

Since $4.24 > \chi^2_{1,0.95} = 3.84$, we reject at the 5% level the null hypothesis that the two models have equal error rates: model B’s advantage on the discordant cases is unlikely to be chance ($p \approx 0.04$).

The 9,800 concordant cases are irrelevant because they carry no information about the *difference* between the models: whether both are right or both are wrong on a case, that case shifts both models’ accuracies by the same amount. McNemar’s test conditions on the set of disagreements and asks only whether the 63:41 split is compatible with a fair coin — which is also why the test remains valid on heavily imbalanced test sets where accuracy itself is uninformative (Section 3.3.1).

Solution to Exercise 3.9. Applying Eq. (3.20) coordinate-wise with $\lambda = 0.4$:

$$(1, 0, 0) + 0.4 [(0, 1, 0) - (1, 0, 0)] = (1, 0, 0) + 0.4 (-1, 1, 0) = (0.6, 0.4, 0).$$

The result is invalid as a data point because a one-hot block encodes a *categorical* fact — the applicant’s region — and $(0.6, 0.4, 0)$ asserts membership of 60% in one region and 40% in another. No real applicant generates such a row; it violates the encoder’s sum-to-one, single-one structure, and any coefficient interpretation (Example 3.1) built on “the region-A indicator” silently breaks.

SMOTE-NC’s repair. SMOTE-NC interpolates only the *numeric* coordinates via Eq. (3.20), and sets each *categorical* feature of the synthetic point to the majority value of that feature among the k minority-class nearest neighbours (with the categorical features also entering the distance computation through a fixed penalty term). The synthetic row is then a legal row: valid one-hot blocks, interpolated numerics.

Residual risk. The synthetic point marries numeric values interpolated between two (possibly different-region) borrowers with a categorical value voted from the neighbourhood, so it can realise *combinations* of features that never co-occur in reality — e.g. transaction magnitudes typical of Nairobi CBD attached to a rural-county label. The class-conditional joint distribution is thereby distorted, and, as Section 3.7.2 already warned for plain

SMOTE, the altered class prior means the resulting scores are not calibrated: recalibrate (Section 3.5) before applying Proposition 3.1.

Solution to Exercise 3.10. Let $w = c_{FN}/c_{FP}$ and suppose training re-weights each positive example by w (equivalently, imagine each positive duplicated w times). For a scorer that is calibrated *on the re-weighted distribution*, the probability it reports is the re-weighted posterior. If p denotes the true (original-distribution) posterior $P(y = 1 \mid \mathbf{x})$, re-weighting the positive class by w transforms it to

$$p_w = \frac{w p}{w p + (1 - p)},$$

since the class-conditional densities are untouched and only the effective prior odds are multiplied by w . Thresholding the weighted model at 0.5 flags iff

$$p_w \geq \frac{1}{2} \iff w p \geq 1 - p \iff p \geq \frac{1}{1 + w} = \frac{c_{FP}}{c_{FP} + c_{FN}} = t^*,$$

which is exactly the unweighted model thresholded at t^* of Eq. (3.13). This is Elkan (2001)'s theorem: for Bayes-optimal (calibrated) scorers, cost-proportionate re-weighting and threshold moving induce the *same* decision rule.

Why try threshold moving first, then? Because the equivalence is exact only in the idealised limit, and every practical consideration favours the decision-level route. (i) It is *free and reversible*: no retraining, and the cost matrix can be revised — or differ across products — while one model serves them all, each business line simply reading a different operating point off the same score. (ii) It *preserves calibration*: re-weighting deliberately distorts the class prior, so the weighted model's outputs are no longer probabilities on production traffic and must be recalibrated before anyone prices risk with them. (iii) With finite samples and misspecified models the two routes genuinely diverge, and re-weighting can degrade the ranking it was meant to help. (iv) It is *auditable*: “we score with model M and decline above 7.4% calibrated risk, derived from this cost matrix” is a one-line policy a regulator can inspect. Escalate to weighting or resampling only when the evidence says the *ranking itself* — not the decision rule — is the weak link (Section 3.7.1).

Chapter 4: Decision Trees, Ensembles and Hyperparameter Tuning

Solution to Exercise 4.1. *Independent voters.* With $p = 0.55$, the number of correct voters is $K \sim \text{Binomial}(M, 0.55)$ and the majority is correct when

$K > M/2$. The normal approximation $K \approx \mathcal{N}(Mp, Mp(1-p))$ is justified for these M by the usual rule $Mp(1-p) \gg 1$ (it equals 2.7, 25.0 and 247.7 respectively; for $M = 11$ the approximation is rough and the exact sum is preferable, but it errs by under two points here). Writing $z = (M/2 - Mp) / \sqrt{Mp(1-p)}$ and $P_{\text{maj}} \approx 1 - \Phi(z) = \Phi(-z)$:

M	$-z$	P_{maj} (normal)	P_{maj} (exact)
11	0.332	0.630	0.633
101	1.005	0.843	0.846
1001	3.164	0.9992	0.9992

The committee converges on certainty, exactly as [Proposition 4.1](#) promises.

Correlated blocks. If the M voters consist of ten blocks of $M/10$ perfectly correlated clones, every clone in a block casts the same vote, so the committee behaves as *ten* independent voters, whatever M is. For $M = 1001$ (≈ 100 clones per block) the relevant computation is the $M = 10$ (even; take $M = 11$ as the odd comparator) row, not the $M = 1001$ row: $P_{\text{maj}} \approx 0.63$, and no increase of M at fixed block structure improves it. The *effective committee size* is the number of independent error patterns, not the number of members. This is [Eq. \(4.5\)](#) in disguise — within-block correlation $\rho = 1$ contributes pure floor — and it is precisely the situation of a forest whose trees share their root split: trees that agree on the first partition of the data are partial clones, adding `n_estimators` buys little, and the productive intervention is decorrelation (`max_features`), not more trees.

Solution to Exercise 4.2. Let $\bar{f} = \sum_m c_m f_m$ with $\sum_m c_m = 1$, $c_m \geq 0$, $\text{Var}[f_m] = \sigma^2$ and pairwise correlation ρ . Then

$$\begin{aligned} \text{Var}[\bar{f}] &= \sum_m c_m^2 \sigma^2 + \sum_{m \neq m'} c_m c_{m'} \rho \sigma^2 \\ &= \sigma^2 \left[(1 - \rho) \sum_m c_m^2 + \rho \left(\sum_m c_m \right)^2 \right] = \sigma^2 \left[(1 - \rho) \sum_m c_m^2 + \rho \right], \end{aligned}$$

using $\sum_{m \neq m'} c_m c_{m'} = (\sum_m c_m)^2 - \sum_m c_m^2 = 1 - \sum_m c_m^2$. Since $\rho \leq 1$, minimising the variance means minimising $\sum_m c_m^2$ subject to $\sum_m c_m = 1$. By Cauchy–Schwarz (or a Lagrange multiplier), $\sum_m c_m^2 \geq \frac{1}{M} (\sum_m c_m)^2 = \frac{1}{M}$, with equality iff $c_m = 1/M$ for all m : *uniform weights are optimal*, and they recover [Proposition 4.2](#).

The optimality is conditional on the exchangeability assumptions. If the members have *unequal* variances — equivalently, unequal skill, as in [Listing 4.2](#) where a tuned LightGBM is simply a better estimator than the logistic baseline — the same Lagrangian calculation with member variances σ_m^2 (and $\rho = 0$ for clarity) gives optimal weights $c_m \propto 1/\sigma_m^2$: precision weighting. Soft voting with unequal weights is the practical, integer-granularity version of this, and stacking ([Section 4.8](#)) is its data-driven completion — the meta-learner *estimates* the weights instead of assuming exchangeability.

Solution to Exercise 4.3. *The limit without logarithms.* From $1 - x \leq e^{-x}$ with $x = \frac{1}{n}$,

$$\left(1 - \frac{1}{n}\right)^n \leq e^{-1}.$$

For the matching lower bound, apply $1 - x \geq e^{-x/(1-x)}$ (valid for $0 \leq x < 1$, and itself a rearrangement of $1 + u \leq e^u$ with $u = x/(1-x)$) to get $\left(1 - \frac{1}{n}\right)^n \geq e^{-n \cdot \frac{1/n}{1-1/n}} = e^{-n/(n-1)} \rightarrow e^{-1}$. The two bounds pinch the sequence to e^{-1} . For $n = 20$, $\left(\frac{19}{20}\right)^{20} = 0.3585$, already within 2.5% of $e^{-1} \approx 0.3679$ — the asymptotics arrive early, which is why “37% out of bag” is a safe rule of thumb even for modest samples.

Why an OOB-chosen threshold misbehaves. (i) *Statistical:* each OOB prediction $\bar{f}_{-i}^{\text{OOB}}(x_i)$ in Eq. (4.6) averages only the $\approx 0.37M$ trees that missed row i , whereas the deployed model averages all M . A mean of $0.37M$ trees is a noisier, differently calibrated score than a mean of M trees, so the probability distribution on which the OOB threshold was optimised is not the distribution the production model emits; with the asymmetric costs of the case study (KES 10,000 vs KES 800), even a small calibration shift moves the cost-optimal cut-off materially. (ii) *Temporal:* the OOB estimate randomises over rows, so its implicit population is the historical mixture of the training window. The threshold’s economics depend on the *future* default base rate and score distribution, which drift; the temporal validation window of Chapter 3 is the only estimate that rehearses the deployment condition. In short, OOB may rank models acceptably, but the decision threshold should be set — and re-set, per Chapter 11 — on out-of-time data.

Solution to Exercise 4.4. After stage m , the new weight mass on the rows that f_m got wrong is

$$W_{\text{wrong}}^{(m+1)} = \sum_{i: f_m(x_i) \neq y_i} w_i^{(m)} e^{+\alpha_m} = e^{\alpha_m} \varepsilon_m W^{(m)}, \quad W_{\text{corr}}^{(m+1)} = e^{-\alpha_m} (1 - \varepsilon_m) W^{(m)},$$

where $W^{(m)} = \sum_i w_i^{(m)}$. Substituting $e^{\alpha_m} = \sqrt{(1 - \varepsilon_m)/\varepsilon_m}$ from Proposition 4.4,

$$W_{\text{wrong}}^{(m+1)} = \sqrt{\frac{1 - \varepsilon_m}{\varepsilon_m}} \varepsilon_m W^{(m)} = \sqrt{\varepsilon_m (1 - \varepsilon_m)} W^{(m)} = W_{\text{corr}}^{(m+1)}.$$

The two masses are equal, so the weighted error of f_m on the updated weights is exactly $\frac{1}{2}$: each learner is rendered useless against the distribution it leaves behind. The implication is that AdaBoost can never profitably select the same hypothesis twice in a row — the re-weighting is an *orthogonalisation* step that forces every stage to contribute something the previous stage did not, which is the sequential counterpart of the decorrelation that bagging obtains by randomisation. (Hypotheses can recur later, once the weight distribution has moved on.)

Solution to Exercise 4.5. (a) *Binary log-loss.* With $y \in \{0, 1\}$, logits F , and $\sigma(F) = (1 + e^{-F})^{-1}$,

$$L(y, F) = -[y \ln \sigma(F) + (1 - y) \ln(1 - \sigma(F))].$$

Using $\sigma'(F) = \sigma(F)(1 - \sigma(F))$,

$$\frac{\partial L}{\partial F} = -\left[y \frac{\sigma'}{\sigma} - (1 - y) \frac{\sigma'}{1 - \sigma}\right] = -[y(1 - \sigma) - (1 - y)\sigma] = \sigma(F) - y,$$

so the pseudo-residual [Eq. \(4.7\)](#) is $r = y - \sigma(F)$: the error in predicted probability, as claimed in [Section 4.6.3](#).

(b) *Pinball loss.* At quantile level $\tau \in (0, 1)$,

$$L_\tau(y, F) = \begin{cases} \tau(y - F), & y \geq F, \\ (1 - \tau)(F - y), & y < F, \end{cases} \quad r = -\frac{\partial L_\tau}{\partial F} = \begin{cases} \tau, & y > F, \\ \tau - 1, & y < F, \end{cases}$$

(the loss is not differentiable at $y = F$; any subgradient value in $[\tau - 1, \tau]$ serves). The boosted model then nudges its prediction up by a constant amount whenever it under-predicts and down whenever it over-predicts, with asymmetric magnitudes τ and $1 - \tau$; the fixed point of that asymmetry is the τ -quantile.

Use in collections. A model of the $\tau = 0.9$ quantile of repayment delay answers, per borrower, “by what day will this loan have been repaid with 90% confidence?” That is the planning quantity a collections operation needs: it sizes the follow-up queue conservatively, sets the day on which an unpaid loan becomes genuinely anomalous (rather than merely late-ish) and therefore worth an agent’s call, and prices expected recovery timelines into cash-flow forecasts. The conditional *mean* delay would understate all three, because delay distributions are heavy-tailed and the cost of under-provisioning is asymmetric — the same cost-sensitivity logic as [Chapter 3](#), applied to a regression target.

Solution to Exercise 4.6. Under squared error $L = \frac{1}{2}(y - F)^2$ we have $g_i = \partial_F L = F(x_i) - y_i = -r_i$ (minus the ordinary residual) and $h_i = \partial_F^2 L = 1$. Hence for leaf j , $G_j = -\sum_{i \in I_j} r_i$ and $H_j = |I_j|$, and [Eq. \(4.11\)](#) gives

$$w_j^* = -\frac{G_j}{H_j + \lambda} = \frac{\sum_{i \in I_j} r_i}{|I_j| + \lambda} \xrightarrow{\lambda=0} \frac{1}{|I_j|} \sum_{i \in I_j} r_i,$$

the mean residual in the leaf — so vanilla gradient boosting ([Section 4.6.3](#)) is recovered as the unregularised special case. For $\lambda > 0$,

$$w_j^* = \frac{H_j}{H_j + \lambda} \cdot \underbrace{\frac{1}{H_j} \sum_{i \in I_j} r_i}_{\text{mean residual}},$$

a shrinkage of the mean residual by the factor $H_j/(H_j + \lambda) \in (0, 1)$. The factor is close to 1 for well-populated leaves and small for thin ones: a leaf holding $H_j = 500$ loans with $\lambda = 5$ keeps 99% of its estimate, while a leaf holding 5 loans keeps only 50%. The ridge penalty is thus an automatic scepticism dial — conclusions drawn from few borrowers are pulled toward “no adjustment” — which is exactly the behaviour one wants in a 4.1%-default portfolio where thin leaves are dominated by noise, and it complements the hard floor that `min_child_samples` imposes in [Listing 4.4](#).

Solution to Exercise 4.7. *Counterexample.* Hold every feature fixed except the delinquency feature `days_past_due` (x , in days), and suppose the unconstrained model’s fitted risk is piecewise constant:

$$F(x) = \begin{cases} 0.10, & 0 \leq x < 10, \\ 0.30, & 10 \leq x < 20, \\ 0.20, & 20 \leq x < 30, \\ 0.50, & x \geq 30. \end{cases}$$

Any reasonable notion of the *average* response is increasing — the least-squares linear fit of F on x has positive slope, and the overall trend runs from 0.10 to 0.50 — yet the model is not monotone: it assigns *lower* risk at $x = 25$ than at $x = 15$.

Governance failure. Consider two applicants identical in every respect except that applicant A has 15 days of arrears and applicant B has 25. Suppose the approval threshold sits at 0.25: A is declined, B is approved. The adverse-action reason issued to A necessarily cites delinquency — it is the only feature in play — yet B, with *strictly worse* delinquency, was approved. The stated reason is therefore falsified by the lender’s own decisions: A’s appeal, a regulator’s file review, or a journalist’s paired test will surface the inconsistency, and under the automated-decision provisions of the Kenya DPA (2019) the lender must stand behind an explanation its model contradicts. The average trend is irrelevant to this failure because adverse-action reasons are given to *individuals*, who live at specific points of the curve, not on its regression line. The monotonic constraint of [Definition 4.5](#) removes the failure mode by construction, at the cost of the (noise-driven) dip — which is precisely the trade described in [Section 4.7.3](#).

Solution to Exercise 4.8. By [Proposition 4.5](#), an equal-resolution grid with $B = 729$ and $d = 6$ uses $r = 729^{1/6} = 3$ distinct values per axis, so each of the two important hyperparameters is probed at just *three* values, each combination repeated across the $3^4 = 81$ settings of the four irrelevant axes. Random search probes 729 distinct values of each important axis.

Probability the grid never enters the best decile of both important axes simultaneously. Model each axis as $[0, 1]$ with the “best decile” an interval of length

0.1 whose location is unknown, and take the grid's three values per axis to be evenly spaced with a uniformly random offset, $\{u, u + \frac{1}{3}, u + \frac{2}{3}\}$ with $u \sim \text{U}[0, \frac{1}{3})$ (assumptions stated: independent axes, uniform target location, one-third spacing). Since the target interval $0.1 < \frac{1}{3}$, at most one grid value can fall in it, and the probability that one does is $3 \times 0.1 = 0.3$ per axis. Because the grid is a Cartesian product, hitting the decile marginally on both important axes guarantees a trial at the intersection; the two axes are independent, so

$$P(\text{grid hits both}) = 0.3 \times 0.3 = 0.09, \quad P(\text{grid never does}) = 0.91.$$

For random search, a single trial lands in both deciles with probability $0.1 \times 0.1 = 0.01$, so

$$P(\text{random never does}) = (1 - 0.01)^{729} = 0.99^{729} \approx e^{-7.29} \approx 7 \times 10^{-4}.$$

The grid misses the jointly-best region nine times out of ten; random search finds it essentially always. This is Fig. 4.11 with numbers attached, and it is the practical content of low effective dimensionality (Bergstra and Bengio, 2012).

Solution to Exercise 4.9. With $N = 81$, $b = 25$ and $\eta = 3$, the rungs are:

Rung	1	2	3	4	5
Configurations alive	81	27	9	3	1
Budget each (rounds)	25	75	225	675	2025
Cost of rung	2025	2025	2025	2025	2025

Each rung costs exactly $81 \times 25 = 2025$ configuration-rounds — the $1/\eta$ survival rate and the $\times \eta$ budget growth cancel — so the total is $5 \times 2025 = 10,125$ configuration-rounds. Running all 81 configurations to the full 2,025 rounds would cost $81 \times 2025 = 164,025$: successive halving explores the same 81 candidates for 6.2% of the cost, a $16.2\times$ saving, at the price of trusting rung-1 rankings.

Where the trust fails. The schedule assumes that performance after 25 boosting rounds predicts performance after 2,025. The clearest counterexample in our own chapter is the learning rate (Eq. (4.8)): a configuration with $\nu = 0.02$ is deliberately slow — at 25 rounds it has barely moved from the base rate and looks terrible next to a $\nu = 0.3$ configuration — yet it is exactly the kind of run that attains the *lowest* final validation loss. Aggressive halving systematically executes the low- ν candidates at rung 1 and crowns a fast-starting, earlier-overfitting configuration. Mitigations: hedge across brackets (Hyperband, Li et al., 2017), delay pruning (`n_warmup_steps` in Listing 4.6), or normalise the comparison by learning-curve extrapolation rather than raw score.

Solution to Exercise 4.10. Each measured score is $\hat{\mathcal{V}}_t = v + \varepsilon_t$ with $\varepsilon_t = sZ_t$ and $Z_t \sim \mathcal{N}(0, 1)$ i.i.d. The search reports

$$\max_t \hat{\mathcal{V}}_t = v + s \max_t Z_t \implies \mathbb{E}[\max_t \hat{\mathcal{V}}_t] = v + s \mathbb{E}[\max_{t \leq T} Z_t],$$

since v is common and the maximum commutes with the shift. All T configurations are *equally good* by construction, so the entire excess over v is optimism manufactured by selection. Using the standard extreme-value approximation $\mathbb{E}[\max_{t \leq T} Z_t] \approx \sqrt{2 \ln T}$: for $T = 100$, $\sqrt{2 \ln 100} = \sqrt{9.21} \approx 3.03$, so with fold-level noise $s = 0.004$ in PR-AUC the expected reported best is

$$v + 0.004 \times 3.03 \approx v + 0.012.$$

An HPO study of one hundred trials will thus claim roughly $+0.012$ PR-AUC that is not there, even when tuning does nothing at all. Two readings matter for Lab 4. First, the magnitude is the same order as the optimism gap students typically measure between `study.best_value` and the sealed out-of-time score — part of that gap is drift, but a predictable slice of it is pure winner’s curse, and Table 4.7’s tuned-model row would read ≈ 0.28 rather than 0.294 if the inner-loop best were taken at face value. Second, the formula shows the levers: optimism grows only as $\sqrt{\ln T}$ (slowly — more trials are not catastrophic) but linearly in s , so the cheapest honesty gain is reducing fold noise: more folds, repeated CV, or larger validation windows. And, of course, reporting from data the search never touched (Fig. 4.13) removes the curse entirely — which is the point of the chapter’s final pitfall box.

Chapter 5: Unsupervised Learning and Dimensionality Reduction

Solution to Exercise 5.1. Let $J^*(k)$ denote the optimal inertia with k clusters. Take an optimal solution for k with centroids $\mu_1, \dots, \mu_k$ and assignment c . Build a feasible $(k + 1)$ -cluster solution by choosing any point x_r that is not already alone in its cluster, adding the centroid $\mu_{k+1} = x_r$, and reassigning x_r to it. Every other point keeps its assignment and hence its contribution to Eq. (5.1); the contribution of x_r falls from $\|x_r - \mu_{c(r)}\|^2 \geq 0$ to 0. Therefore $J^*(k + 1) \leq J^*(k)$. At $k = m$ we may place one centroid on each point, giving $J^*(m) = 0$.

Consequently “minimise inertia over k ” always returns $k = m$, which is the degenerate clustering in which every wallet is its own segment: the criterion is vacuous because model complexity is unpenalised. The elbow heuristic is not vacuous because it does not minimise J ; it examines the *rate* at which J falls and looks for the point past which additional clusters

buy little. That is an informal appeal to a complexity penalty, which BIC (Eq. (5.9)) makes formal — and this is precisely why BIC has a genuine interior minimum where inertia does not.

Solution to Exercise 5.2. Partition $C_1 = \{0, 1, 2\}$, $C_2 = \{10, 11, 12\}$. By symmetry it suffices to do C_1 .

- $x = 0$: $a = \frac{1+2}{2} = 1.5$; $b = \frac{10+11+12}{3} = 11$; $s = (11 - 1.5)/11 = 0.8636$.
- $x = 1$: $a = \frac{1+1}{2} = 1$; $b = \frac{9+10+11}{3} = 10$; $s = (10 - 1)/10 = 0.9000$.
- $x = 2$: $a = \frac{2+1}{2} = 1.5$; $b = \frac{8+9+10}{3} = 9$; $s = (9 - 1.5)/9 = 0.8333$.

C_2 mirrors these exactly, so $\bar{s} = \frac{1}{3}(0.8636 + 0.9000 + 0.8333) = 0.8657$.

Partition $C'_1 = \{0, 1\}$, $C'_2 = \{2, 10, 11, 12\}$.

- $x = 0$: $a = 1$; $b = \frac{2+10+11+12}{4} = 8.75$; $s = (8.75 - 1)/8.75 = 0.8857$.
- $x = 1$: $a = 1$; $b = \frac{1+9+10+11}{4} = 7.75$; $s = 0.8710$.
- $x = 2$: $a = \frac{8+9+10}{3} = 9$; $b = \frac{2+1}{2} = 1.5$; $s = (1.5 - 9)/9 = -0.8333$.
- $x = 10$: $a = \frac{8+1+2}{3} = 3.6\bar{6}$; $b = \frac{10+9}{2} = 9.5$; $s = (9.5 - 3.6\bar{6})/9.5 = 0.6140$.
- $x = 11$: $a = \frac{9+1+1}{3} = 3.6\bar{6}$; $b = 10.5$; $s = 0.6508$.
- $x = 12$: $a = \frac{10+2+1}{3} = 4.3\bar{3}$; $b = 11.5$; $s = 0.6232$.

$\bar{s} = \frac{1}{6}(0.8857 + 0.8710 - 0.8333 + 0.6140 + 0.6508 + 0.6232) = 0.4686$.

Comment. The natural partition scores far higher (0.866 versus 0.469), so the silhouette correctly prefers it. More instructive is the single strongly negative value, $s(2) = -0.833$: the point $x = 2$ is on average much closer to C'_1 than to the cluster it was assigned to. This is exactly what the silhouette *diagram* is for. An analyst looking only at the mean would see a mediocre 0.469; an analyst looking at the per-point profile sees immediately *which* point is misassigned and can fix it. Note also that $\bar{s}$ for the bad partition is still positive, because four of six points are content — averages hide misassignment, profiles do not.

Solution to Exercise 5.3. Take the six points, with axis 1 in Kenyan shillings and axis 2 as a count of transactions:

	P_1	P_2	P_3	P_4	P_5	P_6
x_1 (KES)	1000	1100	1200	9000	9100	9200
x_2 (count)	2	40	78	2	40	78

Raw data. Squared distances are dominated by x_1 , whose spread is ~ 8000 against ~ 76 for x_2 . K -means with $k = 2$ therefore splits on value: $\{P_1, P_2, P_3\}$ versus $\{P_4, P_5, P_6\}$. (Check: the within-cluster sum of squares for this split is $2 \cdot (20000 + 2 \cdot 1352.7) \approx 4.5 \times 10^4$, whereas splitting on count gives clusters spanning 8000 in x_1 and costs $\sim 1.3 \times 10^8$.)

Standardised data. After centring and dividing by the sample standard deviations ($\sigma_1 \approx 4271$, $\sigma_2 \approx 34.9$), both axes have unit spread and the configuration becomes a symmetric 2×3 grid at $x_1 \approx \pm 0.94$ and $x_2 \approx \{-1.09, 0, 1.09\}$. The two candidate splits now cost the same by symmetry along x_1 , but the three-level x_2 axis has the larger total spread (± 1.09 against ± 0.94), so K -means prefers to cut along x_2 , returning e.g. $\{P_1, P_4\}$ (low count) against $\{P_2, P_3, P_5, P_6\}$. Perturb the numbers slightly to break the tie in whichever direction you like; the point is that the two answers are different and both are “correct” for their respective metrics.

Which to defend. The standardised one — but the honest defence is not that standardisation is a rule. It is that the raw-data partition answers a question nobody asked: “are these transactions large or small in KES?” is answerable with a single threshold and needs no clustering at all. If the business genuinely wants value tiers, use a threshold and say so. Clustering earns its keep only when several dimensions jointly matter, and that requires them to be commensurable. Where a monetary feature is heavy-tailed, prefer a quantile or log transform to plain standardisation, since a handful of whale wallets will otherwise dominate the standardised axis too.

Solution to Exercise 5.4. Construction. Work in $\mathbb{R}^2$ with $\varepsilon = 1.05$ and $\text{minPts} = 8$. Build two clusters, each a dense blob plus one outlying “antenna” point:

- Cluster A : ten points inside a ball of radius 0.05 centred at $(-0.6, 0)$, plus a single point $p_A = (0, 0)$.
- Cluster B : ten points inside a ball of radius 0.05 centred at $(2.6, 0)$, plus a single point $p_B = (2, 0)$.
- The candidate border point $q = (1, 0)$.

Every blob point is core: it sees the other nine blob points (mutual distance ≤ 0.1) and its antenna (distance $0.6 \pm 0.05 \leq \varepsilon$), so $|N_\varepsilon| = 11 \geq 8$. *Each antenna is core:* p_A sees its blob (ten points at ≈ 0.6), itself, and q (distance exactly $1 \leq \varepsilon$), so $|N_\varepsilon(p_A)| = 12 \geq 8$. *q is not core:* its ε -ball contains only q , p_A and p_B , since both blobs sit at distance $\approx 1.6 > \varepsilon$; thus $|N_\varepsilon(q)| = 3 < 8$. Finally A and B are distinct components, because $d(p_A, p_B) = 2 > \varepsilon$ and q — the only object between them — is not core, so it cannot propagate reachability.

Therefore q is a border point directly density-reachable from p_A and from p_B , which belong to different clusters. DBSCAN assigns q to whichever cluster is expanded first: process p_A before p_B and q joins A ; reverse the visitation order and q joins B . Its label is genuinely order-dependent.

Why this does not undermine reproducibility. By Proposition 5.3, the core-point set and the partition of core points into clusters are order-invariant. The ambiguity therefore never changes the number of clusters, their shape, or the membership of any core point — only the label of a point sitting in a low-density bridge, whose ϵ -ball is by construction almost empty. Operationally: the segments you name and price are determined by their cores, and a point whose label flips is exactly one about which you should not be making a confident decision. If bit-for-bit determinism is required for audit, fix the input ordering and record it; or use HDBSCAN, which assigns such points by membership strength rather than by visitation order.

Solution to Exercise 5.5. With $k = 2$, $\pi_1 = \pi_2 = \frac{1}{2}$, and shared variance σ^2 , the Gaussian normalising constants cancel in Eq. (5.5):

$$\gamma_{i1} = \frac{\exp\left(-\frac{(x_i - \mu_1)^2}{2\sigma^2}\right)}{\exp\left(-\frac{(x_i - \mu_1)^2}{2\sigma^2}\right) + \exp\left(-\frac{(x_i - \mu_2)^2}{2\sigma^2}\right)} = \frac{1}{1 + \exp\left(\frac{(x_i - \mu_1)^2 - (x_i - \mu_2)^2}{2\sigma^2}\right)}.$$

Expanding the difference of squares, $(x - \mu_1)^2 - (x - \mu_2)^2 = -2x(\mu_1 - \mu_2) + \mu_1^2 - \mu_2^2 = -2(\mu_1 - \mu_2)\left(x - \frac{\mu_1 + \mu_2}{2}\right)$. Hence

$$\gamma_{i1} = \sigma_{\text{logistic}}(w(x_i - x_0)), \quad w = \frac{\mu_1 - \mu_2}{\sigma^2}, \quad x_0 = \frac{\mu_1 + \mu_2}{2},$$

where $\sigma_{\text{logistic}}(z) = (1 + e^{-z})^{-1}$. So the responsibility is exactly a logistic function of x with **slope parameter** $w = (\mu_1 - \mu_2)/\sigma^2$ and **midpoint** at the arithmetic mean of the two component means, where $\gamma_{i1} = \frac{1}{2}$.

As $\sigma^2 \rightarrow 0^+$ with $\mu_1 \neq \mu_2$ fixed, $|w| \rightarrow \infty$: the logistic curve steepens into a step at x_0 , so $\gamma_{i1} \rightarrow \mathbb{1}\{x_i \text{ nearer } \mu_1\}$ for every $x_i \neq x_0$. This is precisely Proposition 5.5 in one dimension: the midpoint x_0 is the perpendicular bisector (here, a single point), and the soft assignment collapses to K -means' hard nearest-centroid rule. The general lesson is that σ^2 acts as a temperature: large σ^2 gives soft, hedged memberships; small σ^2 gives confident hard ones. Reporting a segment membership of 0.51 versus 0.49 as though it were a fact is a failure to read this parameter.

Solution to Exercise 5.6. Divergence. Take $m \geq 2$ distinct points and the family

$$\theta_\epsilon : \quad \pi_1 = \pi_2 = \frac{1}{2}, \quad \mu_1 = x_1, \quad \sigma_1^2 = \epsilon, \quad \mu_2 = \bar{x}, \quad \sigma_2^2 = s^2 > 0 \text{ fixed.}$$

The first component's density at x_1 is $\mathcal{N}(x_1 \mid x_1, \epsilon) = (2\pi\epsilon)^{-1/2}$, so the contribution of observation 1 to the log-likelihood is at least $\log(\frac{1}{2}(2\pi\epsilon)^{-1/2}) \rightarrow +\infty$ as $\epsilon \rightarrow 0^+$. Every other observation $i \geq 2$ retains the second component's contribution, bounded below by $\log(\frac{1}{2}\mathcal{N}(x_i \mid \bar{x}, s^2)) > -\infty$, which does not depend on ϵ . Hence $\ell(\theta_\epsilon) \rightarrow +\infty$: the likelihood is unbounded above and no global maximiser exists. Geometrically, the fitted model has spiked onto a single data point and explains nothing.

Effect of reg_covar. Adding $\lambda > 0$ to the diagonal replaces σ_j^2 by $\sigma_j^2 + \lambda$, so the achievable density at a point is bounded by $(2\pi\lambda)^{-1/2}$ and the per-observation log-likelihood by $\log(\pi_{\max}(2\pi\lambda)^{-1/2}) < \infty$. Summing over m observations bounds ℓ above, restoring existence of a maximum on the (now compact-in-effect) parameter region, and the degenerate spikes are removed from the search space entirely.

Price paid. Three costs. (i) The estimator is biased: every fitted covariance is inflated by λ , so genuinely tight clusters are reported as looser than they are, and density-based anomaly scores are correspondingly flattened. (ii) The reported likelihood, and therefore BIC, is no longer that of the model you claim to be fitting, so BIC comparisons are only valid at a *fixed* λ . (iii) λ is a scale-dependent hyperparameter: the same value means something different on standardised and unstandardised features, which is one more reason to fix the scaling before the mixture. In practice keep λ at the smallest value that prevents collapse (the scikit-learn default 10^{-6} on standardised data is usually right) and treat a fit that needs a large λ as evidence that k is too big.

Solution to Exercise 5.7. Free parameters split into weights, means and covariances. Weights contribute $k - 1$ (they sum to one); means contribute kn . Covariances:

covariance_type	Covariance parameters	Total p
full	$k \frac{n(n+1)}{2}$	$(k - 1) + kn + k \frac{n(n+1)}{2}$
tied	$\frac{n(n+1)}{2}$	$(k - 1) + kn + \frac{n(n+1)}{2}$
diag	kn	$(k - 1) + 2kn$
spherical	k	$(k - 1) + kn + k$

For $n = 32, k = 5$: $k - 1 = 4$ and $kn = 160$.

- full: $\frac{32 \cdot 33}{2} = 528$ per component, so $p = 4 + 160 + 5(528) = \mathbf{2804}$.
- tied: $p = 4 + 160 + 528 = 692$.
- diag: $p = 4 + 2(160) = \mathbf{324}$.
- spherical: $p = 4 + 160 + 5 = 169$.

With $m = 150,000$, $\ln m = \ln(1.5 \times 10^5) \approx 11.9184$. For full to beat diag on BIC we need

$$p_{\text{full}} \ln m - 2 \ln \hat{L}_{\text{full}} < p_{\text{diag}} \ln m - 2 \ln \hat{L}_{\text{diag}},$$

i.e.

$$\begin{aligned} \ln \hat{L}_{\text{full}} - \ln \hat{L}_{\text{diag}} &> \frac{1}{2}(2804 - 324)(11.9184) \\ &= \frac{1}{2}(2480)(11.9184) \approx \mathbf{14,778}. \end{aligned}$$

Averaged over the sample this is only $14,778/150,000 \approx 0.0985$ nats per observation — about a 10% relative improvement in average likelihood. So with 150,000 wallets, full covariances are a *cheap* upgrade: modest correlation structure within segments will pay for 2480 extra parameters comfortably. Repeat the arithmetic at $m = 2000$ and the required gap becomes ≈ 0.5 nats per observation, at which point diag usually wins. The general moral is that the “right” covariance type is a function of sample size, not of taste.

Solution to Exercise 5.8. Independent coordinates. A short simulation (numpy default generator, seed 0; $m = 1000$ points and one query point drawn from the same distribution) gives ratios of the order

n	2	10	100	1000
i.i.d. coordinates	121	3.04	0.44	0.12
5-factor latent model	179	10.4	8.88	7.55

(means over 20 replications; the $n = 2$ column is enormous and wildly variable because $D_{\min}$ can be arbitrarily small in low dimension, so read the *trend* across columns, not the individual digits). In the i.i.d. row the contrast between nearest and furthest neighbour collapses roughly like $n^{-1/2}$, exactly as Proposition 5.6 predicts from $\text{Var}[D_n^2]/(\mathbb{E}[D_n^2])^2 = \mathcal{O}(1/n)$.

Latent factor model. Generate $u_i \in \mathbb{R}^5$ i.i.d. standard normal and set $x_i = Au_i + 0.05 \eta_i$ with $A \in \mathbb{R}^{n \times 5}$ fixed and η_i small isotropic noise (second row of the table). The ratio now stays around 8–10 from $n = 10$ all the way to $n = 1000$: raising the ambient dimension by two orders of magnitude does essentially nothing, because the distances are governed by the five latent coordinates and the extra measured dimensions carry almost no independent variability.

Explanation. The proof of Proposition 5.6 needs the coordinate contributions δ_t to be (near-)independent, so that $\text{Var}[D_n^2]$ grows like n while $\mathbb{E}[D_n^2]$ grows like n , forcing the relative variance to 0. Under the factor model the δ_t are strongly dependent: the effective, or intrinsic, dimension is 5, not n , and it is the intrinsic dimension that governs concentration. This is the entire justification for the chapter’s strategy. We do not escape the curse by wishing it away; we escape it because real behavioural features are redundant, and PCA finds the small number of directions along

which the data actually varies. It is also why one should measure the explained-variance curve before concluding that a 200-feature problem is high-dimensional: it may have an intrinsic dimension of 20.

Solution to Exercise 5.9. Proof. From the proof of Proposition 5.7,

$$E(W) = \frac{1}{m} \sum_i \|x_i\|^2 - \text{tr}(W^\top S W).$$

For centred data $\frac{1}{m} \sum_i \|x_i\|^2 = \text{tr}(S) = \sum_{t=1}^n \lambda_t$. At the optimum $W = W_d$ we have, by Theorem 5.1, $\text{tr}(W_d^\top S W_d) = \sum_{t \leq d} \lambda_t$. Hence

$$E(W_d) = \sum_{t=1}^n \lambda_t - \sum_{t \leq d} \lambda_t = \sum_{t > d} \lambda_t,$$

as claimed: the mean squared reconstruction error is exactly the variance you threw away.

Numerical verification. With $S = \begin{pmatrix} 4 & 2 \\ 2 & 4 \end{pmatrix}$ from Example 5.2, $\lambda_1 = 6$, $\lambda_2 = 2$, $v_1 = \frac{1}{\sqrt{2}}(1, 1)^\top$. For $d = 1$ the formula predicts $E = \lambda_2 = 2$. Take three centred points consistent with S , for instance

$$x_1 = (3, 1)^\top, \quad x_2 = (-1, -3)^\top, \quad x_3 = (-2, 2)^\top.$$

Projection: $z_i = v_1^\top x_i$ gives $z_1 = 4/\sqrt{2} = 2\sqrt{2}$, $z_2 = -4/\sqrt{2} = -2\sqrt{2}$, $z_3 = 0$. Reconstruction $\hat{x}_i = z_i v_1$: $\hat{x}_1 = (2, 2)^\top$, $\hat{x}_2 = (-2, -2)^\top$, $\hat{x}_3 = (0, 0)^\top$. Residuals: $x_1 - \hat{x}_1 = (1, -1)^\top$ with squared norm 2; $x_2 - \hat{x}_2 = (1, -1)^\top$, squared norm 2; $x_3 - \hat{x}_3 = (-2, 2)^\top$, squared norm 8. Mean = $(2 + 2 + 8)/3 = 4$.

The mean over this particular triple is 4, not 2, because three points do not reproduce S : their own sample covariance is $\begin{pmatrix} 14/3 & 2/3 \\ 2/3 & 14/3 \end{pmatrix}$, whose second eigenvalue is $14/3 - 2/3 = 4$ — and 4 is exactly what the formula predicts for *their* covariance. The identity holds sample-wise, and checking it this way is a useful reminder that λ_t are properties of the sample you actually projected, not of an assumed population.

Solution to Exercise 5.10. A defensible one-page answer covers four things.

What to measure next. (i) Whether the disparity is in the *clustering* or in the *limit rule*: compute segment membership rates by county, then, within segment, compare limits — if limits are equal within segment, the mechanism is composition, not pricing. (ii) The behavioural distance between this segment and its neighbours: if the silhouette for this cluster is weak, the segment may be an artefact and the disparity a modelling choice rather than a finding. (iii) Whether the underlying features are themselves proxies: regress county on the top principal components and see how predictable it

is; high predictability means the “behavioural” representation substantially encodes geography. (iv) Outcome data where available — if this segment’s historical default rate genuinely differs, the disparity may be justified; if it does not, the 40% limit reduction lacks a risk basis.

Instruments engaged. Both discussed in the chapter’s regulatory box. The Data Protection Act, 2019 governs the derived segment as personal data: lawful basis, purpose limitation, and the data subject’s rights of access and objection apply to an inferred behavioural profile as much as to the raw transaction log. The CBK Digital Credit Providers Regulations, 2022 require pricing and terms to be disclosed and non-exploitative and credit decisions to be explicable — an unexplainable “cluster 3” limit rule does not meet that standard, and a limit reduction concentrated on two counties invites scrutiny of whether the differentiation is risk-based or merely correlational.

Two remedies with costs. (i) *Re-fit excluding geography-loaded features* (agent-location density, merchant-mix features tied to regional paybills). Cost: some genuine predictive signal is lost, so expected credit losses rise; and the remedy is incomplete, since geography can be reconstructed from residual features. (ii) *Keep the segmentation but sever it from pricing*, using segments only for product design and communication while limits are set by the audited, monotone scorecard of [Chapter 4](#) with fairness constraints. Cost: loses the operational simplicity of segment-based pricing, and requires a second governance process for the scorecard. A third, cheaper option worth listing: cap the maximum limit differential across segments, which bounds the harm without discarding the model.

When shipping unchanged is defensible. If the disparity is *explained* by a legitimate, disclosed, risk-relevant behavioural difference that survives conditioning on geography — for example, the segment is defined by seasonal single-harvest income, the lower limit reflects a measured higher default rate for bullet-repayment products, the county concentration is a consequence of where that livelihood occurs rather than of county itself, and an applicant from either county with non-seasonal income lands in a different segment. The test is counterfactual: does an otherwise identical borrower from a different county receive the same limit? If yes, the segment is tracking livelihood, not location, and the position is arguable — provided the reasoning, the tests and the residual disparity are documented before launch, and monitored afterwards.

Chapter 6: Deep Learning Fundamentals

Solution to Exercise 6.1. By Proposition [6.1](#), the three linear hidden layers collapse to a single affine map. Writing $\mathbf{W}^{(1)} \in \mathbb{R}^{128 \times 40}$, $\mathbf{W}^{(2)} \in \mathbb{R}^{64 \times 128}$,

$\mathbf{W}^{(3)} \in \mathbb{R}^{32 \times 64}$ and $\mathbf{w}^{(4)} \in \mathbb{R}^{1 \times 32}$, the network computes

$$\hat{p} = \sigma(\tilde{\mathbf{w}}^\top \mathbf{x} + \tilde{b}), \quad \tilde{\mathbf{w}}^\top = \mathbf{w}^{(4)} \mathbf{W}^{(3)} \mathbf{W}^{(2)} \mathbf{W}^{(1)} \in \mathbb{R}^{1 \times 40}.$$

The model class is therefore **logistic regression on the original 40 features** — nothing more. Note in particular that the widths 128, 64, 32 do not enlarge the class: the composite matrix has rank at most 1 regardless, because the output layer is one-dimensional.

Counting parameters layer by layer, using Remark 6.1:

$$\underbrace{41 \times 128}_{5248} + \underbrace{129 \times 64}_{8256} + \underbrace{65 \times 32}_{2080} + \underbrace{33 \times 1}_{33} = 15,617,$$

against $40 + 1 = 41$ for the equivalent logistic regression. The network therefore carries $15,617 - 41 = 15,576$ redundant parameters, roughly 381 times the minimum, while representing exactly the same set of functions.

Two qualifications are worth making, because the exercise is not *only* a counting exercise. First, redundant parameterisation is not free at training time: it multiplies memory traffic and gradient computation for no gain in expressiveness. Second, the over-parameterised form is not perfectly equivalent *as an optimisation problem*: the loss surface of a deep linear network is non-convex in θ even though the induced function class is the convex-friendly logistic one, and the factorised parameterisation induces an implicit bias in the trajectory of gradient descent. The correct diagnosis remains that the architecture is a mistake: if a linear model is wanted, fit a linear model; if non-linearity is wanted, insert an activation function.

Solution to Exercise 6.2. Part 1. By Proposition 6.2, $\sigma'(z) = s(1-s)$ with $s = \sigma(z)$. Solving $s(1-s) = 0.01$, that is $s^2 - s + 0.01 = 0$, gives

$$s = \frac{1 \pm \sqrt{1 - 0.04}}{2} = \frac{1 \pm \sqrt{0.96}}{2} \in \{0.010102, 0.989898\}.$$

Since $s \mapsto s(1-s)$ is a downward parabola, $\sigma'(z) < 0.01$ precisely when s lies outside $[0.010102, 0.989898]$. Converting the upper root back to the logit scale,

$$z = \ln \frac{s}{1-s} = \ln \frac{0.989898}{0.010102} = \ln(97.99) = 4.5849,$$

and by the symmetry $\sigma(-z) = 1 - \sigma(z)$ the lower root gives -4.5849 . Hence

$$\sigma'(z) < 0.01 \iff |z| > 4.585 \quad (\text{to three decimal places}).$$

A pre-activation of magnitude only about 4.6 — entirely unremarkable for unstandardised inputs — already suppresses the gradient by a factor of at least 25 relative to its maximum of $1/4$.

Part 2. With $\|\mathbf{W}^{(l)}\| = 1$ and $\gamma = 1/4$, Proposition 6.10 gives the bound $(\gamma w)^{L-1} = 4^{-(L-1)}$. Requiring $4^{-(L-1)} \leq 10^{-6}$,

$$(L-1) \ln 4 \geq 6 \ln 10 \implies L-1 \geq \frac{6 \times 2.302585}{1.386294} = 9.97,$$

so $L-1 = 10$ multiplicative factors suffice, that is a network of $\mathbf{L} = 11$ layers. Ten sigmoid layers are enough to reduce the error signal reaching the first layer by six orders of magnitude *even when the weight matrices are perfectly scaled*. This is the quantitative content of the claim that deep sigmoid stacks do not train, and it is why Section 6.9 treats initialisation and Section 6.10 treats normalisation as necessities rather than refinements.

Solution to Exercise 6.3. Take the convention of Example 6.2, in which $\mathbf{z}^{(1)} = \mathbf{W}^{(1)\top} \mathbf{x} + \mathbf{b}^{(1)}$, and set $\mathbf{x} = (-0.5, 1.2)^\top$:

$$z_1^{(1)} = 1(-0.5) + 0.5(1.2) = 0.1, \quad z_2^{(1)} = -2(-0.5) + 1(1.2) - 0.4 = 1.8,$$

so $\mathbf{z}^{(1)} = (0.1, 1.8)^\top$ and, since both components are positive, $\mathbf{h}^{(1)} = \text{ReLU}(\mathbf{z}^{(1)}) = (0.1, 1.8)^\top$. The output logit is

$$z^{(2)} = 2(0.1) - 1(1.8) - 0.1 = -1.7, \quad \hat{p} = \sigma(-1.7) = \frac{1}{1 + e^{1.7}} = 0.15447.$$

With $y = 1$ the loss is $\mathcal{L} = -\ln(0.15447) = 1.868$, and by Proposition 6.5 the output error signal is

$$\delta^{(2)} = \hat{p} - y = 0.15447 - 1 = -0.84553.$$

It is negative, correctly instructing the network to *increase* $z^{(2)}$ for this borrower.

Applying (6.17) at the output layer,

$$\frac{\partial \mathcal{L}}{\partial \mathbf{w}^{(2)}} = \delta^{(2)} \mathbf{h}^{(1)} = -0.84553 \begin{pmatrix} 0.1 \\ 1.8 \end{pmatrix} = \begin{pmatrix} -0.08455 \\ -1.52196 \end{pmatrix}, \quad \frac{\partial \mathcal{L}}{\partial b^{(2)}} = -0.84553.$$

The second component is eighteen times the first, purely because $h_2^{(1)} = 1.8$ is eighteen times $h_1^{(1)}$: gradient magnitude at a weight is proportional to the activation it multiplies, which is the algebraic reason unscaled activations produce unbalanced updates.

Applying the recursion (6.16) with $\text{ReLU}'(z) = 1$ for $z > 0$,

$$\delta^{(1)} = \left(\mathbf{w}^{(2)} \delta^{(2)} \right) \odot \begin{pmatrix} 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 2(-0.84553) \\ -1(-0.84553) \end{pmatrix} = \begin{pmatrix} -1.69107 \\ 0.84553 \end{pmatrix},$$

and, since $z_k^{(1)} = \sum_j W_{jk}^{(1)} x_j + b_k^{(1)}$ implies $\partial \mathcal{L} / \partial W_{jk}^{(1)} = x_j \delta_k^{(1)}$,

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(1)}} = \mathbf{x} \delta^{(1)\top} = \begin{pmatrix} 0.84553 & -0.42277 \\ -2.02928 & 1.01464 \end{pmatrix}, \quad \frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(1)}} = \begin{pmatrix} -1.69107 \\ 0.84553 \end{pmatrix}.$$

The comparison with Example 6.2. Neither hidden unit fails to update here: both pre-activations are positive, so $\text{ReLU}'(z_k^{(1)}) = 1$ for $k = 1, 2$ and both rows of $\partial\mathcal{L}/\partial\mathbf{W}^{(1)}$ are non-zero. This is the opposite of Example 6.2, where the second unit had $z_2^{(1)} = -1.70 < 0$, its activation derivative was zero, and the Hadamard product in (6.16) annihilated its error signal, leaving its incoming weights untouched by that example.

The instructive point is that the two examples together show ReLU inactivity is a property of the *example*, not of the unit. Unit 2 was silent for the first borrower and highly active for the second, and over a mini-batch it receives gradient from whichever examples excite it. A unit becomes *dead* in the sense of the pitfall in Section 6.3 only when its pre-activation is negative for *every* example in the training set, at which point no batch can revive it.

Solution to Exercise 6.4. Take $\mathcal{L}(y, \hat{p}) = \frac{1}{2}(y - \hat{p})^2$ with $\hat{p} = \sigma(z)$. By the chain rule and Proposition 6.2,

$$\frac{\partial\mathcal{L}}{\partial z} = \frac{\partial\mathcal{L}}{\partial\hat{p}} \cdot \frac{d\hat{p}}{dz} = -(y - \hat{p}) \sigma'(z) = (\hat{p} - y) \hat{p} (1 - \hat{p}).$$

Comparing with $\partial\mathcal{L}/\partial z = \hat{p} - y$ from Proposition 6.5, the squared-error gradient carries the extra factor $\hat{p}(1 - \hat{p}) = \sigma'(z) \leq \frac{1}{4}$.

That factor is harmful in the worst possible place. Consider an example with $y = 1$ that the network confidently misclassifies, $\hat{p} = 0.01$. The residual $\hat{p} - y = -0.99$ is nearly maximal, but the gradient is $-0.99 \times 0.01 \times 0.99 = -0.0098$: the network learns *a hundred times more slowly* from the example it gets most badly wrong. Cross-entropy, by contrast, delivers the full residual -0.99 . The saturating factor makes the learning rate effectively example-dependent in exactly the wrong direction, and it re-introduces the vanishing-gradient mechanism of Proposition 6.10 at the output layer, where Proposition 6.5 had shown it could be eliminated by pairing the loss with the activation correctly. A secondary objection is that the squared error composed with a sigmoid is non-convex in z even for a single linear layer, whereas cross-entropy is convex, so the choice also costs optimisation guarantees in the simplest case.

Solution to Exercise 6.5. By Proposition 6.6, $\text{Var}[\hat{\mathbf{g}}_B] = \text{Var}[\mathbf{g}_i]/B$, so the standard deviation scales as $B^{-1/2}$. Increasing B from 64 to 512 multiplies the batch size by 8 and therefore multiplies the gradient standard deviation by

$$\sqrt{\frac{64}{512}} = \frac{1}{\sqrt{8}} = 0.354,$$

a reduction of about 64.6%.

Is the trade favourable? Under the stated cost model — time strictly proportional to B — it is not. Eight times the computation buys a factor of

$\sqrt{8} \approx 2.83$ in noise. The comparison that matters is against the alternative use of the same budget: eight successive steps at $B = 64$. Those eight steps consume the same computation, and if the parameters move little over the eight steps their averaged gradient also has standard deviation reduced by $\sqrt{8}$ — the same noise reduction, but with eight actual parameter updates instead of one. Small batches therefore dominate at equal compute in this idealised accounting, which is the theoretical reason mini-batch SGD outperforms batch gradient descent per unit of work.

The assumptions under which this holds, and why practice departs from them:

1. *The variance model.* Examples are drawn i.i.d. (with replacement); without replacement a finite-population correction applies, which matters only when B is a non-negligible fraction of n .
2. *Stationarity within the comparison.* The eight-step argument assumes $\text{Var}[\mathbf{g}_i]$ and ∇J are approximately constant over those eight updates. Near a sharp minimum or with a large learning rate this fails.
3. *Strict proportionality of time to B .* This is the assumption that most clearly breaks on real hardware. A GPU processing a batch of 64 is largely idle; a batch of 512 may cost far less than eight times as much wall-clock time, and in that regime the large batch wins outright.
4. *Fixed learning rate.* Larger batches permit a larger stable η , which can convert the noise reduction into faster progress rather than merely quieter steps.

Finally, noise is not purely a cost: [Keskar et al. \(2017\)](#) give evidence that the extra gradient noise of small batches biases the optimiser toward flatter minima that generalise better, so a pure variance-minimisation objective is the wrong target in the first place.

Solution to Exercise 6.6. Apply Proposition 6.7 with $\lambda_{\min} = 0.02$ and $\lambda_{\max} = 8$.

Largest stable learning rate. Convergence holds if and only if $0 < \eta < 2/\lambda_{\max}$, so

$$\eta_{\text{crit}} = \frac{2}{8} = 0.25,$$

and any $\eta \geq 0.25$ diverges along the sharpest curvature direction.

Optimal learning rate.

$$\eta^* = \frac{2}{\lambda_{\min} + \lambda_{\max}} = \frac{2}{8.02} = 0.24938.$$

Note how close η^* lies to η_{crit} : the optimal step is within 0.25% of the divergence boundary. This is characteristic of ill-conditioned problems and

explains why tuning η on a badly conditioned loss feels like balancing on a knife edge — and why standardising inputs, which reduces κ , is worth more than heroic learning-rate search.

Contraction factor. With $\kappa = 8/0.02 = 400$,

$$\rho = \frac{\kappa - 1}{\kappa + 1} = \frac{399}{401} = \mathbf{0.995012}.$$

Iterations for a hundredfold error reduction. Requiring $\rho^t \leq 10^{-2}$,

$$t \geq \frac{\ln 10^{-2}}{\ln \rho} = \frac{-4.60517}{-0.0049999} = 921.03 \implies t = \mathbf{922} \text{ iterations.}$$

For comparison, a perfectly conditioned problem with $\kappa = 1$ has $\rho = 0$ and converges in a single step. The 922 iterations are the entire price of the condition number, and since $\rho \approx 1 - 2/\kappa$ for large κ , the iteration count grows roughly linearly in κ : halving the condition number by rescaling one badly scaled feature halves the training time.

Solution to Exercise 6.7. Part 1. By Proposition 6.11, $\varsigma^2 = 2/n_{\text{in}} = 2/256 = 0.0078125$, so

$$\varsigma = \sqrt{0.0078125} = \frac{1}{\sqrt{128}} = \mathbf{0.08839}.$$

This is what `he_normal` uses: a zero-mean Gaussian with that standard deviation.

Part 2. Repeat the derivation with $\text{LReLU}_\alpha(u) = \max(\alpha u, u)$, that is $x = u$ for $u > 0$ and $x = \alpha u$ for $u \leq 0$. Let u be symmetric about zero with variance τ_{pre}^2 . Splitting the expectation over the two half-lines and using symmetry, which makes each half contribute $\frac{1}{2}\mathbb{E}[u^2]$,

$$\begin{aligned} \mathbb{E}[x^2] &= \mathbb{E}[u^2 \mathbf{1}\{u > 0\}] + \alpha^2 \mathbb{E}[u^2 \mathbf{1}\{u \leq 0\}] \\ &= \frac{1}{2} \tau_{\text{pre}}^2 + \alpha^2 \cdot \frac{1}{2} \tau_{\text{pre}}^2 = \frac{1}{2} (1 + \alpha^2) \tau_{\text{pre}}^2. \end{aligned}$$

Substituting into $\text{Var}(z) = n_{\text{in}} \varsigma^2 \mathbb{E}[x^2]$ from the first part of Proposition 6.11 and requiring $\text{Var}(z) = \tau_{\text{pre}}^2$ for variance preservation across layers,

$$n_{\text{in}} \varsigma^2 \frac{1}{2} (1 + \alpha^2) \tau_{\text{pre}}^2 = \tau_{\text{pre}}^2 \implies \varsigma^2 = \frac{2}{(1 + \alpha^2) n_{\text{in}}},$$

as required. Two sanity checks confirm the formula. At $\alpha = 0$ the activation is ordinary ReLU and the result reduces to $2/n_{\text{in}}$. At $\alpha = 1$ the activation is the identity, no half of the distribution is discarded, and the result becomes $1/n_{\text{in}}$ — the classical variance-preserving scale for a linear layer. For the usual $\alpha = 0.01$ the correction factor is $1/(1 + 10^{-4})$, numerically indistinguishable from He, so in practice `he_normal` is used for leaky ReLU without adjustment.

Solution to Exercise 6.8. Part 1. In Proposition 6.12 the penalty term is $q(1 - q) \sum_j w_j^2 x_j^2$ with $q = 1 - p$ the keep probability. At $p = 0.5$ we have $q = 0.5$ and

$$q(1 - q) = 0.5 \times 0.5 = \mathbf{0.25},$$

so the implied penalty is $0.25 \sum_j w_j^2 x_j^2$, and the point prediction is additionally shrunk deterministically by the factor $q = 0.5$. Averaged over the training set, the effective per-coordinate ridge coefficient is $0.25 \mathbb{E}[x_j^2]$: dropout is a ridge penalty *weighted by each feature's second moment*. Note also that $q(1 - q)$ is maximised at $q = 1/2$, so $p = 0.5$ is the most aggressive drop rate in this sense; both $p \rightarrow 0$ and $p \rightarrow 1$ send the penalty to zero, the latter for the uninteresting reason that the model is destroyed.

Part 2. Suppose feature j is left in raw KES with typical magnitude 10^6 , so $\mathbb{E}[x_j^2] \approx 10^{12}$, while a standardised feature k has $\mathbb{E}[x_k^2] = 1$. The implied penalties are then in the ratio

$$\frac{0.25 \times 10^{12}}{0.25 \times 1} = 10^{12}.$$

Coordinate j is penalised a trillion times more heavily than coordinate k , which drives w_j to essentially zero: the feature is silently deleted from the model by the regulariser, no matter how predictive it is. The reverse case is equally bad — a feature measured in millionths would be effectively unregularised and free to overfit.

This is the same lesson as Section 6.12, arriving from a different direction. There, unstandardised inputs damaged the *conditioning* of the optimisation via Proposition 6.7; here they damage the *regularisation* by making the implied penalty an arbitrary function of the measurement unit. In both cases the defect is that a modelling decision has become a consequence of whether an analyst recorded an amount in shillings or in thousands of shillings. Standardising the inputs makes $\mathbb{E}[x_j^2] \approx 1$ for every j and restores the intended uniform penalty.

Solution to Exercise 6.9. Counting. Of 120,000 agents, 71% appear fewer than five times, that is $0.71 \times 120,000 = 85,200$ low-support agents, leaving 34,800 with support of five or more.

Without collapsing, the table has one row per agent plus one reserved out-of-vocabulary row:

$$(120,000 + 1) \times 16 = \mathbf{1,920,016} \text{ parameters.}$$

Collapsing the 85,200 low-support agents into the reserved index 0 leaves 34,800 individual rows plus that shared row:

$$(34,800 + 1) \times 16 = \mathbf{556,816} \text{ parameters,}$$

a saving of 1,363,200 parameters, or 71% of the table — unsurprisingly the same proportion as the collapsed vocabulary, since the cost is linear in the number of rows.

Which variant to submit, and why. Submit the collapsed variant. The statistical argument is decisive: each low-support agent has 16 free parameters fitted from at most four labelled observations. With a 4% base rate, the overwhelming majority of those agents have *no defaults at all* in the training set, so the fitted embedding encodes the accident of which handful of borrowers happened to use that kiosk rather than any property of the agent. That is memorisation, and it will not replicate on the test fold.

Three further arguments reinforce the choice. It trains the out-of-vocabulary path: index 0 receives gradient from 85,200 agents' worth of examples, so a genuinely unseen agent at inference time is mapped to a meaningful average rather than to an untrained initialisation — the failure mode of the pitfall in Section 6.13. It reduces the parameter count from roughly 400 times the rest of the network to roughly 115 times, which materially reduces the memory footprint of the served artefact. And it is more defensible under the governance expectations of Section 6.15, since a reviewer can be shown the support threshold as an explicit, documented modelling decision.

What evidence would settle it. Fit both variants under an identical protocol — same frozen splits, same preprocessing, same search budget, same early-stopping criterion — and compare, in order: test PR-AUC and expected cost; performance restricted to the *low-support agent subgroup*, which is where the two variants differ and where the aggregate metric may hide the difference; and calibration by county. A useful diagnostic beyond the headline metrics is to plot the norm of each learned embedding row against that agent's training support in the uncollapsed model: if the rare rows have drifted far from their initialisation, the model is fitting noise, and the plot makes that visible to a non-technical reviewer. Finally, the support threshold itself should be treated as a hyperparameter — 5 is a convention, not a result — and swept over, say, $\{3, 5, 10, 25\}$ on the validation fold.

Solution to Exercise 6.10. Proposition 6.14 establishes that $z \mapsto \sigma(z/T)$ is strictly increasing, hence order-preserving. Everything follows from distinguishing quantities that depend on the scores *only through their order* from those that depend on their *values*.

1. **ROC-AUC: unchanged.** It is a functional of the ranking alone (the probability that a random positive outranks a random negative), and the ranking is preserved.
2. **PR-AUC: unchanged.** Precision and recall at each operating point depend only on which examples fall above the cut, and the family of sets obtainable by thresholding is identical before and after scaling.

3. **Brier score: changes.** It is the mean squared difference between predicted probability and outcome, a function of the values themselves. This is precisely the quantity temperature scaling is fitted to improve; with $T = 1.8 > 1$ the logits shrink toward zero and the probabilities toward 0.5, which reduces the Brier score for an over-confident model and would increase it for an under-confident one.
4. **Expected cost at the fixed threshold $\hat{p} = 0.15$: changes.** A fixed threshold on the *probability* scale is not a fixed threshold on the score scale. Before scaling, $\hat{p} = 0.15$ corresponds to the logit $\text{logit}(0.15) = -1.7346$; after scaling, $\sigma(z/1.8) = 0.15$ requires $z = 1.8 \times (-1.7346) = -3.1223$. A different set of applicants is declined, so the confusion matrix and hence the cost move.
5. **Expected cost at the re-derived cost-optimal threshold: unchanged.** Optimising the threshold means minimising cost over the family of decision sets obtainable by thresholding the score. Since the transformation is a strictly increasing bijection, that family is identical before and after, so the minimum over it is identical too. The optimal *threshold value* moves; the optimal *cost* does not.

Remark B.1. The last two items together carry the practical message. Temperature scaling cannot improve the best achievable cost — item 5 — so it is not a performance technique. What it does is make the probability scale trustworthy, so that the analytically derived threshold is the right one. Under the cost matrix of the running case study the indifference point is where $\hat{p} \cdot 10,000 = (1 - \hat{p}) \cdot 800$, that is

$$\hat{p}^* = \frac{800}{10,800} = 0.0741.$$

Applying that threshold to *uncalibrated* scores gives the cost-optimal decision only by luck; applying it after calibration is a principled derivation. That is the entire value proposition: item 5 says calibration costs nothing in ranking, and this remark says it buys correctness in the mapping from scores to decisions.

Chapter 7 — Computer Vision with Convolutional Neural Networks

Solution to Exercise 7.1. Apply Proposition 7.1: (7.2) for shapes and (7.3) for counts. For `padding="same"` with odd f and stride s , the output side is $\lceil n_{\text{in}}/s \rceil$.

Layer	Output shape	Parameters
Input	$224 \times 224 \times 3$	0
Conv2D(64, 7, s=2, same)	$112 \times 112 \times 64$	$7^2 \cdot 3 \cdot 64 + 64 = 9,472$
MaxPooling2D(3, s=2, same)	$56 \times 56 \times 64$	0
Conv2D(128, 3, same)	$56 \times 56 \times 128$	$3^2 \cdot 64 \cdot 128 + 128 = 73,856$
MaxPooling2D(2)	$28 \times 28 \times 128$	0
Conv2D(256, 3, same)	$28 \times 28 \times 256$	$3^2 \cdot 128 \cdot 256 + 256 = 295,168$
MaxPooling2D(2)	$14 \times 14 \times 256$	0
GlobalAveragePooling2D	256	0
Dense(5, softmax)	5	$256 \cdot 5 + 5 = 1,285$
Total		379,781

With a $384 \times 384 \times 3$ input, **no parameter count changes at all**. The spatial dimensions become $192 \rightarrow 96 \rightarrow 96 \rightarrow 48 \rightarrow 48 \rightarrow 24$, and global average pooling still produces a 256-vector, so the dense head is unaffected. This is Proposition 7.1: kernel tensors are indexed by $(f, f, c_{\text{in}}, c_{\text{out}})$ and never by H or W .

What does change is *activation* memory and compute. The first convolution's output tensor grows from $112^2 \cdot 64$ to $192^2 \cdot 64$ entries, a factor of $(192/112)^2 \approx 2.94$; since every layer scales the same way, both peak memory and multiply-accumulate cost rise by roughly $(384/224)^2 \approx 2.94$. The common student error is to expect a bigger model; the correct statement is a bigger *computation* with the same model.

Solution to Exercise 7.2. Use $r_0 = 1, j_0 = 1, r_\ell = r_{\ell-1} + (f_\ell - 1)j_{\ell-1}, j_\ell = j_{\ell-1}s_\ell$.

ℓ	f_ℓ	s_ℓ	r_ℓ	j_ℓ
1	7	2	$1 + 6 \cdot 1 = 7$	2
2	3	2	$7 + 2 \cdot 2 = 11$	4
3	3	1	$11 + 2 \cdot 4 = 19$	4
4	3	1	$19 + 2 \cdot 4 = 27$	4
5	3	2	$27 + 2 \cdot 4 = 35$	8
6	3	1	$35 + 2 \cdot 8 = 51$	8

After layer 6 the jump is $j = 8$, so each further 3×3 stride-one layer adds $(3 - 1) \cdot 8 = 16$ pixels. Starting from 51: 67, 83, 99, 115. Three extra layers give 99, which does *not* exceed 100; **four** extra layers are required, reaching 115.

Two remarks worth making to students. First, receptive field grows in proportion to the accumulated stride, so layers placed after a downsample are far more "efficient" per layer than early ones — this is why architectures front-load striding. Second, the theoretical receptive field is an upper bound on influence, not a description of it: the *effective* receptive field is

approximately Gaussian and considerably smaller, because centre pixels contribute through many more paths than edge pixels.

Solution to Exercise 7.3. Take the one-dimensional input $x = (1, 0, 0, 0, 0)$, kernel $K = (1, 0, 0)$, padding $p = 0$, stride $s = 2$. The admissible left edges are $t = 0, 2$ (and $t = 3$ is excluded since $3 + 2 > 5$), so

$$\mathcal{C}(x) = (x_0, x_2, x_4) = (1, 0, 0).$$

Now shift the input by one position: $x' = (0, 1, 0, 0, 0)$. Then

$$\mathcal{C}(x') = (x'_0, x'_2, x'_4) = (0, 0, 0).$$

The output of the shifted input is the zero vector, which is not any translate of $(1, 0, 0)$: the feature has disappeared entirely rather than moved. This violates equivariance. Contrast a shift of *two* positions, $x'' = (0, 0, 1, 0, 0)$, which gives $\mathcal{C}(x'') = (0, 1, 0)$ — exactly $\mathcal{C}(x)$ translated by $1 = \tau/s$, as Proposition 7.2 predicts.

Practical consequence. A borrower photographing an identity card by hand produces images whose content is shifted by arbitrary amounts, not by multiples of the stride. A strided network's response to the same document therefore varies with sub-stride jitter, and since the accumulated stride of a ResNet-50 backbone is 32, the relevant jitter is not one pixel but up to thirty-one.

The two mitigations named in Section 7.2.2. First, include small random translations in the augmentation policy, so that approximate invariance is learned from data rather than assumed from the architecture; by the argument of Section 7.4.5 this is legitimate precisely because a small shift genuinely does preserve the label. Second, avoid evaluating on a single fixed crop: averaging predictions over a small ensemble of crops averages out the phase of the sampling grid, at a cost linear in the number of crops.

Solution to Exercise 7.4. With $C = 512$, a plain 3×3 convolution costs $9C^2 = 9 \cdot 512^2 = 2,359,296$ weights.

A bottleneck of width $b = 128$ costs

$$\underbrace{1^2 \cdot 512 \cdot 128}_{65,536} + \underbrace{3^2 \cdot 128^2}_{147,456} + \underbrace{1^2 \cdot 128 \cdot 512}_{65,536} = 278,528,$$

so the plain block is $2,359,296 / 278,528 = 8.47$ times more expensive.

Break-even solves $9C^2 = 2Cb + 9b^2$, i.e. $9b^2 + 2Cb - 9C^2 = 0$, whose positive root is

$$b = \frac{-2C + \sqrt{4C^2 + 324C^2}}{18} = \frac{C(\sqrt{82} - 1)}{9} \approx 0.895 C \approx 458.3.$$

ResNet uses $b = C/4 = 128$, far below break-even, and the reason is that parameter parity is the wrong objective. The design goal is to spend a fixed

budget on as much *depth* as possible, because Theorem 7.1 makes depth trainable and Section 7.2.4 shows it to be a better use of a weight budget than width. Choosing b near break-even would buy no depth at all. Choosing b much smaller than $C/4$ would eventually starve the 3×3 layer of capacity; $C/4$ is the empirical compromise, and the $8.5\times$ saving is what funds the 16 blocks of stages 2–5.

Solution to Exercise 7.5. (a) $\alpha\beta^2\gamma^2 = 1.2 \cdot 1.1^2 \cdot 1.15^2 = 1.2 \cdot 1.21 \cdot 1.3225 = 1.9203$, which is ≈ 2 as required.

(b) Using $d = \alpha^\phi$, $w = \beta^\phi$, $r = \gamma^\phi$ and the FLOP multiplier $(\alpha\beta^2\gamma^2)^\phi$ from Proposition 7.4:

ϕ	d	w	r	FLOP multiplier
1	1.200	1.100	1.150	1.920
2	1.440	1.210	1.322	3.687
3	1.728	1.331	1.521	7.081

(c) Solve $1.9203^\phi = 8$:

$$\phi = \frac{\ln 8}{\ln 1.9203} = \frac{2.0794}{0.6526} = 3.187.$$

The implied resolution factor is $\gamma^{3.187} = 1.15^{3.187} = 1.561$, so the input side becomes $224 \times 1.561 \approx 350$ pixels. In practice one would round to a network-friendly value such as 352 (a multiple of 32, the total stride), and note that depth and width scale to $1.20^{3.187} = 1.77$ and $1.10^{3.187} = 1.37$ respectively.

(d) The arithmetic gives a compute budget, not an accuracy guarantee, and the two should not be confused. Compound scaling was calibrated on ImageNet, where 1.2 million labelled images make capacity the binding constraint. On a five-class document task with a few thousand images the binding constraint is data, and Figure 7.14 shows exactly which regime that puts us in: the flat left-hand region where additional capacity buys nothing. A larger ϕ would raise the parameter count, the training time and the deployed size, increase the risk of overfitting, and in all likelihood lower the held-out macro- F_1 . The defensible use of this calculation is the reverse direction: given a fixed serving budget, it tells you which family member you can afford.

Solution to Exercise 7.6. Write $\bar{A}^k = (H'W')^{-1} \sum_{i,j} A_{i,j}^k$ and $y^c = \sum_k w_k^c \bar{A}^k + b^c$. Since $\bar{A}^k$ is linear in the entries of A^k with coefficient $1/(H'W')$ on each, and b^c is a constant,

$$\frac{\partial y^c}{\partial A_{i,j}^k} = w_k^c \cdot \frac{\partial \bar{A}^k}{\partial A_{i,j}^k} = \frac{w_k^c}{H'W'} \quad \text{for every } (i,j).$$

Substituting into the definition of α_k^c ,

$$\alpha_k^c = \frac{1}{H'W'} \sum_{i,j} \frac{w_k^c}{H'W'} = \frac{1}{H'W'} \cdot H'W' \cdot \frac{w_k^c}{H'W'} = \frac{w_k^c}{H'W'}.$$

Hence

$$L^c = \text{ReLU} \left(\sum_k \frac{w_k^c}{H'W'} A^k \right) = \frac{1}{H'W'} \text{ReLU} \left(\sum_k w_k^c A^k \right),$$

using $\text{ReLU}(\lambda u) = \lambda \text{ReLU}(u)$ for $\lambda > 0$. The map is therefore the class activation map up to a positive constant, and since Grad-CAM maps are normalised to $[0, 1]$ before display, the constant is invisible.

The bias. The term b^c contributes nothing, because it does not depend on A^k and so $\partial b^c / \partial A_{i,j}^k = 0$. Intuitively, a class-independent offset shifts every score equally and therefore carries no spatial information.

What breaks with a hidden layer. If the head is

$$y^c = \sum_m v_m^c \text{ReLU} \left(\sum_k u_{mk} \bar{A}^k \right),$$

the derivative becomes $\partial y^c / \partial A_{i,j}^k = (H'W')^{-1} \sum_m v_m^c u_{mk} \mathbb{1}[\text{unit } m \text{ active}]$, which still does not depend on (i, j) but now depends on the *input* through the activation pattern. The proportionality to a fixed weight vector is lost: there is no longer a single set of class weights w_k^c , and α_k^c becomes example-specific. Grad-CAM remains well defined — that is precisely the generalisation it offers — but the identification with CAM, and with it the guarantee that the map reflects a fixed linear read-out, no longer holds.

Solution to Exercise 7.7. (a) By Proposition 7.5,

$$t^* = \frac{C_{\text{FP}}}{C_{\text{FP}} + C_{\text{FN}}} = \frac{800}{800 + 10,000} = \frac{800}{10,800} = 0.0741.$$

Escalate to human review whenever the tamper probability exceeds about 7.4%.

(b) The always-approve policy never raises a false positive and incurs a false negative on every tampered document, so its expected cost per application is

$$\pi C_{\text{FN}} = 0.04 \times 10,000 = \text{KES } 400.$$

For comparison, always-reject costs $(1 - \pi)C_{\text{FP}} = 0.96 \times 800 = \text{KES } 768$. Always-approve is therefore the better of the two trivial policies, and it is the baseline to beat: a model that cannot get below KES 400 per application is not earning its keep.

(c) Accuracy at $t = 0.5$ is misleading for two compounding reasons. It is measured at the wrong threshold: 0.5 is optimal only when $C_{FP} = C_{FN}$, and here the costs differ by a factor of 12.5, so the reported operating point is not one the business would ever choose. And accuracy itself is uninformative under a 4% positive rate, because the always-approve policy achieves 96% — a number that sounds excellent and corresponds to catching no fraud at all.

Solution to Exercise 7.8. *Inference mode* uses the stored running statistics:

$$\hat{z}_R = \frac{1.5 - 0}{1} = 1.5, \quad \tilde{z}_R = 1.2 \times 1.5 - 0.3 = 1.5.$$

Training mode uses the current batch statistics:

$$\hat{z}_B = \frac{1.5 - 0.8}{2.5} = 0.28, \quad \tilde{z}_B = 1.2 \times 0.28 - 0.3 = 0.036.$$

The difference is $1.5 - 0.036 = 1.464$, i.e. the activation that the head was trained to read as 1.5 arrives as 0.036 — a change of over 97% of its value, and a change of sign relative to the shift β .

Why this compounds. ResNet-50 contains 53 batch-normalisation layers. The discrepancy is an affine map of z with slope $\gamma (\sigma_R^{-1} - \sigma_B^{-1})$, so each layer applies its own such distortion and the distortions compose multiplicatively through the stack. Even a modest per-layer factor of, say, 0.7 in effective scale becomes $0.7^{53} \approx 2 \times 10^{-9}$ over the whole backbone. The features reaching the head bear no relation to the features it was trained on in Phase A, which is why validation accuracy does not degrade gracefully but collapses within one or two epochs of unfreezing. Note also that the corruption is *persistent*: the running statistics themselves are overwritten during training, so the damage survives a switch back to inference mode and the checkpoint on disk is already spoiled. This is why the fix must be applied before training, not diagnosed afterwards.

Solution to Exercise 7.9.

Transformation	Cassava	Document	Reason
Horizontal flip	yes	no	A mirrored leaf is still a diseased leaf; mirrored text on an ID is itself evidence of tampering.
$\pm 90^\circ$ rotation	yes	no	Leaves have no canonical orientation; documents do, and a sideways ID is a capture failure the model should flag, not ignore.
$\pm 8^\circ$ rotation	yes	yes	Both are photographed handheld; small tilt is ordinary capture noise in each domain.
Brightness $\pm 20\%$	yes	yes	Field lighting and indoor lighting both vary widely; brightness is not diagnostic of either label.
Random JPEG compression	yes	caution	Realistic for leaves. For documents, compression artefacts can be a re-photography signal, so this risks erasing evidence.
Random 30×30 occlusion	yes	yes	Both benefit: it discourages reliance on a single region and mimics fingers, glare and shadow.

Formal argument. The decomposition in Section 7.4.5 writes the augmented risk as

$$(\bar{f}(\mathbf{x}) - y)^2 + \text{Var}_T[f(T\mathbf{x})],$$

so the objective explicitly rewards a model whose output does *not* vary across G . That is desirable only when the label is genuinely constant on the orbit $\{T\mathbf{x} : T \in G\}$.

Suppose it is not. Take $\mathbf{x}$ a genuine document with $y = 0$, and T the horizontal flip, with $y(T\mathbf{x}) = 1$ because mirrored text indicates tampering. The augmented objective still trains $f(T\mathbf{x})$ towards $y(\mathbf{x}) = 0$, so the pair $(T\mathbf{x}, 0)$ enters the training set as a *mislabelled* example. Worse, the variance term actively penalises the model for distinguishing $\mathbf{x}$ from $T\mathbf{x}$ — it is not merely noise but a regulariser pulling towards the wrong invariance. The Bayes-optimal classifier for the augmented distribution is therefore strictly worse than the Bayes-optimal classifier for the true one, and no amount of additional data will recover the gap.

Solution to Exercise 7.10. (a) Storage is $P \times b/8$ bytes.

Model	Parameters	float32	float16	int8
EfficientNetV2-B0	7.2M	28.8 MB	14.4 MB	7.2 MB
ResNet50V2	25.6M	102.4 MB	51.2 MB	25.6 MB

(b) Against a 10 MB cap, only **EfficientNetV2-B0 in int8** (7.2 MB) qualifies. B0 in float16 is 14.4 MB and still fails; ResNet50V2 fails at every precision, needing better than $3\times$ compression beyond int8 to fit — achievable

only with pruning or distillation, which is a different project. This is the concrete sense in which backbone choice is a product decision rather than an accuracy decision: on this requirement the ResNet is excluded before its accuracy is even measured.

(c) For values in $[-3, 3]$ quantised symmetrically to 8 bits, the step is

$$\Delta = \frac{2r}{255} = \frac{6}{255} = 0.02353,$$

and the maximum rounding error is $\Delta/2 = 0.01176$, i.e. about 0.2% of the full range.

Whether that matters is best judged against the noise already in the system. The cassava dataset carries substantial label noise — disease grades assigned from photographs by human annotators disagree at rates in the several-percent range — and the model’s own uncertainty is far larger than a 0.2% perturbation of individual weights. Empirically, int8 post-training quantisation with a representative dataset typically costs well under one point of accuracy on this class of task. The honest framing for a design review is therefore: the quantisation error is roughly an order of magnitude smaller than the label noise, so it should not be the first thing to blame if accuracy disappoints. The caveats worth stating are that per-tensor quantisation is riskier than per-channel for layers whose weight ranges vary widely across channels, and that the effect must be *measured* on the held-out set rather than assumed — a one-line comparison of the float and TFLite models that belongs in every deployment notebook.

Bibliography

- Akiba, T., Sano, S., Yanase, T., Ohta, T., and Koyama, M. (2019). Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 2623–2631.
- Arthur, D. and Vassilvitskii, S. (2007). *k*-means++: The advantages of careful seeding. In *Proceedings of the 18th Annual ACM-SIAM Symposium on Discrete Algorithms*, pages 1027–1035.
- Ba, J. L., Kiros, J. R., and Hinton, G. E. (2016). Layer normalization. *arXiv preprint arXiv:1607.06450*.
- Baydin, A. G., Pearlmutter, B. A., Radul, A. A., and Siskind, J. M. (2018). Automatic differentiation in machine learning: A survey. *Journal of Machine Learning Research*, 18(153):1–43.
- Bergstra, J., Bardenet, R., Bengio, Y., and Kégl, B. (2011). Algorithms for hyper-parameter optimization. In *Advances in Neural Information Processing Systems*, volume 24.
- Bergstra, J. and Bengio, Y. (2012). Random search for hyper-parameter optimization. *Journal of Machine Learning Research*, 13:281–305.
- Beyer, K., Goldstein, J., Ramakrishnan, R., and Shaft, U. (1999). When is “nearest neighbor” meaningful? In *International Conference on Database Theory (ICDT)*, pages 217–235.
- Bishop, C. M. and Bishop, H. (2024). *Deep Learning: Foundations and Concepts*. Springer.
- Bjorkegren, D. and Grissen, D. (2020). Behavior revealed in mobile phone usage prediction credit repayment. In *The World Bank Economic Review*, volume 34.
- Box, G. E. P. and Cox, D. R. (1964). An analysis of transformations. *Journal of the Royal Statistical Society: Series B*, 26(2):211–243.

- Breiman, L. (1996). Bagging predictors. *Machine Learning*, 24(2):123–140.
- Breiman, L. (2001a). Random forests. *Machine Learning*, 45(1):5–32.
- Breiman, L. (2001b). Statistical modeling: The two cultures. *Statistical Science*, 16(3):199–231.
- Breiman, L., Friedman, J. H., Olshen, R. A., and Stone, C. J. (1984). *Classification and Regression Trees*. Wadsworth, Belmont, CA.
- Brier, G. W. (1950). Verification of forecasts expressed in terms of probability. *Monthly Weather Review*, 78(1):1–3.
- Buolamwini, J. and Gebru, T. (2018). Gender shades: Intersectional accuracy disparities in commercial gender classification. In *Proceedings of the 1st Conference on Fairness, Accountability and Transparency (FAccT)*, pages 77–91.
- Campello, R. J. G. B., Moulavi, D., and Sander, J. (2013). Density-based clustering based on hierarchical density estimates. In *Advances in Knowledge Discovery and Data Mining (PAKDD)*, pages 160–172.
- Central Bank of Kenya (2022). Central bank of kenya (digital credit providers) regulations, 2022.
- Chawla, N. V., Bowyer, K. W., Hall, L., and Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16:321–357.
- Chen, T. and Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proc. 22nd ACM SIGKDD*, pages 785–794.
- Cybenko, G. (1989). Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals and Systems*, 2(4):303–314.
- Davis, J. and Goadrich, M. (2006). The relationship between precision-recall and ROC curves. In *Proceedings of the 23rd International Conference on Machine Learning (ICML)*, pages 233–240.
- Dempster, A. P., Laird, N. M., and Rubin, D. B. (1977). Maximum likelihood from incomplete data via the EM algorithm. *Journal of the Royal Statistical Society, Series B*, 39(1):1–38.
- Dietterich, T. G. (1998). Approximate statistical tests for comparing supervised classification learning algorithms. *Neural Computation*, 10(7):1895–1923.
- Elkan, C. (2001). The foundations of cost-sensitive learning. In *Proceedings of the 17th International Joint Conference Artificial Intelligence (IJCAI)*, pages 973–978.

- Ester, M., Kriegel, H.-P., Sander, J., and Xu, X. (1996). A density-based algorithm for discovering clusters in large spatial databases with noise. In *Proceedings of the 2nd International Conference on Knowledge Discovery and Data Mining (KDD)*, pages 226–231.
- Fawcett, T. (2006). An introduction to ROC analysis. *Pattern Recognition Letters*, 27(8):861–874.
- Freund, Y. and Schapire, R. E. (1997). A decision-theoretic generalization of on-line learning and an application to boosting. *Journal of Computer and System Sciences*, 55(1):119–139.
- Friedman, J. H. (2001). Greedy function approximation: a gradient boosting machine. *Annals of Statistics*, 29(5):1189–1232.
- Géron, A. (2022). *Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow*. O'Reilly Media, 3rd edition.
- Glorot, X. and Bengio, Y. (2010). Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the 13th International Conference on Artificial Intelligence and Statistics (AISTATS)*, pages 249–256.
- Grinsztajn, L., Oyallon, E., and Varoquaux, G. (2022a). Why do tree-based models still outperform deep learning on typical tabular data? In *Advances in Neural Information Processing Systems*, volume 35.
- Grinsztajn, L., Oyallon, E., and Varoquaux, G. (2022b). Why do tree-based models still outperform deep learning on typical tabular data? In *Advances in Neural Information Processing Systems 35 (NeurIPS), Datasets and Benchmarks Track*.
- Guo, C. and Berkhahn, F. (2016). Entity embeddings of categorical variables. *arXiv preprint arXiv:1604.06737*.
- Guo, C., Pleiss, G., Sun, Y., and Weinberger, K. Q. (2017). On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, pages 1321–1330.
- Halko, N., Martinsson, P.-G., and Tropp, J. A. (2011). Finding structure with randomness: Probabilistic algorithms for constructing approximate matrix decompositions. *SIAM Review*, 53(2):217–288.
- He, K., Zhang, X., Ren, S., and Sun, J. (2015). Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 1026–1034.

- He, K., Zhang, X., Ren, S., and Sun, J. (2016). Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778.
- Hornik, K. (1991). Approximation capabilities of multilayer feedforward networks. *Neural Networks*, 4(2):251–257.
- Hotelling, H. (1933). Analysis of a complex of statistical variables into principal components. *Journal of Educational Psychology*, 24(6):417–441.
- Huyen, C. (2022). *Designing Machine Learning Systems*. O’Reilly Media.
- Ioffe, S. and Szegedy, C. (2015). Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, pages 448–456.
- Kaufman, S., Rosset, S., Perlich, C., and Stitelman, O. (2012). Leakage in data mining: Formulation, detection, and avoidance. *ACM TKDD*, 6(4).
- Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., and Liu, T.-Y. (2017). LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems*, volume 30.
- Keskar, N. S., Mudigere, D., Nocedal, J., Smelyanskiy, M., and Tang, P. T. P. (2017). On large-batch training for deep learning: Generalization gap and sharp minima. In *Proceedings of the 5th International Conference on Learning Representations (ICLR)*.
- Kingma, D. P. and Ba, J. (2015). Adam: A method for stochastic optimization. In *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*.
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S., and Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33.
- Li, L., Jamieson, K., DeSalvo, G., Rostamizadeh, A., and Talwalkar, A. (2017). Hyperband: A novel bandit-based approach to hyperparameter optimization. *Journal of Machine Learning Research*, 18(185):1–52.
- Lin, T.-Y., Goyal, P., Girshick, R., He, K., and Dollár, P. (2017). Focal loss for dense object detection. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 2980–2988.
- Liu, F. T., Ting, K. M., and Zhou, Z.-H. (2008). Isolation forest. In *Eighth IEEE International Conference on Data Mining*, pages 413–422.

- Lloyd, S. P. (1982). Least squares quantization in PCM. *IEEE Transactions on Information Theory*, 28(2):129–137.
- Loshchilov, I. and Hutter, F. (2019). Decoupled weight decay regularization. In *Proceedings of the 7th International Conference on Learning Representations (ICLR)*.
- Lundberg, S. M. and Lee, S.-I. (2017). A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems*, volume 30.
- McInnes, L., Healy, J., and Melville, J. (2018). UMAP: Uniform manifold approximation and projection for dimension reduction. *arXiv preprint arXiv:1802.03426*.
- Melis, G., Dyer, C., and Blunsom, P. (2018). On the state of the art of evaluation in neural language models. In *International Conference on Learning Representations*.
- Micci-Barreca, D. (2001). A preprocessing scheme for high-cardinality categorical attributes in classification and prediction problems. *ACM SIGKDD Explorations Newsletter*, 3(1):27–32.
- Niculescu-Mizil, A. and Caruana, R. (2005). Predicting good probabilities with supervised learning. In *Proceedings of the 22nd International Conference on Machine Learning (ICML)*, pages 625–632.
- Pearson, K. (1901). On lines and planes of closest fit to systems of points in space. *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, 2(11):559–572.
- Platt, J. C. (1999). Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. In *Advances in Large Margin Classifiers*, pages 61–74. MIT Press.
- Prince, S. J. D. (2024). *Understanding Deep Learning*. MIT Press.
- Prokhorenkova, L., Gusev, G., Vorobev, A., Dorogush, A. V., and Gulin, A. (2018). CatBoost: unbiased boosting with categorical features. In *Advances in Neural Information Processing Systems*, volume 31.
- Republic of Kenya (2019). The data protection act, no. 24 of 2019. Government Printer, Nairobi.
- Rosenblatt, F. (1958). The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6):386–408.

- Rousseeuw, P. J. (1987). Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics*, 20:53–65.
- Rubin, D. B. (1976). Inference and missing data. *Biometrika*, 63(3):581–592.
- Schölkopf, B., Smola, A., and Müller, K.-R. (1998). Nonlinear component analysis as a kernel eigenvalue problem. *Neural Computation*, 10(5):1299–1319.
- Schwarz, G. (1978). Estimating the dimension of a model. *The Annals of Statistics*, 6(2):461–464.
- Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J.-F., and Dennison, D. (2015). Hidden technical debt in machine learning systems. In *Advances in Neural Information Processing Systems* 28.
- Selvaraju, R. R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., and Batra, D. (2017). Grad-cam: Visual explanations from deep networks via gradient-based localization. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*.
- Siddiqi, N. (2017). *Intelligent Credit Scoring*. Wiley, 2nd edition.
- Snoek, J., Larochelle, H., and Adams, R. P. (2012). Practical Bayesian optimization of machine learning algorithms. In *Advances in Neural Information Processing Systems*, volume 25.
- Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., and Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(56):1929–1958.
- Sundararajan, M., Taly, A., and Yan, Q. (2017). Axiomatic attribution for deep networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, pages 3319–3328.
- Tan, M. and Le, Q. V. (2019). Efficientnet: Rethinking model scaling for convolutional neural networks. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*.
- Tan, M. and Le, Q. V. (2021). Efficientnetv2: Smaller models and faster training. In *Proceedings of the 38th International Conference on Machine Learning (ICML)*.
- van der Maaten, L. and Hinton, G. (2008). Visualizing data using *t*-SNE. *Journal of Machine Learning Research*, 9:2579–2605.

- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30.
- Wattenberg, M., Viégas, F., and Johnson, I. (2016). How to use *t*-SNE effectively. *Distill*.
- Wolpert, D. H. (1992). Stacked generalization. *Neural Networks*, 5(2):241–259.
- Yeo, I.-K. and Johnson, R. A. (2000). A new family of power transformations to improve normality or symmetry. *Biometrika*, 87(4):954–959.
- Yosinski, J., Clune, J., Bengio, Y., and Lipson, H. (2014). How transferable are features in deep neural networks? In *Advances in Neural Information Processing Systems (NeurIPS)*.